GnssProcessor: Mathematical Reference

Revision date: June 25, 2026

Contents

1 Introduction **4**

I Foundations **4**

2 Notation and conventions **4**

3 State vector layout **6**
3.1 Schematic layout . . . 6
3.2 Component-by-component description . . . 7
3.3 Dynamic indexing . . . 9

4 Observation equations **10**
4.1 Code (pseudorange) observation . . . 10
4.2 Carrier-phase observation . . . 10
4.3 Geometric range and linearisation . . . 10
4.4 Stochastic model . . . 11

5 Differencing transformations **11**
5.1 Between-satellite single difference (BSSD) . . . 11
5.2 Between-receiver single difference (BRSD) . . . 12
5.3 Double difference (DD) . . . 12
5.4 S-transform structure . . . 12

II Estimation engine **13**

6 Estimation engine: the Kalman filter and RTS smoother **13**
6.1 Time update . . . 14
6.2 Measurement update . . . 15
6.3 Iterated relinearisation (IEKF) . . . 16
6.4 Innovation and residual gating . . . 17
6.5 Backward RTS smoother . . . 18

III Cycle-slip detection and repair **19**

7 Cycle-slip detection and repair **19**
7.1 Detector registry and composition . . . 20
7.2 Test-statistic combinations . . . 20
7.2.1 Melbourne–Wübbena (wide-lane) . . . 20
7.2.2 Extra-wide-lane (EWL) . . . 21
7.2.3 GRAPHIC (single-frequency half-sum) . . . 21
7.2.4 Geometry-free phase (staged) . . . 22
7.2.5 Ionosphere-free phase (staged) . . . 22
7.3 TurboEdit dual-frequency resolution . . . 23
7.3.1 The 2×2 system . . . 23
7.3.2 Integer rounding and validation . . . 24
7.3.3 MW-only / code-event null line . . . 24
7.3.4 Gap-spanning GF bridge . . . 25
7.3.5 Atomic reconciliation . . . 26

IV Integer ambiguity resolution **26**

8 Integer ambiguity resolution **26**
8.1 The integer least-squares problem . . . 26
8.2 Decorrelation: LDL^{T} factorisation and the integer $\boldsymbol{Z}$-transform . . . 27
8.3 Wide-lane / narrow-lane decomposition (PPP path) . . . 28
8.4 Partial / subset ambiguity resolution . . . 29
8.5 Validation: ratio test and fixed-failure-rate gate . . . 30
8.6 Feedback: constrained pseudo-observation update . . . 31
8.7 Configuration summary . . . 32

V Inertial aiding **32**

9 Inertial aiding **32**
9.1 Strapdown mechanization . . . 33
9.2 INS error-state dynamics . . . 34
9.3 Covariance splice and attitude feedback . . . 34
9.4 Coupling modes and the GNSS update . . . 35

VI Real-time SSR corrections **35**

10 Real-time SSR corrections **36**
10.1 Orbit correction — refines ρ_r^i . . . 36
10.2 Clock correction — refines $-c\,\mathrm{d}t^i$ . . . 37
10.3 IOD gate at correction-set boundaries . . . 37
10.4 Satellite biases — refine $-B_s^i$ and $-b_s^i$ . . . 37
10.5 Satellite-bias STATE gate vs. SSR code biases . . . 38
10.6 GLONASS . . . 39
10.7 Ionosphere — $\mu_s I_{r,1}^i$ is not SSR-fed . . . 39
10.8 Provenance and code pointers . . . 40

VII Real-time engine **40**

11 Real-time engine **40**
11.1 Streaming control flow . . . 41
11.2 NTRIP transport . . . 41
11.3 RTCM 3 framing and CRC . . . 42
11.4 MSM observation decode . . . 42
11.5 Broadcast nav, SSR and station coordinates . . . 43
11.6 Time-system handling . . . 44
11.7 Observation/SSR synchronisation . . . 44
11.8 Ephemeris management . . . 45
11.9 A-priori seeding from RTCM 1005/1006 . . . 45
11.10 The per-epoch real-time update . . . 45

VIII Network mode **47**

12 Network mode **47**
12.1 Joint multi-station observation model . . . 47
12.2 The explicit satellite phase-bias state . . . 48
12.3 Rank deficiency and the minimum-constraint S-basis . . . 48
12.4 Per-station Schur reduction (arrowhead normals) . . . 49
12.5 Constellation scope . . . 50
12.6 The closed loop: producer → held-out user . . . 51

13 Worked example: estimability in a two-receiver network **51**
13.1 Code only: clocks and one gauge freedom . . . 51
13.2 Adding carrier phase: the satellite phase bias needs the network . . . 53

IX Reference **54**

14 Pipeline and architecture diagrams **54**

A Generated reference tables **54**
A.1 Processing modes . . . 54
A.2 Execution mode and data source . . . 60
A.3 Differencing regime . . . 60
A.4 Config-to-differencing mapping . . . 61
A.5 Feature toggles . . . 61
A.6 Model-correction switches . . . 63
A.7 Logging and trace channels . . . 65
A.8 Cycle-slip detector registry . . . 67
A.9 State-vector key schema . . . 67

1 Introduction

This document is the mathematical companion to `GnssProcessor`, a C++17 post-processing and real-time GNSS engine implementing PPP, PPP-RTK (single-receiver and network), and double-differenced baseline modes. Its scope is the estimation mathematics and design rationale — the material that ages slowly because it is settled theory or algebra, not churning code.

The reference is organised as a set of *component documents*, each of which also compiles to a standalone PDF. This combined volume gathers them in roughly the order the engine applies them:

- **Foundations** (Sections 2 to 5) — notation, the variable-dimension state vector, the *uncombined* pseudorange / carrier-phase observation model, and the between-satellite single-difference (BSSD) and double-difference (DD) transformations. Every other component builds on this.
- **Estimation engine** (Section 6) — the variable-dimension Kalman filter (Joseph-form update, prior-anchored iterated relinearisation, robust gating) and the backward RTS smoother.
- **Cycle-slip detection and repair** (Section 7) — the linear-combination test statistics (Melbourne–Wübbena, geometry-free, GRAPHIC, extra-wide-lane) and the TurboEdit 2×2 integer repair.
- **Integer ambiguity resolution** (Section 8) — the MLAMBDA integer least-squares search, wide-lane / narrow-lane decomposition, validation, and constrained feedback.
- **Inertial aiding** (Section 9) — the strapdown mechanisation and 15-state INS error model of the tightly-coupled IMU path.
- **Real-time SSR corrections** (Section 10) — how State Space Representation orbit, clock, and bias corrections refine the undifferenced observation model.
- **Real-time engine** (Section 11) — the streaming architecture: NTRIP transport, RTCM/MSM decoding, observation/SSR time synchronisation, and the per-epoch real-time update path.
- **Network mode** (Section 12) — the multi-station PPP-RTK network estimator that *generates* the SSR-style corrections consumed by the single-receiver modes: the joint observation model, the S-basis / minimum-constraint datum, and the per-station Schur reduction.

Implementation details (function signatures, class responsibilities, config field reference) live in the headers under `include/` and are rendered via Doxygen. When this document references a specific algorithmic component, it cites the source file directly (e.g. `src/core/filter/KalmanFilter.cpp`).

The notation follows the IGS conventions adopted by Subirana, Zornoza, and Hernández-Pajares [10] and Teunissen and Montenbruck [13]; see Section 2 for the symbol table this document depends on.

Part I
Foundations

2 Notation and conventions

Indices and sets. We use lower-case Latin letters for indices and calligraphic upper-case for sets:

- $r \in \mathcal{R}$ — receiver. Single-receiver modes (PPP, PPP-RTK) have $|\mathcal{R}| = 1$; baseline modes have a fixed reference and one or more rovers.
- $g \in \mathcal{G} = \{\text{GPS}, \text{GAL}, \text{BDS}, \text{GLO}, \text{QZS}, \dots\}$ — constellation.
- $i \in \mathcal{S}_g$ — satellite of constellation g. p_g denotes the pivot satellite of constellation g under the BSSD parameterisation (Section 5).
- $s \in \mathcal{B}$ — signal/observation type, identified by its RINEX-3 three-character code (e.g. `C1C`, `L2W`, `L5X`).
- $k \in \mathbb{N}$ — discrete epoch index.

Vectors and matrices. Bold lower-case for vectors ($\boldsymbol{x}$, $\boldsymbol{r}$); bold upper-case for matrices ($\boldsymbol{H}$, $\boldsymbol{P}$). $\boldsymbol{a}^{\mathrm{T}}$ is the transpose. A hat denotes a state estimate; a tilde a one-step prediction: $\hat{\boldsymbol{x}}_{k|k}$ vs. $\tilde{\boldsymbol{x}}_{k|k-1}$.

Observables. For each receiver/satellite/signal triple, the engine consumes two real-valued observables (in metres) per epoch:

- $P^i_{r,s}$ — pseudorange (code), in metres.
- $\Phi^i_{r,s}$ — carrier phase, scaled to metres via λ_s. The accumulated cycle count $\Phi^i_{r,s} \cdot \lambda_s^{-1}$ is the integer ambiguity plus a fractional bias.

Where the distinction matters, s may further denote the band ($L_1, L_2, L_5, \dots$) and the tracking attribute (e.g. `C`, `W`, `X`).

Frequencies and wavelengths. f_s is the carrier frequency (Hz) and $\lambda_s = c/f_s$ the wavelength (m). The frequency-squared scaling factor used to project the L1-equivalent slant ionospheric delay onto signal s is

$$\mu_s \;:=\; \frac{f_1^2}{f_s^2}. \tag{1}$$

Differencing operators. Three differencing operations recur throughout the document:

$$\Delta q^{i,p_g}_{r,s} := q^i_{r,s} - q^{p_g}_{r,s}, \quad \nabla q^i_{r_1r_2,s} := q^i_{r_1,s} - q^i_{r_2,s}, \quad \nabla\Delta q^{i,p_g}_{r_1r_2,s} := \Delta\big(\nabla q^i_{r_1r_2,s}\big).$$

Read Δq as "between-satellite single difference of q relative to pivot p_g," ∇q as "between-receiver single difference," and $\nabla\Delta q$ as "double difference."

Sign conventions. We adopt the convention that the slant ionospheric delay $I^i_{r,s}$ enters *additively* on the code observable and *subtractively* on the carrier phase observable; this is the source of the well-known "code–phase divergence." All other range-additive terms (geometry, clocks, troposphere, biases) carry the same sign on both observables. See Equations (4) and (5) for the explicit equations.

Symbol table.

Symbol	Units	Meaning
$\boldsymbol{r}_r, \boldsymbol{r}^i$	m	ECEF position of receiver/satellite
ρ_r^i	m	geometric range $\|\boldsymbol{r}^i - \boldsymbol{r}_r\|$
$\boldsymbol{e}_r^i$	—	unit LOS from r to i in ECEF
$c\,\mathrm{d}t_r$	m	receiver clock offset (scaled by c)
$c\,\mathrm{d}t^i$	m	satellite clock offset (from CLK product)
$c\,\delta t_r^{(g)}$	m	receiver inter-system time offset for constellation $g \neq g_{\mathrm{ref}}$
T_r^i	m	total tropospheric path delay
$\mathrm{ZHD}_r^i, \mathrm{ZWD}_r^i$	m	zenith hydrostatic / wet components
$I_{r,1}^i$	m	L1-equivalent slant ionospheric delay
$N_{r,s}^i$	cyc	integer carrier-phase ambiguity
$b_{r,s}, b_s^i$	m	receiver / satellite phase biases
$\bar{b}_s^i$	m	estimable satellite phase bias (network mode; Section 3)
$B_{r,s}, B_s^i$	m	receiver / satellite code biases (OSB)
$B_{r,k}^{(g,b)}$	m	per-FCN-slot GLONASS receiver IFB, slot k, band b
$\varepsilon^P, \varepsilon^\Phi$	m	residual (multipath + thermal)
$\boldsymbol{v}_r$	m/s	receiver velocity (optional / IMU state)
$\lambda_{\mathrm{WL}},\ \lambda_{\mathrm{NL}}$	m	wide- / narrow-lane wavelength $c/(f_1 \mp f_2)$
$\boldsymbol{C}_b^e$	—	body-to-ECEF attitude (direction-cosine) matrix

3 State vector layout

The Kalman filter operates on a state vector $\boldsymbol{x}_k \in \mathbb{R}^{n_k}$ whose dimension n_k *varies across epochs*: per-satellite and per-(satellite, signal) substates are added when a new sat/signal is first observed and pruned (or marked stale) when the satellite drops below the elevation mask or its phase track is interrupted by a cycle slip without recovery. All bookkeeping — key→index maps, free-list re-use, and dimensional reshaping of $\boldsymbol{x}$ and $\boldsymbol{P}$ — is encapsulated in `StateManager`; `KalmanFilter` consumes the state through that interface and never indexes raw memory by hard-coded offsets.

3.1 Schematic layout

In schematic form, suppressing the epoch index k:

$$\begin{aligned}\boldsymbol{x} \;=\; \big[\; & \underbrace{\boldsymbol{r}_r^{\mathrm{T}}}_{3},\ \underbrace{\boldsymbol{v}_r^{\mathrm{T}}}_{0\text{ or }3},\ \underbrace{c\,\mathrm{d}t_r}_{1},\ \underbrace{(c\,\delta t_r^{(g)})_{g\neq g_{\mathrm{ref}}}^{\mathrm{T}}}_{|\mathcal{G}|-1},\\ & \underbrace{\mathrm{ZWD}_r,\ (g_N, g_E)}_{1+0\text{ or }1+2},\ \underbrace{(c\,\mathrm{d}t_r^{\phi})}_{0\text{ or }1},\\ & \underbrace{(I_{r,1}^i)_{i\in\mathcal{V}_k}^{\mathrm{T}}}_{|\mathcal{V}_k|},\ \underbrace{(N_{r,s}^i)_{(i,s)\in\mathcal{A}_k}^{\mathrm{T}}}_{|\mathcal{A}_k|},\\ & \underbrace{(B_{r,s})_{s\in\mathcal{B}}^{\mathrm{T}}}_{|\mathcal{B}|},\ \underbrace{(B_s^i)_{(i,s)\in\mathcal{D}_k}^{\mathrm{T}}}_{|\mathcal{D}_k|}\;\big]^{\mathrm{T}}. \end{aligned} \tag{2}$$

where

- $\mathcal{V}_k$ is the set of satellites currently visible at epoch k (i.e. above the elevation mask, with valid orbit/clock products and at least one tracked signal),
- $\mathcal{A}_k \subseteq \mathcal{V}_k \times \mathcal{B}$ is the set of phase-tracked (satellite, signal) pairs,
- $\mathcal{D}_k \subseteq \mathcal{V}_k \times \mathcal{B}$ is the subset of (satellite, signal) pairs for which the BIA product lacks a satellite OSB and a state-space estimate is therefore created (see `estimation.estimate_sat_biases`).

For brevity the schematic suppresses three conditional refinements detailed below: the rate states that accompany velocity (a receiver clock-rate and one inter-system clock-rate offset per non-reference constellation), the nine-state inertial augmentation of Section 9, and the fact that the entire receiver-side clock and inter-system-offset block is removed under the BSSD parameterisation (Section 5.1).

3.2 Component-by-component description

Position $\boldsymbol{r}_r \in \mathbb{R}^3$. ECEF Cartesian receiver position. Always present. Initialised either from the RINEX header approximate position or, after the first epoch, from a Weighted Least-Squares (WLS) seed solution to within the linearisation radius of the geometric range model (Section 4.3).

Velocity $\boldsymbol{v}_r \in \mathbb{R}^3$. Optional. Enabled by `estimation.estimate_velocity = true`, or forced on by tightly-coupled IMU integration. When present, position and velocity form a coupled continuous-white-noise-acceleration (CWNA, integrated-random-walk) block driven by a *single* acceleration spectral density `process_noise.acceleration_psd` (Equation (20)); the scalar `position_q` of the velocity-free case is then unused. Enabling velocity also introduces a receiver clock-rate state $c\,\dot{\mathrm{d}}t_r$, and one inter-system clock-rate offset per non-reference constellation, each CWNA-coupled to its bias counterpart (`clock_rate_q`, `system_time_rate_offset_q`).

Receiver clock $c\,\mathrm{d}t_r$. Single state, in metres. Process noise is large (`clock_bias_q` $\sim 10^3\,\mathrm{m}^2$ per second) so the state is effectively re-estimated each epoch; this captures oscillator drift without imposing a stochastic model that may not match the receiver's actual oscillator. Under the BSSD (between-satellite single-difference) parameterisation (Section 5.1) the receiver clock is unobservable and the state is dropped entirely — as are the inter-system offsets below — so the whole receiver-side clock block vanishes in PPP-RTK SD mode.

Inter-system time offsets $c\,\delta t_r^{(g)}$. One per active constellation other than the chosen reference (default GPS). These absorb the per-constellation clock biases that arise from mixing analysis-centre-defined clock products across systems ("GAL − GPS time offset," etc.).

Tropospheric ZWD and gradients. ZWD_r models the zenith wet delay; the zenith hydrostatic component ZHD_r is treated deterministically via the Saastamoinen model [9]. Total slant tropospheric delay is then

$$T_r^i \;=\; m_h(\theta_r^i)\,\mathrm{ZHD}_r \;+\; m_w(\theta_r^i)\,\mathrm{ZWD}_r \;+\; m_w(\theta_r^i)\cot\theta_r^i\big[g_N\cos\alpha_r^i + g_E\sin\alpha_r^i\big], \tag{3}$$

with elevation θ_r^i, azimuth α_r^i, and the Niell or VMF mapping functions m_h, m_w. Gradients (g_N, g_E) are optional.

Per-satellite slant ionosphere $I^i_{r,1}$. PPP-RTK-only. One state per visible satellite, expressed at the L1 frequency; signal-specific delays follow Equation (1). Initial value seeded from the Klobuchar broadcast model when no external ionospheric product is available. Its process noise is a random walk whose zenith spectral density `ionosphere_q` is projected onto the slant by the squared obliquity factor $1/\sin^2\theta^i_r$ (Equation (19)), letting low-elevation states vary faster. Modelling these per-sat states explicitly is what permits PPP-RTK to retain integer ambiguity recovery without forming the ionosphere-free combination (which amplifies code noise by a factor of ~ 3).

Carrier-phase ambiguities $N^i_{r,s}$. One state per phase-tracked (i, s) pair, in cycles. Treated as float during the prediction/update cycle and resolved to integers post-hoc by the LAMBDA-family algorithms in `filter/AmbiguityResolver` [12] when AR is enabled. Cycle slips trigger a re-seed via `StateManager::resetAmbiguity`, which inflates the diagonal covariance entry while preserving the cross-state correlations that permit a consistent re-fix.

Receiver code biases $B_{r,s}$. These absorb the receiver-side inter-frequency biases that the OSB convention does not reference to zero. They are *not* one per signal: `StateInitializer::resolveReceiverBias` keys them per (system, frequency band) as inter-frequency biases (`"<system>_IFB<band>"`) plus, in the alternative scheme, a single per-system differential code bias (`"<system>_DCB"`). Like the receiver clock they are receiver-common and so are suppressed under BSSD.

Per-slot GLONASS inter-frequency bias $B^{(g,b)}_{r,k}$. Optional, off by default. GLONASS FDMA assigns each slot $k \in [-7, +6]$ a distinct carrier frequency $f = f_{\text{base}} + k\,\Delta f$, so the receiver group delay differs *per slot* and a single per-band IFB state cannot represent it. When `estimation.glonass_ifb.enable = true` (and the prerequisite `estimation.estimate_ifb`; the loader warns and the flag is inert otherwise), `resolveReceiverBias` instead emits one state per (FDMA band $b \in \{\text{G1}, \text{G2}\}$, slot k) with unit measurement coefficient, keyed `"GLONASS_IFB<band>_k<±slot>"`. The central channel $k = 0$ is the held datum and produces no state; its constant is absorbed by the GLONASS inter-system time offset. CDMA bands (G3/G4/G6) have a slot-independent frequency and retain the ordinary per-band treatment above. Crucially, unlike the receiver-common biases these columns *survive* between-satellite single differencing — two GLONASS satellites on different slots do not cancel — so `BssdTransform` and `StateManager` special-case them (via `isGlonassPerSlotIfbKey`) to exempt them from the receiver-block zeroing; only the inter-slot *differences* remain estimable in SD.

Satellite code biases B^i_s. Per (satellite, signal) pair, but *only when the BIA file does not supply an OSB* for that pair — typically a third-frequency signal in a triple-frequency setup. When present, these states are weakly stochastic (`sat_bias_q` $\sim 10^{-10}$) since the underlying physical biases are nearly stationary on the daily timescale.

Satellite phase biases $\bar{b}^i_s$ (network mode). Present only in the network producer (`ProcessingMode::PPP_RTK_Network`) and gated by `estimation.estimate_sat_phase_biases` (default off, inert in every other mode). The user side imports the satellite phase bias as a known SSR correction (Section 10) or absorbs it into the float ambiguity; the network instead carries an *explicit* per-(satellite, signal) phase-bias state on the undifferenced phase rows (the estimable bias of Odijk [6], lumping the network pivot-receiver ambiguity). On a single station's row it is co-linear with the per-receiver float ambiguity and is separable only *across* stations in the joint

network estimator (Section 12). It lives in a *distinct* state map from the satellite *code* biases above (`StateManager::sat_phase_bias_indices_`), keyed `"<sat>|<signal>|PB"`, and carries near-stationary process noise (`sat_phase_bias_q` $\sim 10^{-12}$).

Inertial augmentation. When a tightly-coupled IMU is active the core state grows by nine further entries — a three-axis attitude error and the accelerometer and gyroscope biases — propagated by the strapdown mechanisation rather than the random-walk process model. These are treated in full in Section 9.

Baseline differential troposphere. In baseline mode the per-station tropospheric, clock, ionosphere, and bias states are absent; an optional single differential zenith-wet-delay state $\Delta\mathrm{ZWD}_{r_{\mathrm{rov}}r_{\mathrm{ref}}}$ may be carried (`baseline.estimate_differential_tropo`, Section 5.3).

3.3 Dynamic indexing

The variable-dimension blocks are addressed via key→index maps maintained by `StateManager`. The canonical key formats are defined once in `include/filter/StateKeySchema.hpp`, each with a `make`/`parse` pair:

- iono key: `"<sys><prn>"`, e.g. `"G05"` (delimiter-free);
- ambiguity key: `"<sys><prn>_<signal>"`, e.g. `"G05_L1C"` — split on the *first* underscore, so an iono-free combination `"G05_L1C_L5Q"` retains its compound signal;
- satellite signal-bias key (code): `"<sys><prn>|<signal>"`, e.g. `"G05|C1W"` — a *pipe* delimiter and a distinct map, *not* the underscore ambiguity namespace;
- satellite phase-bias key (network): `"<sys><prn>|<signal>|PB"`, e.g. `"G05|L1C|PB"` — the trailing `|PB` keeps it disjoint from the code-bias namespace, in its own map (`sat_phase_bias_indices_`);
- receiver code-bias key: `"<system>_IFB<band>"` or `"<system>_DCB"`; the per-slot GLONASS variant appends an FCN marker, `"GLONASS_IFB<band>_k<±slot>"` (e.g. `"GLONASS_IFB1_k-4"`);
- inter-system time-offset key: the bare system name (`"Galileo"`, …; GPS is the reference and never appears);
- double-difference ambiguity key (baseline): `"DD|<cons>|<sat>|<code>|<pivot>"`.

This single source of truth eliminates the off-by-one errors that plague hand-managed dynamic state vectors and makes the BSSD/DD S-transforms (Section 5) implementable as block operations on the relevant key sets.

Ambiguity basis. By default the ambiguity block carries one per-band float ambiguity per tracked (satellite, signal) in metres. Setting `estimation.state_ambiguity_basis = "nl_wl"` switches the whole run to a narrow-lane / wide-lane parameterisation: per satellite a narrow-lane state (key `"<sat>_NL_<band>"`) and one wide-lane state per band pair (`"<sat>_WL_<a>_<b>"`, semantically $N_a - N_b$), both in *cycles* rather than metres so the wide-lane and LAMBDA arithmetic stays native to integers, with the wavelength scaling moved into the Jacobian. The two bases never coexist in a single state vector.

4 Observation equations

This section gives the full *undifferenced* observation model that the filter linearises around at each epoch. The PPP-RTK parameterisation is uncombined: per-signal pseudorange and carrier phase are consumed directly, without forming ionosphere-free or wide-lane combinations as a preprocessing step. Combinations (IF, MW, GF) are still computed for diagnostic and slip-detection purposes (Section 7; see `slip/MwSlipDetector`, `slip/GraphicSlipDetector`) but they do not enter the measurement update path.

4.1 Code (pseudorange) observation

For receiver r tracking satellite i on signal s, the observed pseudorange is

$$\boxed{P^i_{r,s} \;=\; \rho^i_r \;+\; c(\mathrm{d}t_r - \mathrm{d}t^i) \;+\; c\,\delta t_r^{(g(i))} \;+\; T^i_r \;+\; \mu_s\, I^i_{r,1} \;+\; B_{r,s} - B^i_s \;+\; \varepsilon^P_{r,i,s}.} \tag{4}$$

4.2 Carrier-phase observation

The corresponding carrier-phase observation, scaled to metres by λ_s, is

$$\boxed{\Phi^i_{r,s} \;=\; \rho^i_r \;+\; c(\mathrm{d}t_r - \mathrm{d}t^i) \;+\; c\,\delta t_r^{(g(i))} \;+\; T^i_r \;-\; \mu_s\, I^i_{r,1} \;+\; \lambda_s\, N^i_{r,s} \;+\; b_{r,s} - b^i_s \;+\; \varepsilon^\Phi_{r,i,s}.} \tag{5}$$

The defining structural difference between Equations (4) and (5) is the sign of the ionospheric term: $+\mu_s I$ on code, $-\mu_s I$ on phase. Three additional remarks:

1. The phase ambiguity $N^i_{r,s}$ is integer in principle but is contaminated by the receiver-plus-satellite phase biases $b_{r,s} - b^i_s$. In a *float* solution these biases are absorbed into $N^i_{r,s}$, leaving a non-integer state with no structural advantage over the code observation. Recovering integer-valued ambiguities therefore requires either external Fractional Cycle Bias (FCB) products or differencing (Section 5), which cancels common-mode bias terms.
2. The clock product $c\,\mathrm{d}t^i$ is read from the analysis-centre CLK file (post-processing) or refined from the broadcast clock by a streamed SSR correction (real-time, Section 10) and treated as deterministic in the filter; any residual product error appears on the observation and is absorbed by the receiver clock state.
3. The system time offset $c\,\delta t_r^{(g(i))}$ is zero by construction when $g(i)$ is the reference constellation (default GPS). Together with the receiver clock $c\,\mathrm{d}t_r$ this means a multi-constellation run carries $|\mathcal{G}|$ clock-like states in total.

4.3 Geometric range and linearisation

The geometric range $\rho^i_r = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|_2$ enters Equations (4) and (5) non-linearly through the receiver position $\boldsymbol{r}_r$. The filter linearises about the current state estimate $\hat{\boldsymbol{r}}_r$:

$$\rho^i_r \;=\; \hat{\rho}^i_r \;-\; (\boldsymbol{e}^i_{\hat{r}})^{\mathrm{T}}(\boldsymbol{r}_r - \hat{\boldsymbol{r}}_r) \;+\; \mathcal{O}\big(\|\boldsymbol{r}_r - \hat{\boldsymbol{r}}_r\|^2\big)\,, \tag{6}$$

with the unit line-of-sight

$$\boldsymbol{e}^i_{\hat{r}} \;:=\; \frac{\boldsymbol{r}^i - \hat{\boldsymbol{r}}_r}{\|\boldsymbol{r}^i - \hat{\boldsymbol{r}}_r\|_2}. \tag{7}$$

The Jacobian row $\boldsymbol{H}_{(i,s)}$ contributing to the design matrix at the position columns is therefore $-(\boldsymbol{e}^i_{\hat{r}})^{\mathrm{T}}$. For sub-metre residual targets this single linearisation per epoch is sufficient; the engine

optionally relinearises within the epoch (`processing.relinearize_geometry`, which defaults to off in code but is enabled in all shipped configurations) to handle long initial position offsets.

The satellite position $\boldsymbol{r}^i$ is itself a function of the transmission time, which depends on ρ_r^i/c and therefore on $\boldsymbol{r}_r$. Resolving this implicit dependency by fixed-point iteration (typically 1–2 sweeps) is performed by `model/ModelGeometry`.

4.4 Stochastic model

Observation noise covariance is structured per epoch as a block diagonal matrix with entries

$$\sigma^2_{r,i,s} \;=\; \sigma^2_{0,\mathrm{type}(s)} \;\cdot\; w(\theta^i_r)^{-1} \;\cdot\; \xi(\mathrm{C/N_0 \text{ at } } (r,i,s)), \tag{8}$$

combining a per-type baseline ($\sigma_{0,\mathrm{code}}$, $\sigma_{0,\mathrm{phase}}$ from `filter.measurement_noise`), an elevation-dependent down-weighting factor $w(\theta) \in \{1, \sin\theta, \sin^2\theta, \dots\}$ selected by `filter.stochastic_model`, and an optional carrier-to-noise ratio scaling ξ parameterised by `snr_model_params`. Cross-correlations between observations are assumed zero in the undifferenced parameterisation; differencing introduces structured correlations handled in Section 5.

5 Differencing transformations

PPP-RTK and baseline modes do not consume the undifferenced observations of Equations (4) and (5) directly. Each forms a linear combination across satellites (BSSD), receivers (BRSD), or both (DD), absorbing common-mode terms into the cancellation and recovering integer-friendly residual structure. This section gives both the algebraic cancellations and the matrix-form S-transforms that map undifferenced parameters to differenced ones.

5.1 Between-satellite single difference (BSSD)

For each constellation g, choose a pivot satellite p_g (typically the highest-elevation tracked sat with a stable phase lock; see `filter/PivotManager`, "sticky pivot" policy). For any non-pivot satellite $i \in \mathcal{S}_g \setminus \{p_g\}$ and signal s, define

$$\begin{aligned} \Delta P^{i,p_g}_{r,s} &:= P^i_{r,s} - P^{p_g}_{r,s}, \\ \Delta \Phi^{i,p_g}_{r,s} &:= \Phi^i_{r,s} - \Phi^{p_g}_{r,s}. \end{aligned} \tag{9}$$

Substituting Equations (4) and (5) and cancelling the receiver-side common terms ($c\,\mathrm{d}t_r$, $c\,\delta t_r^{(g)}$, $B_{r,s}$, $b_{r,s}$) yields

$$\boxed{\begin{aligned} \Delta P^{i,p_g}_{r,s} &= \Delta\rho^{i,p_g}_r - c\,\Delta \mathrm{d}t^{i,p_g} + \Delta T^{i,p_g}_r + \mu_s\,\Delta I^{i,p_g}_{r,1} - \Delta B^{i,p_g}_s + \Delta\varepsilon^{P}, \qquad (10)\\ \Delta \Phi^{i,p_g}_{r,s} &= \Delta\rho^{i,p_g}_r - c\,\Delta \mathrm{d}t^{i,p_g} + \Delta T^{i,p_g}_r - \mu_s\,\Delta I^{i,p_g}_{r,1} + \lambda_s\,\Delta N^{i,p_g}_{r,s} - \Delta b^{i,p_g}_s + \Delta\varepsilon^{\Phi}. \qquad (11) \end{aligned}}$$

The cancellations are exact only when both observations enter the filter with the same modelling: same tropospheric mapping (which is why ΔT is small but not zero), same antenna PCV correction, and so on.

Pivot transitions. When the incumbent pivot leaves the visible set, p_g is re-elected from the remaining satellites. The float ambiguities $\{\Delta N_{r,s}^{i,p_g^{\text{old}}}\}_{i,s}$ must be re-expressed relative to p_g^{new}. Rather than discarding and re-seeding (which destroys the integer information accumulated since the last fix), the engine applies the linear S-transformation $\boldsymbol{x}^{\text{new}} = \boldsymbol{S}\,\boldsymbol{x}^{\text{old}}$ with $\boldsymbol{S}$ chosen so that $\Delta N_{r,s}^{p_g^{\text{new}},p_g^{\text{old}}}$ becomes the new reference (zero). The covariance transforms as $\boldsymbol{P}^{\text{new}} = \boldsymbol{S}\,\boldsymbol{P}^{\text{old}}\,\boldsymbol{S}^{\mathrm{T}}$. These are two distinct code paths sharing the algebra above: in PPP-RTK the pivot re-election and the "sticky-pivot" retention policy live in `filter/PivotManager`, whereas the baseline DD path performs the analogous re-reference through the ambiguity-rename S-transform in `filter/DdPivotManager`, governed by the `baseline.enable_pivot_handover` flag.

5.2 Between-receiver single difference (BRSD)

In baseline modes there is at least one rover r_{rov} and exactly one fixed-coordinate reference r_{ref}. The BRSD operator forms

$$\nabla P^i_{r_{\text{rov}}r_{\text{ref}},s} \;:=\; P^i_{r_{\text{rov}},s} - P^i_{r_{\text{ref}},s}, \tag{12}$$

and analogously for $\nabla\Phi$. The cancellations produced by $\nabla\cdot$ are dual to those produced by $\Delta\cdot$: BRSD cancels *satellite*-side common terms ($c\,\mathrm{d}t^i$, B^i_s, b^i_s) but preserves receiver clock and biases of *both* stations. No state-space estimation of satellite biases is required when ∇ is applied — they disappear before reaching the filter.

5.3 Double difference (DD)

The composition $\nabla\Delta\cdot = \Delta\big(\nabla\,\cdot\big)$ is order-independent up to noise correlation: $\Delta\big(\nabla\,\cdot\big) = \nabla\big(\Delta\,\cdot\big)$. Applying it to the carrier-phase model and collecting common-mode cancellations:

$$\nabla\Delta\Phi^{i,p_g}_{r_{\text{rov}}r_{\text{ref}},s} \;=\; \nabla\Delta\rho^{i,p_g}_{r_{\text{rov}}r_{\text{ref}}} + \nabla\Delta T^{i,p_g}_{r_{\text{rov}}r_{\text{ref}}} - \mu_s\,\nabla\Delta I^{i,p_g}_{r_{\text{rov}}r_{\text{ref}},1} + \lambda_s\,\nabla\Delta N^{i,p_g}_{r_{\text{rov}}r_{\text{ref}},s} + \nabla\Delta\varepsilon^{\Phi}. \tag{13}$$

What survives in Equation (13) is precisely the geometry term (linear in the rover position perturbation), the differential atmospheric delays, and the integer DD ambiguity — the structural form that makes carrier-phase RTK possible.

Short vs. long baselines. On short baselines ($\lesssim 10\,\text{km}$) the differential atmospheric terms in Equation (13) are negligible at the millimetre level and need not be modelled. As baseline length grows, the differential ionosphere $\nabla\Delta I$ scales linearly with separation and saturates at typically 1 m for separations of a few hundred kilometres, swamping the carrier wavelength. Two mitigations are available in `GnssProcessor`:

- `baseline.use_iono_free` forms the dual-frequency ionosphere-free combination at the DD level, eliminating the first-order ionospheric term at the cost of a non-integer ambiguity (AR is automatically excluded for IF rows) and $\sim 3\times$ amplified code noise;
- `baseline.estimate_differential_tropo` adds a single $\Delta\text{ZWD}_{r_{\text{rov}}r_{\text{ref}}}$ state with slow random-walk dynamics, capturing the residual differential wet delay that survives the BRSD on baselines with significant altitude difference.

5.4 S-transform structure

Both BSSD and DD admit a matrix-form representation as $\boldsymbol{x}_{\text{diff}} = \boldsymbol{S}\,\boldsymbol{x}_{\text{undiff}}$ with $\boldsymbol{S}$ acting as a block design transform. For BSSD restricted to the ambiguity block of one constellation g on one

signal s, with $|\mathcal{S}_g|$ tracked sats, $\boldsymbol{S}$ is a $(|\mathcal{S}_g|-1)\times|\mathcal{S}_g|$ matrix of the form

$$\boldsymbol{S}_{\mathrm{BSSD}} = \begin{bmatrix} 1 & 0 & \cdots & 0 & -1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & -1 & 0 & \cdots & 0 \\ \vdots & & \ddots & & \vdots & & & \\ 0 & 0 & \cdots & 0 & -1 & 0 & \cdots & 1 \end{bmatrix}, \tag{14}$$

where the column of -1's sits at the index of the pivot p_g. The DD operator is the *composition* of the two single-difference operators applied in sequence, $\boldsymbol{S}_{\mathrm{DD}} = \boldsymbol{S}_{\mathrm{BSSD}}\,\boldsymbol{S}_{\mathrm{BRSD}}$, realised in code as `BrsdTransform::apply` followed by `BssdTransform::differenceBrsd` (`include/measurement/BssdTransform.hpp`). Only on a fully populated satellite × receiver observation grid does this composition coincide with the Kronecker product $\mathbf{D}_{\mathrm{sat}} \otimes \mathbf{D}_{\mathrm{rx}}$ of the between-satellite and between-receiver differencing matrices acting on the vectorised observations — a useful mental model, but not what the non-rectangular real-data case computes. The transformed covariance is $\boldsymbol{P}_{\mathrm{diff}} = \boldsymbol{S}\,\boldsymbol{P}_{\mathrm{undiff}}\,\boldsymbol{S}^{\mathrm{T}}$, a generally dense matrix even when $\boldsymbol{P}_{\mathrm{undiff}}$ is diagonal (because each differenced row borrows from the same pivot column). This dense correlation structure is the algebraic reason DD-based filters cannot be naively implemented as independent per-pair scalar updates — the correctly correlated joint update is what unlocks integer ambiguity recovery via the LAMBDA decorrelation. The general theory and a thorough catalogue of admissible S-transforms is given in Teunissen [11] and in the rank-defect treatment of Jonge [4].

Implementation notes. Two facts about how `GnssProcessor` realises the transforms above are worth stating explicitly. First, the chosen S-system is never materialised as an explicit minimum-constraint $\boldsymbol{S}$-basis matrix; it is realised *implicitly* through the pivot-differencing operator (`measurement/BssdTransform`), a faithful, S-basis-invariant realisation of the uncombined parameterisation of Odijk [6]. The three regimes — undifferenced, BSSD, and DD — are dispatched polymorphically by a `DifferencingMode` strategy in `measurement/MeasurementBuilder`, not by inline config branches. Second, integer recovery on a DD row requires that all four contributing observations share the same RINEX signal code; *cross-attribute aliased* rows (rover code ≠ reference code, flagged `brsd_aliased` and not `integer_capable`) are carried as float and excluded from ambiguity resolution.

Part II
Estimation engine

6 Estimation engine: the Kalman filter and RTS smoother

This section documents the recursive estimator at the core of the engine: a variable-dimension extended Kalman filter (EKF) with an optional prior-anchored iterated (IEKF) relinearisation pass, robust post-fit residual gating, and a backward Rauch–Tung–Striebel (RTS) smoother for post-processing modes. The state vector $\boldsymbol{x}$ is the uncombined PPP / PPP-RTK parameterisation of Section 5: receiver position (and optionally velocity), receiver clock and per-system time offsets, zenith wet troposphere and gradients, per-satellite slant ionosphere, receiver/satellite code and phase biases, and one float ambiguity per tracked carrier. Its dimension changes epoch to epoch as satellites rise and set and as ambiguities are reset; the layout and the dynamic key→index maps are owned exclusively by `filter/StateManager`, and the filter operates on that state without any fixed-index assumptions (`src/core/filter/KalmanFilter.cpp`).

Throughout we write $\boldsymbol{F}$ for the state-transition matrix, $\boldsymbol{Q}$ for the discrete process-noise covariance, $\boldsymbol{R}$ for the measurement-noise covariance, $\boldsymbol{S}$ for the innovation covariance, $\boldsymbol{K}$ for the Kalman gain, and $\boldsymbol{\nu}$ for the innovation (measurement minus predicted measurement). Predicted (a-priori) and updated (a-posteriori) quantities use the tilde/hat and $k|k-1$ / $k|k$ conditioning convention fixed in Section 2.

6.1 Time update

State transition. The transition matrix $\boldsymbol{F}(\Delta t)$ for epoch interval $\Delta t = t_k - t_{k-1}$ is identity plus a small number of coupling blocks, assembled by `StateManager::getTransitionMatrix` (`src/core/filter/StateManager.cpp`). The only off-diagonal couplings are the kinematic integrators

$$\boldsymbol{F}[\,\boldsymbol{r},\boldsymbol{v}\,] = \Delta t\,\boldsymbol{I}_3, \qquad \boldsymbol{F}[\,c\mathrm{d}t,\ c\dot{\mathrm{d}t}\,] = \Delta t, \qquad \boldsymbol{F}[\,c\delta t^{(g)},\ c\dot{\delta t}^{(g)}\,] = \Delta t, \tag{15}$$

present only when the corresponding rate state exists (`hasVelocity()`, clock-rate, per-system rate). Every remaining state — ambiguities, per-satellite ionosphere, zenith wet delay, gradients, code and satellite biases, and any rate-free inter-system offset — has $\boldsymbol{F} = \boldsymbol{I}$ and is propagated as a pure random walk with its mean held constant. Under the BSSD (single-difference) parameterisation the receiver-clock, phase-clock (`getPhaseClockIndex`), inter-system-offset, and frozen-pivot per-satellite ionosphere (`getFrozenPivotSatellites`) rows are unobservable (their $\boldsymbol{H}$ columns are zero); their position rows in Equation (15) are suppressed and, for each, the corresponding cross-covariances are zeroed in place after propagation while the diagonal variance is preserved (the `freeze_state` block in `src/core/filter/KalmanFilter.cpp`).

The propagation itself is the standard EKF time update

$$\boxed{\begin{aligned} \tilde{\boldsymbol{x}}_{k|k-1} &= \boldsymbol{F}(\Delta t)\,\hat{\boldsymbol{x}}_{k-1|k-1}, && (16)\\ \tilde{\boldsymbol{P}}_{k|k-1} &= \boldsymbol{F}(\Delta t)\,\boldsymbol{P}_{k-1|k-1}\,\boldsymbol{F}(\Delta t)^{\mathrm{T}} + \boldsymbol{Q}(\Delta t), && (17) \end{aligned}}$$

implemented in `KalmanFilter::predict`. A non-positive Δt is treated as a no-op (duplicate or out-of-order epoch), short-circuiting before Equation (17).

Process-noise model. The discrete process noise $\boldsymbol{Q}(\Delta t)$ is built from continuous spectral densities by `StateManager::getProcessNoiseMatrix`. Two block structures appear. Pure random-walk states have a scalar discrete variance increment

$$\boldsymbol{Q}_{ii} = q_i\,\Delta t, \tag{18}$$

where q_i is the continuous variance rate (m^2/s) read from `filter.process_noise.*`. Representative entries are zenith wet troposphere (`troposphere_q`), the receiver code bias (`code_bias_q`), satellite phase bias (`sat_bias_q`), phase clock (`phase_clock_q`), and the float ambiguity (`ambiguity_q`). The ambiguity density defaults to a numerical safety floor (10^{-12}) rather than a physical value, and per-key overrides are supported for arc-specific reseeding. The per-satellite slant ionosphere uses a random walk whose zenith density is mapped to the slant by the squared obliquity factor,

$$\boldsymbol{Q}_{ii} = \frac{q_{\mathrm{iono}}}{\sin^2\theta^i}\,\Delta t, \qquad \sin^2\theta^i \text{ floored at } 10^{-2}\ (\theta \approx 5.7^\circ), \tag{19}$$

with q_{iono} = `ionosphere_q` and optional per-satellite overrides driven by the adaptive-iono scheme.

Coupled position/velocity, clock/clock-rate, and inter-system offset/rate pairs use the continuous white-noise-acceleration (CWNA, integrated-random-walk) discretisation. For a coupled pair $(\xi, \dot{\xi})$ driven by acceleration PSD q,

$$\boldsymbol{Q}_{[\xi,\dot{\xi}]} = \begin{bmatrix} \frac{1}{3}\, q\, \Delta t^3 & \frac{1}{2}\, q\, \Delta t^2 \\ \frac{1}{2}\, q\, \Delta t^2 & q\, \Delta t \end{bmatrix}, \tag{20}$$

with q given by `acceleration_psd_q` (position/velocity, the 3×3 block-diagonal extension of Equation (20) over the ECEF axes), `clock_rate_q`, and `system_time_rate_offset_q` respectively. When the velocity state is present the rate-free position density `position_q` is unused. Baseline (double-difference) mode reduces $\boldsymbol{Q}$ to the position block, a per-ambiguity random-walk term, and an optional differential-ZWD entry $\boldsymbol{Q}_{ii} = \sigma^2 \Delta t$ governed by `baseline.differential_tropo_process_noise_m_per_sqrt_s`.

Aided time update. When inertial aiding is active, `KalmanFilter::predictImu` performs a strapdown mechanisation of the nominal position, velocity, and attitude and splices a 15×15 INS error-state transition $\boldsymbol{\Phi}$ and noise $\boldsymbol{Q}_{\text{ins}}$ — with error order $[\delta\boldsymbol{p},\, \delta\boldsymbol{v},\, \delta\boldsymbol{\psi},\, \delta\boldsymbol{b}_a,\, \delta\boldsymbol{b}_g]$ — into the GNSS-dimensioned $\boldsymbol{F}$ and $\boldsymbol{Q}$ over the integrated interval, after which Equation (17) runs on the full state with explicit symmetrisation $\boldsymbol{P} \leftarrow \frac{1}{2}(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}})$. The generalised `predictAided` path applies the same splice for arbitrary aiding sources and delegates byte-identically to the GNSS-only Equation (17) when no aiding payload is present.

6.2 Measurement update

The measurement update (`KalmanFilter::update`) is a *batch* update: all observation rows of the epoch are processed simultaneously, not sequentially/scalar. A sparse-$\boldsymbol{H}$ optimisation accelerates the four design-matrix products ($\boldsymbol{HP}$, $\boldsymbol{HPH}^{\mathrm{T}}$, $\boldsymbol{KH}$, and the $\boldsymbol{H}$-touching parts of the gain), and is numerically identical to the dense computation to within a 10^{-12} structural threshold (`src/core/filter/KalmanFilter.cpp`).

Given the design matrix $\boldsymbol{H}$ from the linearisation of the Equation (4) and Equation (5) of the Foundations document (position rows $-(\boldsymbol{e}^i_{\hat{r}})^{\mathrm{T}}$, the Equation (6)), the innovation $\boldsymbol{\nu}$ (`measurement.innovations`), and the measurement-noise covariance $\boldsymbol{R}$ — a full matrix when supplied, otherwise the diagonal $\boldsymbol{R} = \mathrm{diag}(\sigma^2_{r,i,s})$ of the Equation (8) — the innovation covariance and gain are

$$\boldsymbol{S} = \boldsymbol{H}\, \tilde{\boldsymbol{P}}_{k|k-1}\, \boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R}, \tag{21}$$

$$\boldsymbol{K} = \tilde{\boldsymbol{P}}_{k|k-1}\, \boldsymbol{H}^{\mathrm{T}}\, \boldsymbol{S}^{-1}. \tag{22}$$

The gain is never formed by explicitly inverting $\boldsymbol{S}$. Exploiting the symmetry of $\boldsymbol{S}$, the engine factorises it once with an LDL^{T} decomposition and recovers $\boldsymbol{K} = \left(\boldsymbol{S}^{-1}\boldsymbol{H}\tilde{\boldsymbol{P}}\right)^{\mathrm{T}}$ from a single solve against $\boldsymbol{H}\tilde{\boldsymbol{P}}$ (`ldlt.solve(HP).transpose()`); on factorisation failure it falls back to a complete-orthogonal (pseudo-inverse) solve. The state update is

$$\delta\boldsymbol{x} = \boldsymbol{K}\, \boldsymbol{\nu}, \qquad \hat{\boldsymbol{x}}_{k|k} = \tilde{\boldsymbol{x}}_{k|k-1} + \delta\boldsymbol{x}. \tag{23}$$

Joseph-form covariance update. The covariance is updated in the numerically stabilised *Joseph* form, never the short form $(\boldsymbol{I} - \boldsymbol{KH})\tilde{\boldsymbol{P}}$:

$$\boxed{\boldsymbol{P}_{k|k} = (\boldsymbol{I} - \boldsymbol{KH})\, \tilde{\boldsymbol{P}}_{k|k-1}\, (\boldsymbol{I} - \boldsymbol{KH})^{\mathrm{T}} + \boldsymbol{K}\, \boldsymbol{R}\, \boldsymbol{K}^{\mathrm{T}}.} \tag{24}$$

The Joseph form preserves symmetry and positive semi-definiteness for any gain, which matters because the engine deliberately departs from the optimal gain in its robust paths (Section 6.4); the result is symmetrised once more, $\boldsymbol{P} \leftarrow \frac{1}{2}(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}})$, to clear round-off. The factor $(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})$ is assembled from the sparse $\boldsymbol{K}\boldsymbol{H}$ product with a unit shift on the diagonal. The same Joseph kernel, with the explicit gain $\boldsymbol{K} = \boldsymbol{P}\boldsymbol{H}^{\mathrm{T}}\boldsymbol{S}^{-1}$, is exposed as the pure static routine `josephPseudoUpdate` and reused for scalar pseudo-observation constraints (e.g. the GIM ionosphere prior via `applyPseudoObservation`) and for the IEKF pass of Section 6.3.

Post-fit residuals. After the update the post-fit residuals and their (diagonal) covariance are

$$\boldsymbol{r} = \boldsymbol{\nu} - \boldsymbol{H}\,\delta\boldsymbol{x}, \qquad S_{r,ii} = R_{ii} - \left(\boldsymbol{H}\,\boldsymbol{P}_{k|k}\,\boldsymbol{H}^{\mathrm{T}}\right)_{ii}, \tag{25}$$

of which only the diagonal of S_r is materialised. The standardised residual that drives gating (Section 6.4) is

$$s_i = \frac{r_i}{\sqrt{\max\{\, S_{r,ii},\, R_{ii},\, 10^{-12} \,\}}}. \tag{26}$$

The denominator floor at the *working* R_{ii} (the inflated noise actually used in $\boldsymbol{R}$, not the formal noise) is a deliberate robustness compromise: in ambiguity-absorption cases — double-differenced phase with an ambiguity gain near unity — the rigorous denominator $S_{r,ii}$ collapses toward $R\,\boldsymbol{S}^{-1}R$, which would register near-zero residuals as many-σ outliers. The strictly correct standardisation uses $S_{r,ii}$ alone; the floor trades that rigour for stability and is documented as such in source.

6.3 Iterated relinearisation (IEKF)

For long initial position offsets a single linearisation of the Equation (6) per epoch can be insufficient. When `processing.relinearize_geometry` is enabled (and the run is not tightly-coupled IMU), the engine performs a prior-anchored iterated update — a Gauss–Newton / iterated EKF step — after the primary update.

Anchor. The iteration is anchored to the *stashed pre-update prior* $(\boldsymbol{x}^{-}, \boldsymbol{P}^{-})$ that the converged primary Equations (17) and (24) read, captured only when relinearisation is armed (`getLastUpdatePriorX` / `getLastUpdatePriorP`). This is the prior *after* any in-epoch ambiguity augmentation and outlier reset, so it is dimension-matched to the posterior; it is not a top-of-loop snapshot.

Gauss–Newton step. Let $\boldsymbol{x}_i$ be the current linearisation iterate (the committed state at which `measurement` was linearised, with innovations $\boldsymbol{z} - h(\boldsymbol{x}_i)$ and $\boldsymbol{H} = \partial h/\partial \boldsymbol{x}\big|_{\boldsymbol{x}_i}$). The prior-anchored iterate (`KalmanFilter::updateFromPrior`, via `josephPseudoUpdate`) is

$$\boldsymbol{\nu} = \left(\boldsymbol{z} - h(\boldsymbol{x}_i)\right) - \boldsymbol{H}(\boldsymbol{x}^{-} - \boldsymbol{x}_i), \tag{27}$$

$$\boldsymbol{K} = \boldsymbol{P}^{-}\boldsymbol{H}^{\mathrm{T}}\left(\boldsymbol{H}\boldsymbol{P}^{-}\boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R}\right)^{-1}, \qquad \boldsymbol{x}^{+} = \boldsymbol{x}^{-} + \boldsymbol{K}\,\boldsymbol{\nu}, \tag{28}$$

$$\boldsymbol{P}^{+} = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\,\boldsymbol{P}^{-}\,(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})^{\mathrm{T}} + \boldsymbol{K}\,\boldsymbol{R}\,\boldsymbol{K}^{\mathrm{T}}. \tag{29}$$

The offset term $-\boldsymbol{H}(\boldsymbol{x}^{-} - \boldsymbol{x}_i)$ in Equation (27) re-expresses the innovation about the fixed prior and is what makes this a proper MAP / Gauss–Newton iteration: because $\boldsymbol{K}$ and $(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\boldsymbol{P}^{-}$ are always derived from the same fixed prior $(\boldsymbol{x}^{-}, \boldsymbol{P}^{-})$, the covariance does *not* contract a second time and no information is double-counted.

Firing condition. The driver `Application::runGeometryRelinearization` (`src/core/app/Application.cpp`) fires the iteration when all of the following hold:

1. `processing.relinearize_geometry` is on and a prior was stashed at the primary update;
2. the position change since the primary update exceeds a data-dependent threshold — `relinearize_dpos_threshold_phase_m` (default 1 m) when phase is present, or `relinearize_dpos_threshold_code_only_m` (default 100 m) for code-only epochs;
3. the per-run cap `relinearize_max_count` (-1 = unlimited) is not exhausted.

The inner loop recomputes geometry and re-applies Equations (27) to (29) against the *fixed* converged exclusion set, for at most five iterations, declaring convergence when the position step falls below 10^{-4} m. Because GNSS geometry is nearly linear at the metre level, convergence is typically reached in a single pass and the threshold of item 2 means the iteration rarely fires at all. The tightly-coupled IMU configuration instead takes a single legacy *naive* second pass (re-applying the measurement against the posterior), since error-state attitude/bias feedback conflicts with a clean prior re-derivation.

6.4 Innovation and residual gating

The engine performs *no* χ^2 innovation-covariance test. Outlier rejection (`KalmanFilter::validatePostFitResiduals`) operates on the standardised *post-fit* residuals s_i of Equation (26) against a fixed scalar threshold, supplemented by robust median-based spatial detectors for pivot slips. The two top-level thresholds are hardcoded `GnssConfig` defaults (`residual_threshold_` $= 6.0$ and `bad_meas_percent_` $= 0.5$ in `src/core/config/GnssConfig.cpp`) that are deliberately not exposed to JSON: the per-row rejection threshold is $s_{\max} = 6.0$ (read at runtime via `getResidualThreshold`) and the complete-reset trigger fraction is 0.5 (`getBadMeasPercent`); readers should not search for config keys for these.

Per-row rejection. If $\max_i |s_i| > 6.0$, the worst row is dispatched by type: a phase outlier triggers an ambiguity reset (with BSSD pivot handling and a sub-cycle λ-fraction floor), while a code outlier is excluded from the epoch. Correlated same-satellite phase rows on other bands are reset alongside it.

Complete-reset backstop. If the count of rejected rows exceeds $0.5 \times$ (original pre-peel observation count), the filter covariance is re-inflated (`resetFilterCovariance`) — a hard reset that, in the smoother, marks a segment boundary (Section 6.5).

Robust pivot-slip detectors (BSSD). Before the per-row gate, single-difference phase rows are grouped by (pivot, signal) and screened for a coherent pivot slip using median / median-absolute-deviation (MAD) statistics: a forced pivot handover fires when a group is large enough, a majority of its rows exceed threshold, the group median exceeds a metre-scale floor, and the MAD-to-median ratio is small (a tight cluster indicating a common-mode shift rather than scattered noise). Cross-band pooled and code-side variants apply analogous tests with their own floors, and non-pivot multi-band slips are confirmed by integer-cycle consistency ($|r|/\lambda_{\text{band}}$ rounding to the same nonzero integer across bands).

Down-weighting (not gating). Two paths inflate $\boldsymbol{R}$ rather than reject: a persistent EWMA-bias term adds the squared mean bias to R_{ii} once an arc has warmed up, and a Huber post-fit scheme inflates R for sub-cycle rows whose $|s_i|$ exceeds the Huber knee (innovation one-shot or post-fit IRLS variants, toggled by `enable_huber_r_inflation` and `enable_huber_innovation`). These modify the working $\boldsymbol{R}$ that feeds Equations (21) and (26), and are the reason the standardised residual floors at the working noise rather than the formal noise.

6.5 Backward RTS smoother

In post-processing modes, enabling `processing.run_smoother` runs a fixed-interval Rauch–Tung–Striebel recursion [7] over the stored forward trajectory (`include/filter/Smoother.hpp`, `src/core/filter/Smoother.cpp`). The flag is force-disabled under `AppMode::RealTime`, since smoothing is non-causal and inherently a post-processing operation.

Forward-pass capture. Rather than reconstruct the a-priori from $\boldsymbol{F}$ and $\boldsymbol{Q}$ on the backward pass, the forward pass caches, per epoch, both the one-step prediction $\tilde{\boldsymbol{x}}_{k|k-1}, \tilde{\boldsymbol{P}}_{k|k-1}$ and the posterior $\hat{\boldsymbol{x}}_{k|k}, \boldsymbol{P}_{k|k}$, together with $\boldsymbol{F}_k$ and the full set of core and dynamic key→index maps (`capturePriorSnapshot` / `capturePosteriorSnapshot` in `src/core/app/Application.cpp`). A capture-ordering invariant makes the stored prior exactly $\boldsymbol{F}_k\hat{\boldsymbol{x}}_{k-1|k-1}$: the a-priori is recorded immediately after `predict()` and before any cycle-slip reseeding or measurement update, and the a-posteriori is recorded *before* integer ambiguity resolution, so the smoother consumes *float* ambiguities only.

Recursion. Initialised at the terminal posterior $\hat{\boldsymbol{x}}_{N|N} = \hat{\boldsymbol{x}}_{N|N}$, $\boldsymbol{P}_{N|N} = \boldsymbol{P}_{N|N}$, the backward recursion is

$$\boxed{\begin{aligned} \boldsymbol{G}_k &= \boldsymbol{P}_{k|k}\,\boldsymbol{F}_{k+1}^{\mathrm{T}}\,\tilde{\boldsymbol{P}}_{k+1|k}^{-1}, && (30)\\ \hat{\boldsymbol{x}}_{k|N} &= \hat{\boldsymbol{x}}_{k|k} + \boldsymbol{G}_k\big(\hat{\boldsymbol{x}}_{k+1|N} - \tilde{\boldsymbol{x}}_{k+1|k}\big), && (31)\\ \boldsymbol{P}_{k|N} &= \boldsymbol{P}_{k|k} + \boldsymbol{G}_k\big(\boldsymbol{P}_{k+1|N} - \tilde{\boldsymbol{P}}_{k+1|k}\big)\boldsymbol{G}_k^{\mathrm{T}}. && (32) \end{aligned}}$$

The smoother gain $\boldsymbol{G}_k$ is realised as an LDL^{T} solve — the predicted covariance is symmetrised, then $\boldsymbol{G}_k = \big(\tilde{\boldsymbol{P}}_{k+1|k}^{-1}(\boldsymbol{P}_{k|k}\boldsymbol{F}^{\mathrm{T}})^{\mathrm{T}}\big)^{\mathrm{T}}$ via `ldlt.solve(...).transpose()` — never an explicit inverse, and the smoothed covariance Equation (32) is symmetrised after the update. If the factorisation fails, the step is abandoned and epoch k keeps its forward posterior.

Variable-dimension reconciliation. Because $\dim \boldsymbol{x}_k \neq \dim \boldsymbol{x}_{k+1}$ in general, Equations (30) to (32) are restricted at each step to the *common-key subspace* — the states present at both k and $k{+}1$, found by intersecting the dynamic key→index maps (`collectCommonIndices`). Let $\boldsymbol{\Pi}_k$ and $\boldsymbol{\Pi}_{k+1}$ select the common rows in each epoch's state space; the implemented state update is then

$$\hat{\boldsymbol{x}}_{k|N} = \hat{\boldsymbol{x}}_{k|k} + \boldsymbol{\Pi}_k^{\mathrm{T}}\,\boldsymbol{G}_{k,C}\big(\boldsymbol{\Pi}_{k+1}\hat{\boldsymbol{x}}_{k+1|N} - \boldsymbol{\Pi}_k\tilde{\boldsymbol{x}}_{k+1|k}\big), \tag{33}$$

with the analogous projection on Equation (32). States outside the subspace retain their forward a-posteriori value. A subtle indexing invariant: $\tilde{\boldsymbol{x}}_{k+1|k}$, $\tilde{\boldsymbol{P}}_{k+1|k}$ and $\boldsymbol{F}_{k+1}$ are sized for epoch k's layout (the predict step does not grow the state), so they are addressed with epoch k's common indices, whereas $\hat{\boldsymbol{x}}_{k+1|N}$, $\boldsymbol{P}_{k+1|N}$ live in epoch $k{+}1$'s layout.

Segment boundaries at ambiguity resets. Two mechanisms of differing severity terminate or trim backward information flow at ambiguity resets:

- A *complete* filter reset at $k{+}1$ (the covariance re-inflation of Section 6.4, recorded as `is_discontinuity`) is a hard segment boundary: it severs the dynamic link $\tilde{\boldsymbol{x}}_{k+1|k} = \boldsymbol{F}\hat{\boldsymbol{x}}_{k|k}$, so no $\boldsymbol{G}_k$ is meaningful and backward flow is terminated — epoch k keeps its forward posterior.
- A *single-satellite* ambiguity reset is handled softly: that satellite's ambiguities are excluded from the common subspace for the $k \to k{+}1$ step, because the re-seeded large-variance prior at $k{+}1$ carries no information backward and smoothing it would only inject noise. The exclusion matches by satellite prefix and is intentionally slightly over-broad, always erring toward the safe forward fallback.

Static-position degeneracy. In static PPP the receiver position is a (near-)constant state, so the position-block smoother gain propagates essentially the same correction to every epoch and the smoothed position is degenerate — constant across epochs — *by construction*. This is expected, not a defect: the smoother's value accrues on the non-position states (troposphere, ionosphere, float ambiguities) and on kinematic runs where position carries genuine epoch-to-epoch dynamics.

Part III
Cycle-slip detection and repair

7 Cycle-slip detection and repair

The uncombined filter (Section 4) never forms a linear combination on the measurement-update path: per-signal $P^i_{r,s}$ and $\Phi^i_{r,s}$ enter Equations (4) and (5) directly. Cycle slips are instead screened by a bank of *diagnostic* combinations, each constructed so that the geometric range ρ^i_r, the clocks $c(\mathrm{d}t_r - \mathrm{d}t^i)$ and the troposphere T^i_r cancel, leaving an observable whose only between-epoch variation is measurement noise plus any integer carrier discontinuity. The active detectors are Melbourne–Wübbena (`src/core/slip/MwSlipDetector.cpp`, the primary) and GRAPHIC (`src/core/slip/GraphicSlipDetector.cpp`); the extra-wide-lane detector (`src/core/slip/EwlSlipDetector.cpp`) runs when a third frequency is present. The geometry-free and ionosphere-free phase detectors (`src/core/slip/GfSlipDetector.cpp`, `src/core/slip/IfPhaseSlipDetector.cpp`) are staged — present in the tree but not compiled into the current build — and their algebra is recorded here for completeness. The bank is composed at run time from the config block `estimation.cycle_slip` (Section 7.1).

Throughout this section we suppress the receiver index r (single-receiver PPP / PPP-RTK) and write $f_1 > f_2$ for the two carriers of a band pair, sorted by frequency so that $f_1 - f_2 > 0$ and $\lambda_1 < \lambda_2$ (`src/core/slip/MwSlipDetector.cpp`, the band-frequency sort). Carrier phase is read from RINEX in cycles and scaled by $\lambda_s = c/f_s$ before combining, so Φ^i_s below is always in metres, consistent with Equation (5). We reserve the operator $\delta\cdot$ for a *temporal* integer slip step (e.g. δN_s is the change in the ambiguity on signal s across the slip epoch); this is deliberately distinct from the between-satellite single difference $\Delta\cdot$ of Section 5, which also uses Δ.

7.1 Detector registry and composition

The detectors are not hard-wired. Each implements a narrow polymorphic contract `ISlipDetector` (`include/slip/ISlipDetector.hpp`) — a single per-epoch entry point

$$\texttt{detect}(\text{SlipEpochContext}) \longrightarrow \{\,\text{SlipEvent}\,\},$$

plus lifecycle hooks (`resetSatellite`, `resetSatelliteSignal`, `enableTracing`, `setCorrections`). A `SlipEvent` is the union of the legacy per-detector result structs; its `granularity` field distinguishes a *per-signal* flag (GRAPHIC, GF, IF: one carrier) from a *band-difference* flag (MW, EWL: a wide-lane integer $\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2$ that, on its own, cannot attribute the jump to one signal — the two-unknown problem Equation (45) resolves downstream). Inputs are bundled in `SlipEpochContext` so the signature is uniform even though only the staged IF detector consumes the extra 3D position sigma and receiver ECEF it carries.

`SlipDetectorRegistry::createEnabled` (`src/core/slip/SlipDetectorRegistry.cpp`) builds the config-selected set from `estimation.cycle_slip.{mw,graphic,ewl,gf,if}` in a deterministic order — MW first (it is the primary and the sole source of the wide-lane integers consumed by ambiguity resolution (Section 8) and the BSSD pivot machinery (`src/core/filter/PivotManager.cpp`)), then the auxiliaries. MW defaults on; the auxiliary detectors default off, preserving the historical MW-only behaviour. A kind that is enabled in config but *unavailable* in this build — the staged `gf` and `if` detectors, whose `.cpp` is not compiled — is skipped with a warning rather than aborting, so a forward-looking config degrades gracefully. `CycleSlipManager` drives the whole set through one loop and recovers the MW detector's richer API (WL-lock queries, pivot-handover hints, per-satellite quality scoring) through a typed down-cast (`asMelbourneWubbena`) rather than widening the base interface.

`turbo_edit` (Section 7.3) sits a level above the registry: it is *not* a detector kind but a *coordination mode* that, when the MW detector is enabled, pairs the MW wide-lane integer with the geometry-free phase jump to recover per-signal integers. It is therefore documented separately from the detector bank below.

7.2 Test-statistic combinations

7.2.1 Melbourne–Wübbena (wide-lane)

The Melbourne–Wübbena (MW) observable is the wide-lane carrier phase minus the narrow-lane code. Introducing the wide- and narrow-lane wavelengths

$$\lambda_{\mathrm{WL}} := \frac{c}{f_1 - f_2}, \qquad \lambda_{\mathrm{NL}} := \frac{c}{f_1 + f_2}, \tag{34}$$

the engine forms (`MwSlipDetector::computeMwMeters`)

$$\boxed{\mathrm{MW}^i = \underbrace{\frac{f_1\,\Phi_1^i - f_2\,\Phi_2^i}{f_1 - f_2}}_{\text{wide-lane phase}} - \underbrace{\frac{f_1\,P_1^i + f_2\,P_2^i}{f_1 + f_2}}_{\text{narrow-lane code}}}. \tag{35}$$

The range-additive terms common to Equations (4) and (5) — geometry, clocks, troposphere — cancel between the two bracketed combinations. The slant ionosphere also cancels: the wide-lane phase carries the ionospheric coefficient $-(f_1\mu_1 - f_2\mu_2)/(f_1 - f_2)$ and the narrow-lane code carries $+(f_1\mu_1 + f_2\mu_2)/(f_1 + f_2)$, and with $\mu_s = f_1^2/f_s^2$ (Equation (1)) these two coefficients are equal, so

the difference (35) removes the ionosphere exactly to first order. What remains is a near-constant equal to the wide-lane ambiguity plus hardware bias,

$$\mathrm{MW}^i \;\approx\; \lambda_{\mathrm{WL}}\, N^i_{\mathrm{WL}} \;+\; b^i_{\mathrm{MW}} \;+\; \varepsilon_{\mathrm{MW}}, \qquad N^i_{\mathrm{WL}} = N^i_1 - N^i_2, \tag{36}$$

so a wide-lane slip $\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2$ appears as a step of $\lambda_{\mathrm{WL}}\,\delta N_{\mathrm{WL}}$. For GPS L1/L2, $\lambda_{\mathrm{WL}} \approx 0.861\,\mathrm{m}$ while the dominant error is the metre-level narrow-lane *code* noise; with code noise $\sigma_P \sim 0.3\,\mathrm{m}$ the MW noise is $\sigma_{\mathrm{MW}} \approx \sigma_P/(\sqrt{2}\,\lambda_{\mathrm{WL}}) \sim 0.25$ wide-lane cycles, i.e. code-noise-limited. The integer-ready (BIA-corrected) variant subtracts the satellite MW bias b^i_{MW}, which has the same algebra as Equation (35) applied to the OSB phase and code biases of Equations (4) and (5).

The level series $\mathrm{MW}^i(k)$ is screened against a robust adaptive gate whose deviation statistic is configurable via `estimation.cycle_slip.mw_detect_reference`. The recommended — and replay-default — choice is the first time difference $\mathrm{TD1}(k) = \mathrm{MW}^i(k) - \mathrm{MW}^i(k-1)$, gated against the median-absolute-deviation of its own history; the second time difference $\mathrm{TD2}(k) = \mathrm{MW}^i(k) - 2\,\mathrm{MW}^i(k-1) + \mathrm{MW}^i(k-2)$ is available (`mw_td_order`) for curvature immunity at a $\sqrt{6}$ noise penalty versus the $\sqrt{2}$ of TD1 (the white-noise scaling of each statistic is the single source of truth in `MwSlipDetector::activeStatScale`). A windowed local median and the legacy Hatch-mean reference are retained for A/B comparison (`local_median`, `hatch_mean`). The threshold itself is fully data-driven — no stochastic-model σ enters — combining a per-arc MAD, an EWMA-smoothed cross-satellite MAD floor and an elevation $1/\sin\theta$ shape, with a breadth cascade (band-pair → constellation → global-no-GLONASS) for sparse or cold-start arcs; the empirical construction lives in `MwSlipDetector::computeAdaptiveThreshold` and is intentionally not reproduced here.

7.2.2 Extra-wide-lane (EWL)

For three-or-more frequency satellites the same MW algebra (Equation (35)) is applied to a pair of *non-L1* carriers (`EwlSlipDetector::computeEwlMwMeters`; band digit ≥ 2). Because the two carriers are spectrally close — $f_1 - f_2$ small — the extra-wide-lane wavelength

$$\lambda_{\mathrm{EWL}} \;=\; \frac{c}{f_1 - f_2} \tag{37}$$

is very long: GPS L2/L5 gives $\lambda_{\mathrm{EWL}} \approx 5.86\,\mathrm{m}$ and Galileo E5a/E5b $\approx 9.77\,\mathrm{m}$. Metre-level code noise then maps to $\lesssim 0.05$ EWL cycles, so the EWL integer $N_{\mathrm{EWL}} = N_2 - N_5$ resolves in a single epoch and a slip δN_{EWL} is sized with near-certainty. Together with the L1/L2 wide-lane (Equation (36)) this yields two independent band-difference constraints,

$$\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2, \qquad \delta N_{\mathrm{EWL}} = \delta N_2 - \delta N_5, \tag{38}$$

narrowing the per-signal triple $(\delta N_1, \delta N_2, \delta N_5)$ to a single remaining degree of freedom [2].

7.2.3 GRAPHIC (single-frequency half-sum)

The GRAPHIC combination averages the code and phase on a *single* band (`GraphicSlipDetector::computeGraphic`):

$$\mathrm{GR}^i_s \;=\; \tfrac{1}{2}\big(P^i_s + \Phi^i_s\big), \tag{39}$$

with Φ^i_s in metres. Because the slant ionosphere enters $+\mu_s I^i_{r,1}$ on the code and $-\mu_s I^i_{r,1}$ on the phase of the *same* signal (Equations (4) and (5)), the half-sum cancels it to first order — GRAPHIC is ionosphere-free without a second frequency, at the cost of retaining half the code

noise. The combination still contains the geometric range, clocks and troposphere plus the constant half-ambiguity $\frac{1}{2}\lambda_s N_s^i$, so the between-epoch first difference

$$\mathrm{dGR}_s^i(k) \;=\; \mathrm{GR}_s^i(k) - \mathrm{GR}_s^i(k-1) \tag{40}$$

removes the constant ambiguity (and cancels range and clocks down to receiver-motion / code-noise level). A δN-cycle slip on signal s therefore produces a jump of $\frac{1}{2}\lambda_s\,\delta N$, and the integer estimate is $\delta N = \mathrm{round}\big(2\,\mathrm{dGR}/\lambda_s\big)$. With $\sigma_P \sim 0.3\,\mathrm{m}$ the post-difference noise is $\sim 0.3\,\mathrm{m}$, so sensitivity is noise-limited to a few L1 cycles; in practice a hard floor `kMinThresholdM` $= 0.5\,\mathrm{m}$ governs on quiet arcs (≈ 5 L1 cycles), keeping GRAPHIC a coarse backstop rather than a sub-cycle detector [1, 14].

7.2.4 Geometry-free phase (staged)

The geometry-free (GF) observable is the between-band phase difference

$$\mathrm{GF}^i \;=\; \Phi_1^i - \Phi_2^i \;\approx\; (\mu_2 - \mu_1)\, I_{r,1}^i \;+\; (\lambda_1 N_1^i - \lambda_2 N_2^i), \tag{41}$$

which removes geometry, clocks and troposphere exactly but retains the *difference* of the two ionospheric delays. Since $I_{r,1}^i$ varies smoothly, a second-order time difference

$$\mathrm{TD2}_{\mathrm{GF}}(k) \;=\; \mathrm{GF}^i(k) - 2\,\mathrm{GF}^i(k-1) + \mathrm{GF}^i(k-2) \tag{42}$$

cancels the ionospheric trend up to quadratic order over the sampling interval, leaving noise plus slip (`GfSlipDetector::computeGfMeters` for Equation (41); the TD2 update is the level-jump test statistic). The detector is intrinsically high-rate ($\geq 1\,\mathrm{Hz}$): at coarse rate the residual ionospheric curvature defeats (42). It is present in the tree but excluded from the build; the same algebra backs the active `src/core/slip/IonoVariabilityMonitor.cpp`.

7.2.5 Ionosphere-free phase (staged)

The dual-frequency ionosphere-free (IF) phase combination uses the coefficients

$$c_1 = \frac{f_1^2}{f_1^2 - f_2^2}, \qquad c_2 = \frac{f_2^2}{f_1^2 - f_2^2}, \qquad c_1 - c_2 = 1, \tag{43}$$

giving $\Phi_{\mathrm{IF}}^i = c_1\Phi_1^i - c_2\Phi_2^i$ and the effective wavelength $\lambda_{\mathrm{IF}} = \big|c_1\lambda_1 - c_2\lambda_2\big|$ (`IfPhaseSlipDetector::computeIfWavelengthMeters`); for GPS L1/L2 $\lambda_{\mathrm{IF}} \approx 0.107\,\mathrm{m}$. Rather than screen the raw combination, the staged detector forms the IF observed-minus-computed (OMC), differences it between satellites against a high-elevation pivot p_g to remove the receiver clock, the inter-system offset $c\,\delta t_r^{(g)}$ and the satellite phase clock, then between epochs to remove the slowly-varying wet troposphere and the constant ambiguity:

$$\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g}(k) = \big[\Phi_{\mathrm{IF}}^i(k) - \Phi_{\mathrm{IF}}^{p_g}(k)\big] - \big[\Phi_{\mathrm{IF}}^i(k-1) - \Phi_{\mathrm{IF}}^{p_g}(k-1)\big], \tag{44}$$

which is near zero for a clean arc and jumps by $\sim \delta N\,\lambda_{\mathrm{IF}}$ on a slip. The between-satellite step does not cancel the differential geometric error $(\boldsymbol{e}^i - \boldsymbol{e}^{p_g})^{\mathrm{T}}\delta\boldsymbol{r}$, so flagging must be gated by the filter's 3D position sigma; this restricts the staged detector to a static site with a precise a priori position, which is why `estimation.enable_if_slip_detector` is currently inert.

7.3 TurboEdit dual-frequency resolution

The TurboEdit resolver is a *pure* MW+GF joint integer solver (`src/core/slip/TurboEditResolver.cpp` and its header `include/slip/TurboEditResolver.hpp`). It owns the per-arc geometry-free jump history and the integer arithmetic, and returns per-signal integer repairs plus a reset-fallback set without mutating filter state; it is wired into both the post-processing `CycleSlipManager` and the real-time engine, and enabled by `estimation.cycle_slip.turbo_edit`.

7.3.1 The 2×2 system

For a dual-frequency phase pair (s_1, s_2) on satellite i, ordered higher-frequency first ($f_{s_1} > f_{s_2}$, hence $\lambda_{s_1} < \lambda_{s_2}$), the resolver pairs two scalar slip observables. The first is the MW wide-lane integer of Equation (36), taken *as input* from the MW detector and never recomputed (the empirically tuned MW path of Section 7.2.1 is the single source of δN_{WL}):

$$\delta N_{\mathrm{WL}} \;=\; \delta N_{s_1} - \delta N_{s_2} \qquad \text{(one equation, two unknowns).} \tag{45}$$

The second is the GF phase jump in metres. Writing the GF level as $\mathrm{GF}(t) = \lambda_{s_1} L_{s_1}(t) - \lambda_{s_2} L_{s_2}(t)$ (carrier in cycles), a simultaneous slip steps its ambiguity content by

$$\delta\mathrm{GF} \;=\; \lambda_{s_1}\,\delta N_{s_1} - \lambda_{s_2}\,\delta N_{s_2}, \tag{46}$$

estimated in-arc by the second-order time difference $\mathrm{TD2}(t) = \mathrm{GF}(t) - 2\,\mathrm{GF}(t-1) + \mathrm{GF}(t-2)$ of Equation (42), which cancels the ionospheric delay up to quadratic order so that at the slip epoch it equals the integer jump plus phase noise only. The TD2 chain requires three *consecutive* epochs; non-consecutive history forces a chain re-seed (Section 7.3.4).

Stacking Equations (45) and (46) gives the linear map from the integer slip pair to the two observed jumps:

$$\boxed{\begin{pmatrix} \delta N_{\mathrm{WL}} \\ \delta\mathrm{GF} \end{pmatrix} \;=\; \underbrace{\begin{pmatrix} 1 & -1 \\ \lambda_{s_1} & -\lambda_{s_2} \end{pmatrix}}_{\boldsymbol{A}} \begin{pmatrix} \delta N_{s_1} \\ \delta N_{s_2} \end{pmatrix}.} \tag{47}$$

The determinant $\det \boldsymbol{A} = \lambda_{s_1} - \lambda_{s_2}$ is non-zero for any genuine cross-band pair — it is the cm-scale quantity `denom` of Equation (49), $\approx -0.0539\,\mathrm{m}$ for GPS L1/L2, *not* the $\sim 0.86\,\mathrm{m}$ wide-lane wavelength — so the system is always invertible:

$$\boldsymbol{A}^{-1} = \frac{1}{\lambda_{s_1} - \lambda_{s_2}} \begin{pmatrix} -\lambda_{s_2} & 1 \\ -\lambda_{s_1} & 1 \end{pmatrix}. \tag{48}$$

The implementation does not invert the full matrix; it solves the second row for δN_{s_2} and back-substitutes the first (`TurboEditResolver.cpp`):

$$\delta N_{s_2} \;=\; \frac{\delta\mathrm{GF} - \lambda_{s_1}\,\delta N_{\mathrm{WL}}}{\lambda_{s_1} - \lambda_{s_2}}, \qquad \delta N_{s_1} \;=\; \delta N_{s_2} + \delta N_{\mathrm{WL}}, \tag{49}$$

with $\delta\mathrm{GF}$ instantiated either by the in-arc TD2 or, across a data gap, by the bridged jump of Section 7.3.4. The denominator $\lambda_{s_1} - \lambda_{s_2}$ (the variable `denom` in the source) equals $\approx -0.0539\,\mathrm{m}$ for GPS L1/L2, so the amplification factor on the wide-lane integer is $\lambda_{s_1}/(\lambda_{s_1} - \lambda_{s_2}) \approx -3.53$: the GF equation must localise δN_{s_2} to well within ± 0.5 cycle despite that lever, which is what the split-capability gate below enforces.

7.3.2 Integer rounding and validation

The float δN_{s_2} from Equation (49) is rounded and its residual recorded:

$$d_2 = \mathrm{round}\big(\delta N_{s_2}\big), \quad f = \big|\delta N_{s_2} - d_2\big|, \quad d_1 = d_2 + \delta N_{\mathrm{WL}}. \tag{50}$$

A candidate is *accepted* as a repair only if the rounding is clean ($f \le 0.30$, the constant `kMaxFractional`; a $1 - 0.30 = 0.70$ cycle margin to the nearest wrong integer), the magnitudes are sane ($|d_1|, |d_2| \le$ `kMaxSlipCycles`), and the pair is non-trivial ($d_1 \neq 0 \vee d_2 \neq 0$). A non-trivial accepted candidate yields `action="repair"`; $(d_1, d_2) = (0, 0)$ is resolved as no-slip and the arc keeps rolling; otherwise the satellite is reset.

Before the joint solve is *trusted*, the GF evidence must be sharp enough to discriminate the integer lattice. The float δN_{s_2} carries $\sigma_{\mathrm{TD2}}/|\lambda_{s_1} - \lambda_{s_2}|$ cycles of noise, so the solve is permitted only when

$$\sigma_{\mathrm{TD2}} \;\le\; \texttt{turbo_edit_split_max_sigma} \cdot \big|\lambda_{s_1} - \lambda_{s_2}\big| \qquad (\texttt{splitCapable}), \tag{51}$$

with σ_{TD2} the arc's own MAD-derived σ when mature, else the cross-satellite tier floor, else NaN (NaN fails by design — "no estimate" is not evidence of sharpness). The default `estimation.cycle_slip.turbo_edit_split_max_sigma` is 0.2 cycles. This gate is the single mechanism making the resolver rate-adaptive: millimetric 1 s noise passes trivially, while coarse-rate (30 s) ionospheric residual passes only while the arc is quiet. On an MW-flagged pair failing the gate the resolver falls back to a satellite reset; a GF-only exceedance on a split-incapable arc is *ignored*, not reset. When $\delta N_{\mathrm{WL}} = 0$ (an equal slip $\delta N_{s_1} = \delta N_{s_2}$, to which MW is blind) a GF-only flag fires on |TD2| exceeding an adaptive threshold $\tau = \max(k_\sigma \hat{\sigma}_{\mathrm{TD2}}, 0.04\,\mathrm{m})$, with k_σ the `turbo_edit_sigma_multiplier`; the floor sits just below $|\lambda_{s_1} - \lambda_{s_2}| \approx 0.054$ m so a ± 1-cycle equal slip remains detectable.

7.3.3 MW-only / code-event null line

The asymmetric case is MW flagged but the GF jump *affirmatively zero*. For a fixed $\delta N_{\mathrm{WL}} = k$, every integer pair $(\delta N_{s_1}, \delta N_{s_1} - k)$ predicts, via Equation (46), a GF jump

$$\delta\mathrm{GF} \;=\; \lambda_{s_2} k - (\lambda_{s_2} - \lambda_{s_1})\, \delta N_{s_1}, \tag{52}$$

whose zero crossing sits at $\delta N_{s_1} = \lambda_{s_2} k/(\lambda_{s_2} - \lambda_{s_1}) \approx 4.53\,k$ for GPS L1/L2 ($\approx 3.95\,k$ for L1/L5). Every small pair $|\delta N_{s_1}| \le 3$ then predicts $|\delta\mathrm{GF}| \ge 8$ cm — orders above phase noise — so a flat GF leaves only *null-line* pairs of $(4, 3)$ / $(9, 7)$-class magnitude: implausible multi-cycle simultaneous slips on both signals with no loss of lock. The Occam-simpler explanation is a narrow-lane *code* excursion (multipath, NLOS, DLL re-lock), since $\mathrm{MW} = \Phi_{\mathrm{WL}} - P_{\mathrm{NL}}$ and one wide-lane cycle ≈ 0.86 m of narrow-lane code.

Accordingly the pair is classified a code-domain artifact — and the caller *skips* the phase reset — when the GF jump is affirmatively zero within a discrimination-bounded gate,

$$|\mathrm{TD2}| \le \underbrace{\texttt{kCodeEventZeroSigma} \cdot \hat{\sigma}_{\mathrm{MAD}}}_{\text{zero gate}} \quad \wedge \quad \text{zero gate} \le \underbrace{\texttt{kVetoZeroGateMaxLambdaFrac} \cdot |\lambda_{s_1} - \lambda_{s_2}|}_{\text{discrimination cap}}, \tag{53}$$

with `kCodeEventZeroSigma` $= 3.5$ ($\approx 99.95\%$ two-sided) and `kVetoZeroGateMaxLambdaFrac` $= 0.5$. The cap is the rate-adaptive safeguard: a zero gate wider than half the smallest GF-visible quantum cannot distinguish stillness from a small slip, so the veto stands down and the MW mandate governs. The classification is further trust-gated — veto enabled, MW flagged, GF not exceeding, no post-slip cooldown, ionospherically stable constellation, no LLI on either signal, and MAD maturity ≥ 10

samples (stricter than detection's 5, because affirming stillness is a state-*preserving* decision). The implied null-line pair is computed for the trace only, never applied; `did_slip` stays false so the GF chain keeps rolling. A genuine null-line slip is not lost: it jumps that signal's Kalman phase innovation by $\geq |4\lambda_{s_1}| \approx 0.76\,\mathrm{m}$ this very epoch, far beyond any residual gate, so the residual-driven reset corrects it one epoch late. The behaviour is toggled by `estimation.cycle_slip.turbo_edit_mw_only_veto` (default on).

7.3.4 Gap-spanning GF bridge

The TD2 chain needs three consecutive epochs; any per-signal dropout or receiver gap re-seeds the chain, making the post-gap TD2 *blind* to a re-lock slip whose GF level jump lives *between* the two chains. When a chain break occurs within `estimation.cycle_slip.turbo_edit_max_gap_s`, the resolver stashes the pre-gap GF level $\mathrm{GF}_{\mathrm{ref}}$, its timestamp, the arc's EWMA GF rate $\dot{\mathrm{GF}}$ and the pre-gap spacing dt_0, and on return forms the drift-corrected across-gap jump

$$\boxed{\delta\mathrm{GF}_{\mathrm{gap}} \;=\; \mathrm{GF}_{\mathrm{now}} - \mathrm{GF}_{\mathrm{ref}} - \dot{\mathrm{GF}}\,\Delta t, \qquad \Delta t = t_{\mathrm{now}} - t_{\mathrm{ref}},} \tag{54}$$

which then plays the role of δGF in Equation (49). Here $\dot{\mathrm{GF}}$ is an EWMA ($\alpha = 0.1$) of the per-epoch GF rate, chosen so its noise does not amplify when projected over the gap. Because the un-cancelled ionospheric residual is random-walk over the gap, the detection threshold and the code-event zero gate are widened by the factor $w = \sqrt{\Delta t/dt_0}$ (for $\Delta t > dt_0$), and the split-capability σ becomes $\sigma_{\mathrm{bridge}} = \hat{\sigma}_{\mathrm{MAD}}\, w$; the zero-gate *discrimination cap* stays un-widened, since the integer lattice of Equation (47) does not stretch with the gap. The bridge cap default is 15 s in code, raised to 60 s in the bundled real-time and replay configs.

An MW-flagged pair across the gap is attributed three ways. (i) A *code event* (`action="code_event"`) when $\delta\mathrm{GF}_{\mathrm{gap}}$ is affirmatively zero within the widened gate — the same null-line logic as Equation (53), applied across the gap — preserving phase through a re-acquisition DLL transient. (ii) A *repair* or *moot* when the joint solve runs with $\delta\mathrm{GF} = \delta\mathrm{GF}_{\mathrm{gap}}$; per signal it is dispatched by observation-domain continuity, a signal observed at every epoch through the gap kept its ambiguity (its nonzero δN is a genuine *repair*), whereas a signal that missed an epoch was pruned-and-reseeded on return (its nonzero δN is *moot* — already absorbed, mandate suppressed). (iii) A *reset* when the across-gap σ is too coarse for the split, or the candidate set fits zero or two integers with no deferral.

Because the GF equation alone is degenerate along the null-line lattice of Equation (52) — $+0.735\,\mathrm{m}$ fits both $(0, -3)$ and $(9, 4)$ at millimetre level; only the integer k separates them — the bridged solve demands a *unique* surviving candidate after three guards: the fractional gate of Equation (50); a GF-separation guard requiring the candidate's own predicted jump to clear the zero gate, $|\lambda_{s_1} d_1 - \lambda_{s_2} d_2| >$ `zero_gate`, lest a marginal noise exceedance mint a multi-cycle repair from noise; and a sibling-pair zero affirmation, whereby a satellite's *other* GF arc that stayed chain-continuous and unflagged through the bridged interval affirms $\delta N = 0$ for both its signals and prunes any candidate demanding a nonzero integer on an affirmed signal. The wide-lane integer feeding the bridged solve is taken in trust order: the published MW integer k; failing that, at the flag epoch, $\mathrm{round}(f) \pm 3$ around the (NL-code-biased) instantaneous re-acquisition float; and on deferral epochs the settled across-gap MW Hatch mean. When the float wide-lane evidence is unsettled or the lattice is ambiguous (two or more acceptable integers), the solve is *deferred* rather than guessed (`estimation.cycle_slip.turbo_edit_wl_settle_max_s`), terminating in a later repair, reset or timeout-reset.

7.3.5 Atomic reconciliation

Per-satellite candidates are reconciled atomically: a cross-pair integer disagreement on a shared signal demotes the whole satellite to reset, and a satellite ending in the reset set has its code-event, moot, clean and pending classifications *withdrawn* — contradictory evidence always resolves to the conservative outcome. Arc bookkeeping keeps the TD2 validity precondition honest: a non-consecutive update partially resets the chain but preserves the slowly-varying MAD calibration window, an absolute gap beyond the configured maximum wipes the arc, and an accepted slip re-seeds with a short cooldown to suppress the TD2 echo; only clean (non-exceeding, non-bridged) TD2 samples feed the MAD window.

Part IV
Integer ambiguity resolution

8 Integer ambiguity resolution

Integer ambiguity resolution (AR) converts the real-valued, bias-contaminated phase ambiguities of the float solution into the integer carrier-cycle counts they represent. The engine implements MLAMBDA [12] over a between-satellite-differenced ambiguity sub-problem extracted from the filter state and covariance. The operative quantity is the ambiguity differenced against a per-constellation pivot p_g (Section 5); the same integer-estimation machinery serves both the single-receiver and the baseline modes, but *what makes that differenced ambiguity an integer differs by mode*. The undifferenced ambiguity $N^i_{r,s}$ of the phase observation equation (Equation (5)) carries a receiver phase bias $b_{r,s}$ and a satellite phase bias b^i_s. In single-receiver PPP / PPP-RTK the pivot difference is a *between-satellite single difference* $\Delta\,\cdot$ (Section 5.1), which cancels the receiver bias $b_{r,s}$ (common to every tracked satellite) but *not* the satellite bias; the residual $\Delta N^{i,p_g}_{r,s}$ is integer only once b^i_s has been removed by a consistent external phase-bias product (BIA / SSR, Section 10). In baseline mode the state already holds true double differences $\nabla\Delta\,\cdot$ (Section 5.3), where the additional between-receiver difference cancels the satellite bias as well, so $\nabla\Delta N$ is integer intrinsically. We retain the resolver's internal label $\nabla\Delta$ for the differenced ambiguity in the equations below — the algebra is identical either way — with the understanding that it denotes the between-satellite single difference Δ in the single-receiver case and the full double difference only in baseline mode. The implementation lives in `src/core/filter/AmbiguityResolver.cpp` (driver, search kernel, WL/NL decomposition, acceptance) with configuration under the JSON path `estimation.ambiguity_resolution.*`.

8.1 The integer least-squares problem

The float solution delivers a real-valued ambiguity estimate $\hat{\boldsymbol{N}}$ (in cycles) with covariance $\boldsymbol{Q}_{\hat{N}}$, both obtained by sub-vector extraction from the filter state $\boldsymbol{x}$ and covariance $\boldsymbol{P}$. Integer ambiguity resolution is the weighted closest-lattice-point problem

$$\boxed{\check{\boldsymbol{N}} \;=\; \arg\min_{\boldsymbol{a}\,\in\,\mathbb{Z}^n} \big(\hat{\boldsymbol{N}}-\boldsymbol{a}\big)^{\mathrm{T}}\,\boldsymbol{Q}_{\hat{N}}^{-1}\,\big(\hat{\boldsymbol{N}}-\boldsymbol{a}\big),} \tag{55}$$

i.e. minimisation of the squared Mahalanobis distance from $\hat{\boldsymbol{N}}$ to a lattice point $\boldsymbol{a}$ in the metric of $\boldsymbol{Q}_{\hat{N}}^{-1}$. This is an NP-hard integer least-squares (ILS) program; LAMBDA and its MLAMBDA variant make it tractable in three invariant stages — (i) an LDL^{T} factorisation of $\boldsymbol{Q}_{\hat{N}}$, (ii) a

decorrelating integer $\boldsymbol{Z}$-transform (lattice reduction), and (iii) a depth-first shrinking-ellipsoid search over the reduced lattice — none of which carries a tuning parameter. We follow the hat/check convention of the notation block: $\hat{\boldsymbol{N}}$ is the float estimate, $\check{\boldsymbol{N}}$ the integer-fixed solution.

Float extraction and rescaling to cycles. The filter state stores per-band ambiguity states in *metres* ($\lambda_s N$, one per tracked (satellite, signal)), not cycles, so the resolver first forms the between-satellite-differenced float and covariance by explicit differencing against the pivot p_g and then rescales by wavelength. (In baseline mode the state already holds the double difference and the differencing step below collapses to a direct selection; see Section 8.6.) For a differenced pair k with satellite index $i(k)$ and pivot index $p(k)$ (`src/core/filter/AmbiguityResolver.cpp`):

$$y_k^{\nabla\Delta} = \boldsymbol{x}_{i(k)} - \boldsymbol{x}_{p(k)}, \qquad \text{(DD float, m)} \tag{56}$$

$$\boldsymbol{Q}_{kl}^{\nabla\Delta} = \boldsymbol{P}_{i_k i_l} - \boldsymbol{P}_{i_k p_l} - \boldsymbol{P}_{p_k i_l} + \boldsymbol{P}_{p_k p_l}, \qquad \text{(DD covariance, m}^2\text{)} \tag{57}$$

$$\hat{\boldsymbol{N}}_k = y_k^{\nabla\Delta}/\lambda_k, \qquad (\boldsymbol{Q}_{\hat{N}})_{kl} = \boldsymbol{Q}_{kl}^{\nabla\Delta}/(\lambda_k\lambda_l). \qquad \text{(cycle domain)} \tag{58}$$

The rescaling (58) is load-bearing: running MLAMBDA on metre-domain floats would round toward integer-*metre* values with no carrier-cycle semantics, and the ratio test (Section 8.5) would degenerate to ≈ 1 irrespective of float quality. The per-pair wavelength λ_k is resolved with GLONASS FDMA per-satellite frequencies where applicable, and $\boldsymbol{Q}^{\nabla\Delta}$ is symmetrised, $\boldsymbol{Q}^{\nabla\Delta} \leftarrow \frac{1}{2}(\boldsymbol{Q}^{\nabla\Delta} + \boldsymbol{Q}^{\nabla\Delta\mathrm{T}})$, before factorisation. The DD pivot p_g is the highest-elevation satellite within each (constellation, signal) group, consistent with the BSSD pivot of Section 5; all remaining satellites in the group are differenced against it.

8.2 Decorrelation: LDL^{T} factorisation and the integer $\boldsymbol{Z}$-transform

Factorisation. MLAMBDA factorises the symmetric positive-definite ambiguity covariance as

$$\boldsymbol{Q}_{\hat{N}} \;=\; \boldsymbol{L}^{\mathrm{T}}\,\boldsymbol{D}\,\boldsymbol{L}, \qquad \boldsymbol{L} \text{ unit lower-triangular}, \;\; \boldsymbol{D} = \mathrm{diag}(d_1, \ldots, d_n), \;\; d_i > 0, \tag{59}$$

factored in reverse index order (the Chang–Yang MLAMBDA convention). A non-positive pivot $d_i \le 0$ aborts the fix, since it signals a non-PD covariance. After reduction, d_i is the *conditional variance* of the i-th decorrelated ambiguity given the later ones; it feeds the success-rate gate of Section 8.5 directly.

The $\boldsymbol{Z}$-transform. The central device of LAMBDA [12] is a unimodular integer transform $\boldsymbol{Z} \in \mathbb{Z}^{n\times n}$, $|\det \boldsymbol{Z}| = 1$, that re-parameterises

$$\boldsymbol{z} = \boldsymbol{Z}^{\mathrm{T}}\hat{\boldsymbol{N}}, \qquad \boldsymbol{Q}_{\hat{z}} = \boldsymbol{Z}^{\mathrm{T}}\boldsymbol{Q}_{\hat{N}}\,\boldsymbol{Z}, \tag{60}$$

so that the transformed ambiguities are nearly decorrelated (small off-diagonal $\boldsymbol{L}$ entries) and their conditional variances are ordered. We write the decorrelated float vector as $\bar{\boldsymbol{z}} := \boldsymbol{Z}^{\mathrm{T}}\hat{\boldsymbol{N}}$ (the quantity the search of Section 8.2 actually rounds; implemented as `z_float = Z.transpose()*a` in `src/core/filter/AmbiguityResolver.cpp`), with components $\bar{z}_k$. Because $\boldsymbol{Z}$ is unimodular the integer minimiser maps back exactly, $\check{\boldsymbol{N}} = (\boldsymbol{Z}^{\mathrm{T}})^{-1}\check{\boldsymbol{z}}$ with $(\boldsymbol{Z}^{\mathrm{T}})^{-1}$ also integer; decorrelation therefore does not alter the solution of (55) — it only makes the search ellipsoid closer to spherical, collapsing the enumeration count by orders of magnitude. The reduction interleaves two operations sweeping $j = n-2$ downward.

Integer Gauss transformations zero the magnitude of sub-diagonal $\boldsymbol{L}$ entries by integer column operations,

$$\mu = \lceil L_{ij} \rfloor, \qquad \boldsymbol{L}_{(:,j)} \mathrel{-}= \mu\, \boldsymbol{L}_{(:,i)}, \qquad \boldsymbol{Z}_{(:,j)} \mathrel{-}= \mu\, \boldsymbol{Z}_{(:,i)}, \tag{61}$$

where $\lceil \cdot \rfloor$ is round-to-nearest, keeping every $|L_{ij}| \le \frac{1}{2}$.

Permutations (conditional adjacent swaps) reorder ambiguities when doing so reduces the later conditional variance. With $\bar{d}_j = d_j + L_{j+1,j}^2\, d_{j+1}$, the swap fires when

$$\bar{d}_j + \varepsilon \;<\; d_{j+1}, \qquad \varepsilon = 10^{-6}, \tag{62}$$

after which d_j, d_{j+1}, the affected rows/columns of $\boldsymbol{L}$ and the columns of $\boldsymbol{Z}$ are updated by the standard rank-one formulas ($\eta = d_j/\bar{d}_j$, $\lambda = d_{j+1} L_{j+1,j}/\bar{d}_j$, $d_j \leftarrow \eta\, d_{j+1}$, $d_{j+1} \leftarrow \bar{d}_j$), and the sweep restarts from $j = n-2$. The guard $\varepsilon = 10^{-6}$ prevents thrashing on near-equal variances and is a numerical safeguard, not an estimator knob; it has no config exposure.

The shrinking-ellipsoid search. The reduced problem is solved by depth-first enumeration of lattice points inside the ellipsoid $\sum_i (z_i - \hat{z}_i)^2/d_i \le \chi^2$, in Schnorr–Euchner order (nearest integer first, then zig-zagging outward). Recursing from the last level $k = n-1$ down to 0,

$$\hat{z}_k = \bar{z}_k + \sum_{i>k} (z_i - \hat{z}_i)\, L_{ik}, \qquad \text{(conditional centre)} \tag{63}$$

$$z_k = \lceil \hat{z}_k \rfloor, \qquad \mathrm{dist}_k = \mathrm{dist}_{k+1} + \frac{(\hat{z}_k - z_k)^2}{d_k}. \qquad \text{(partial distance)} \tag{64}$$

When a complete integer vector reaches level 0 with cumulative distance below the current bound χ^2, it is recorded and the bound shrinks to the worst retained candidate, contracting the ellipsoid as better candidates are found. The search retains the $m = 2$ best candidates (for the ratio test) sorted by ascending squared distance, under a hard iteration cap (`kMaxIter` = 10 000). Its outputs are the two smallest objective values $s_1 \le s_2$ of (55) and the best integer vector $\check{\boldsymbol{z}}$, back-transformed via $\check{\boldsymbol{N}} = (\boldsymbol{Z}^{\mathrm{T}})^{-1}\check{\boldsymbol{z}}$.

8.3 Wide-lane / narrow-lane decomposition (PPP path)

Standard PPP and PPP-RTK AR use a *wide-lane-first* decomposition when Melbourne–Wübbena (MW)-resolved wide-lane (WL) integers are available, halving the search dimension and exploiting the long WL wavelength [5]. The driver attempts the WL-constrained narrow-lane (NL) fix before any unconstrained LAMBDA and — critically — does *not* fall through to an unconstrained search when WL information exists but the constrained fix fails: a wrong-WL fix is worse than no fix, so "no NL AR for satellites whose WL is float" is the enforced hierarchy (`src/core/filter/AmbiguityResolver.cpp`).

WL from Melbourne–Wübbena (integer by contract). The DD wide-lane integer is differenced directly from the MW detector's single-difference WL integers (`src/core/slip/MwSlipDetector`), not searched:

$$\nabla\Delta N_{\mathrm{WL}} = N_{\mathrm{WL}}^{i} - N_{\mathrm{WL}}^{p_g} \in \mathbb{Z}, \qquad \lambda_{\mathrm{WL}} = \frac{c}{f_1 - f_2}. \tag{65}$$

NL search variable. For each dual-frequency DD pair the float narrow-lane ambiguity (ionosphere-free phase combination, in NL cycles) is formed by the transform $\boldsymbol{T}$,

$$\hat{n}_{\mathrm{NL}} = \frac{f_1\,\nabla\Delta N_1 + f_2\,\nabla\Delta N_2}{f_1+f_2} = \frac{1}{f_1+f_2}\left(\frac{f_1}{\lambda_1}\,\nabla\Delta_1 + \frac{f_2}{\lambda_2}\,\nabla\Delta_2\right), \qquad \lambda_{\mathrm{NL}} = \frac{c}{f_1+f_2}, \tag{66}$$

with reduced covariance $\boldsymbol{Q}_{\mathrm{NL}} = \boldsymbol{T}\,\boldsymbol{Q}^{\nabla\Delta}\,\boldsymbol{T}^{\mathrm{T}}$. Only NL-paired DDs enter LAMBDA; unpaired single-signal DDs are excluded so they cannot contaminate the ratio test.

Integer-recentring shift. The float $\hat{n}_{\mathrm{NL}}$ is *not* integer-valued in general: substituting $N_2 = N_1 - \mathrm{WL}$ gives $\hat{n}_{\mathrm{NL}} = N_1 - \frac{f_2}{f_1+f_2}\,\mathrm{WL}$, which for GPS L1/L2 with $\mathrm{WL} = 1$ sits at $N_1 - 0.4379$ — neither integer is meaningfully closer and the ratio test degenerates near 1.6 regardless of float quality. The resolver therefore adds the known-WL offset so the search variable is integer in expectation,

$$\tilde{n}_k = \hat{n}_{\mathrm{NL},k} + \frac{f_2}{f_1+f_2}\,\nabla\Delta N_{\mathrm{WL},k} \;\approx\; N_{1,k} \;\in\; \mathbb{Z} + \text{noise}, \tag{67}$$

runs MLAMBDA on $\tilde{\boldsymbol{n}}$ (a constant per-row shift leaves $\boldsymbol{Q}_{\mathrm{NL}}$ unchanged), then back-shifts $\check{n}_{\mathrm{NL}} = \check{N}_1 - \frac{f_2}{f_1+f_2}\,\mathrm{WL}$.

Back-transform to per-signal DDs. The fixed integers map back to the two original signal DDs in metres,

$$\nabla\Delta_1^{\mathrm{fix}} = \left(\check{n}_{\mathrm{NL}} + \tfrac{f_2}{f_1+f_2}\,\nabla\Delta N_{\mathrm{WL}}\right)\lambda_1, \qquad \nabla\Delta_2^{\mathrm{fix}} = \left(\check{n}_{\mathrm{NL}} - \tfrac{f_1}{f_1+f_2}\,\nabla\Delta N_{\mathrm{WL}}\right)\lambda_2, \tag{68}$$

which re-enter the filter through the constrained update of Section 8.6.

State-basis variants. When `estimation.state_ambiguity_basis` = `nl_wl` the state holds NL/WL ambiguities directly and there are no per-band DD pairs to enumerate; today only the MW-resolved WL integers are applied as constraint rows (a full NL LAMBDA search with WL pre-constrained is future work in this basis). The complementary WL-only mode (`estimation.ambiguity_resolution.wl_only`, in the `per_band` basis) applies the cycle-domain constraint $\nabla\Delta_1/\lambda_1 - \nabla\Delta_2/\lambda_2 = \nabla\Delta N_{\mathrm{WL}}$ as a single pseudo-observation per pair without any NL search, with covariance inflation on the affected single-difference states when MW re-locks at a new integer so the state snaps within one epoch. This `per_band wl_only` path carries no cross-frequency consistency check: it gates each DD WL only on the per-arc fractional residual (`wl_max_frac`, both arcs). The cross-frequency parity check lives instead in the `nl_wl` basis (`applyMwWlConstraintsNlWl_` in `src/core/filter/AmbiguityResolver.cpp`), where it rejects all three WL DDs of a triple-frequency satellite when the telescoping canonical cycle-count consistency $\nabla\Delta_{ab} - \nabla\Delta_{ac} + \nabla\Delta_{bc} \neq 0$ fails; it does *not* run in the `per_band wl_only` path.

8.4 Partial / subset ambiguity resolution

Partial AR operates at *two* levels: a static upstream candidate gate that screens which ambiguities ever enter the integer problem, and an optional dynamic backward-elimination search that recovers a fixable subset when the full set fails validation.

Upstream candidate gating (always on). Subset selection begins when the DD pairs are built (`src/core/filter/AmbiguityResolver.cpp`). A satellite/signal ambiguity is admitted to the integer set only if it passes every filter:

- **Initialised** — the state entry is non-zero (seeded by data), and **non-frozen** — its variance is non-zero (excludes a BSSD-frozen pivot).
- **Elevation** — $\theta_r^i \geq \theta_{\mathrm{AR}}$ (`elevation_mask`).
- **Resolvable** — the satellite is not flagged unresolvable for want of bias/ATX (BIA, ATX) coverage (single-receiver path; in baseline DD the analogous filter is the `exclude_keys` set of non-integer-capable DDs from `MeasurementModel`).
- **Constellation** — $g(i) \in$ `active_systems`, when that list is non-empty.
- **Pairable** — the (constellation, signal) group holds ≥ 2 satellites, so a pivot difference exists.

The constellation filter keeps a system in the *float* solution while excluding it from the candidate set — useful when one constellation's residual model bias would drag down the ratio test.

Dynamic backward-elimination subset (`enable_partial_ar`, default off). With the candidate set fixed, the default behaviour is all-or-nothing: the full set is searched and either passes all of Section 8.5 or the epoch stays float. When `enable_partial_ar` is set, a failure of the full-set battery triggers a greedy backward elimination (`selectFixableSubset_`) instead of falling straight back to float. Starting from the full candidate set, each iteration re-runs the complete MLAMBDA pipeline of Sections 8.1 to 8.2 on the current subset and re-tests the *same* acceptance battery — ratio lower/upper bound (69), bootstrap success rate (70), and per-entry fractional cap (71). On any failure it drops one entry and retries, down to a single ambiguity:

- if the *fractional cap* is the failing gate, the entry with the largest $|\hat{\boldsymbol{N}}_k - \check{\boldsymbol{N}}_k|$ (nearest a half-cycle boundary) is removed;
- otherwise the entry with the largest formal float variance $(\boldsymbol{Q}_{\hat{N}})_{kk}$ — the least precise ambiguity, whose removal most improves both the bootstrap rate and the search conditioning — is removed.

Because the accepted subset is always re-validated by the full battery on its own covariance sub-block, this can only fix *fewer* ambiguities than the full set, never a worse-conditioned set: a wrong elimination order costs fixes, not integrity, and the path is bit-identical to the legacy behaviour when off. The same routine serves the per-band SD, baseline DD, and WL-constrained NL paths uniformly, operating purely in the decorrelation-agnostic cycle domain; each caller maps the surviving indices back to its own state rows and feeds them through the constrained update of Section 8.6.

8.5 Validation: ratio test and fixed-failure-rate gate

A fix is committed only if it passes all three of the following gates (a logical AND). The ratio and fixed-failure-rate (FFR) gates are theoretically distinct: one bounds the discrimination between the two best candidates, the other bounds the absolute success probability.

Ratio test (discrimination).

$$\boxed{\text{ratio} = \frac{s_2}{s_1} \;\geq\; \tau_{\text{ratio}}, \qquad s_1 \leq s_2 \text{ the two smallest ILS objectives,}} \tag{69}$$

with s_1, s_2 from the search of Section 8.2 and threshold τ_{ratio} (`ratio_threshold`). A second upper-bound rejection ratio $> \tau_{\text{ratio}}^{\max}$ (`ratio_upper_bound`) catches the pathological case of a degenerate covariance direction — a near-zero s_1 left by a previous wrong-integer hold — that would otherwise produce ultra-confident but meaningless ratios.

Fixed-failure-rate gate (bootstrap success rate). A geometry-driven, self-tuning lower bound on the ILS success probability,

$$\boxed{P_{s,\text{boot}} = \prod_{i=1}^{n} \operatorname{erf}\left(\frac{1}{2\sqrt{2\,d_i}}\right) \;\geq\; P_s^{\min},} \tag{70}$$

where the d_i are the post-reduction LDL^{T} conditional variances of (59) and $P_s^{\min}$ is `min_success_rate`. Per dimension the factor is $\Pr\big(|\mathcal{N}(0, d_i)| < \frac{1}{2}\big)$ — the probability that sequential bootstrapped rounding lands on the correct integer — and the product is the standard Teunissen lower bound on the true ILS success rate. This ties the acceptance threshold to the actual problem geometry rather than to a fixed ratio alone.

Per-pair residual backstop. The joint ratio test can pass on a borderline set in which one pair sits near a half-cycle boundary; the fix is rejected if any single DD's fractional residual exceeds a cap,

$$\max_k \big|\hat{\boldsymbol{N}}_k - \check{\boldsymbol{N}}_k\big| \;\leq\; \kappa_{\text{frac}} \quad \text{(cycles)}, \tag{71}$$

with $\kappa_{\text{frac}} =$ `nl_max_per_pair_frac`.

Convergence guards. Before any search the resolver short-circuits AR while the float is immature — if the epoch count is below `min_epochs` or the position standard deviation $\sqrt{\boldsymbol{P}_{00} + \boldsymbol{P}_{11} + \boldsymbol{P}_{22}}$ exceeds `max_position_sigma_m` — because a marginal fix accepted before the covariance settles can lock onto wrong integers and then poison every subsequent epoch.

8.6 Feedback: constrained pseudo-observation update

Accepted integers are *not* fed back by the classical perfect back-substitution (the $R = 0$ conditioning form)

$$\boldsymbol{x}_{\text{fix}} = \boldsymbol{x} - \boldsymbol{Q}_{\boldsymbol{aa}}\, \boldsymbol{Q}^{\nabla\Delta-1}\big(y^{\nabla\Delta} - \check{\boldsymbol{N}}\big), \qquad \boldsymbol{P}_{\text{fix}} = \boldsymbol{P} - \boldsymbol{Q}_{\boldsymbol{aa}}\, \boldsymbol{Q}^{\nabla\Delta-1}\boldsymbol{Q}_{\boldsymbol{aa}}^{\mathrm{T}}, \tag{72}$$

but via a *finite-variance Joseph-form pseudo-observation* (`src/core/filter/AmbiguityResolver.cpp`; kernel `KalmanFilter::josephPseudoUpdate` in `include/filter/KalmanFilter.hpp`). The reason is structural: (72) drives the fixed ambiguity rows of $\boldsymbol{P}$ to ≈ 0 variance, so the next epoch's LAMBDA sees a near-singular $\boldsymbol{Q}^{\nabla\Delta}$ and emits meaningless inflated ratios that trip the upper-bound guard of (69) indefinitely. A finite-variance pseudo-observation instead keeps $\boldsymbol{P}$ well-conditioned and lets subsequent epochs re-confirm the integer (asymptoting toward it) or detect a slip through its residual.

For each accepted DD pair k the constraint row, innovation and noise are

$$\boxed{\boldsymbol{H}_{(k,\,i)} = +1, \quad \boldsymbol{H}_{(k,\,p)} = -1, \qquad v_k = \check{\boldsymbol{N}}_k - \big(\boldsymbol{x}_i - \boldsymbol{x}_p\big), \qquad \boldsymbol{R} = \sigma_{\text{hold}}^2\, \boldsymbol{I},} \tag{73}$$

followed by $(\boldsymbol{x}_{\text{fix}}, \boldsymbol{P}_{\text{fix}}) = \textsc{JosephUpdate}(\boldsymbol{x}, \boldsymbol{P}, \boldsymbol{H}, \boldsymbol{v}, \boldsymbol{R})$, with $\sigma_{\text{hold}}^2 =$ `hold_variance`. The DD/baseline path uses a single-column selector $\boldsymbol{H}_{(k,\,\text{idx})} = +1$ since the corresponding state

value already *is* the double difference. Fix-and-hold mode (`estimation.ambiguity_resolution.mode` = 3) re-applies the same constraint each epoch to suppress float drift. This pseudo-observation feedback is the resolution of the mode-3 destabilisation that the earlier $R = 0$ form (72) induced; (72) survives in the code only as the documented, rejected legacy approach.

8.7 Configuration summary

All tunable thresholds live under the JSON path `estimation.ambiguity_resolution.*`, parsed in `src/core/config/GnssConfigLoad.cpp` with defaults in `include/filter/AmbiguityResolver.hpp`. The invariant estimator theory of Sections 8.1 to 8.6 carries no knobs — changing it changes the mathematics, not a policy.

Symbol	Config key (`...ambiguity_resolution.`)	Default	Role
—	`mode`	`0`	0 float / 1 continuous / 2 instantaneous / 3 fix-and-hold
τ_{ratio}	`ratio_threshold`	`3.0`	ratio-test acceptance (69)
$\tau_{\text{ratio}}^{\max}$	`ratio_upper_bound`	`1e4`	degenerate-covariance reject (69)
$P_s^{\min}$	`min_success_rate`	`0.999`	FFR gate (70)
κ_{frac}	`nl_max_per_pair_frac`	`0.15`	per-pair residual cap (71)
θ_{AR}	`elevation_mask`	`20°`	candidate elevation mask
σ^2_{hold}	`hold_variance`	`0.001` m^2	pseudo-obs $\boldsymbol{R}$ (73)
$n_{\min}^{\text{ep}}$	`min_epochs`	`10`	convergence guard
$\sigma_{\text{pos}}^{\max}$	`max_position_sigma_m`	`0.05` m	convergence guard
—	`wl_only`	`false`	MW-WL-only, no NL search
—	`enable_partial_ar`	`false`	backward-elimination subset fix (Section 8.4)
—	`active_systems`	`[]` (all)	constellation subset (Section 8.4)
—	`state_ambiguity_basis` †	`per_band`	`per_band` vs `nl_wl` dispatch

† Path is `estimation.state_ambiguity_basis` — one level above the `ambiguity_resolution` block.

Part V
Inertial aiding

9 Inertial aiding

When an IMU is configured (`imu.enabled`; the sibling boolean `estimation.aiding.imu` is the `AidingSourceManager` selector and back-compat-defaults to `imu.enabled`, so enabling the legacy flag also enables the aiding-IMU source — an explicit `estimation.aiding.imu` wins, see `src/core/config/GnssConfigLoad.cpp`), the inter-epoch time update of Section 6.1 is replaced by a first-order strapdown mechanization in the Earth-centred Earth-fixed (ECEF) frame. The mechanization itself lives in `src/core/filter/ImuIntegrator.cpp`; the live per-epoch predict routes through `Application::run` → `AidingSourceManager::collectPredict` → `src/core/aiding/ImuAidingSource.cpp`, whose `PredictContribution` is spliced into the filter covariance by `KalmanFilter::predictAided` (`src/core/filter/KalmanFilter.cpp`).

This `IAidingSource` / `AidingSourceManager` abstraction (`include/aiding/`) is the composable seam through which every non-GNSS modality — IMU, ZUPT, NHC, baro, wheel odometry, vision — contributes up to three things (a `StateManager`-owned state block, a predict-time Φ/Q sub-block, and either independent measurement rows or extra Jacobian *columns* on existing GNSS rows). The IMU is its first and privileged client; the legacy monolithic `KalmanFilter::predictImu` is retained but no longer on the live path (superseded by `predictAided`). The `ImuAidingSource` replicates `predictImu`'s arithmetic verbatim — the strapdown equations below are unchanged by the refactor.

The formulation is an *error-state* extended Kalman filter. The mechanization propagates the nominal navigation state $(\boldsymbol{r}, \boldsymbol{v}, \boldsymbol{C}_b^e)$ deterministically from the inertial samples, while a 15-element INS error vector

$$\delta\boldsymbol{x}_{\text{ins}} = \begin{bmatrix} \delta\boldsymbol{r} & \delta\boldsymbol{v} & \delta\boldsymbol{\psi} & \delta\boldsymbol{b}_a & \delta\boldsymbol{b}_g \end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^{15} \tag{74}$$

carries the linearised transition and process noise that drive the covariance. Three attitude-error states $\delta\boldsymbol{\psi}$, three accelerometer-bias states $\boldsymbol{b}_a$, and three gyro-bias states $\boldsymbol{b}_g$ augment the existing position state (and, when `estimation.estimate_velocity` is set, the three velocity states); the augmented block is allocated contiguously by `StateManager` (accessors `getAttitudeIndex`, `getAccelBiasIndex`, `getGyroBiasIndex`).

Remark 1 (Implemented scope). The block order of Equation (74) ($[\delta\boldsymbol{r}, \delta\boldsymbol{v}, \delta\boldsymbol{\psi}, \delta\boldsymbol{b}_a, \delta\boldsymbol{b}_g]$) is fixed by `ImuIntegrator`. *No* lever-arm state and *no* sensor scale-factor states are carried, and there is no IMU-to-antenna lever-arm term in the GNSS measurement model (Section 9.4). These are roadmap items, not part of the current implementation; the corresponding terms are omitted from every equation below rather than written and then ignored.

9.1 Strapdown mechanization

For each IMU sample the raw specific force $\tilde{\boldsymbol{a}}$ and angular rate $\tilde{\boldsymbol{\omega}}$ (both in the body frame) are bias-corrected with the current filter estimates,

$$\boldsymbol{f}_b = \tilde{\boldsymbol{a}} - \boldsymbol{b}_a, \qquad \boldsymbol{\omega}_b = \tilde{\boldsymbol{\omega}} - \boldsymbol{b}_g, \tag{75}$$

and integrated over the sample interval Δt. The attitude direction-cosine matrix $\boldsymbol{C}_b^e$ (body $\rightarrow$ ECEF) is updated to first order with Earth-rotation compensation,

$$\boxed{\boldsymbol{C}_b^e \;\leftarrow\; \boldsymbol{C}_b^e\big(\boldsymbol{I} + [\boldsymbol{\omega}_b]_\times\,\Delta t\big) \;-\; [\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{C}_b^e\,\Delta t,} \tag{76}$$

where $\boldsymbol{\omega}_{ie}^e = [\,0,\,0,\,\omega_e\,]^{\mathrm{T}}$ is the ECEF Earth-rotation vector ($\omega_e = 7.292\,115\,146\,7 \times 10^{-5}\,\mathrm{rad\,s^{-1}}$, `common/Constants.hpp`) and $[\cdot]_\times$ is the skew-symmetric (cross-product) matrix. To curb numerical drift, $\boldsymbol{C}_b^e$ is re-orthonormalised by singular-value decomposition every 100 samples. The velocity and position propagate as

$$\boxed{\begin{aligned} \boldsymbol{v}_{k+1} &= \boldsymbol{v}_k + \big(\boldsymbol{C}_b^e\,\boldsymbol{f}_b + \boldsymbol{g}^e(\boldsymbol{r}_k) - 2\,[\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{v}_k\big)\Delta t, && (77)\\ \boldsymbol{r}_{k+1} &= \boldsymbol{r}_k + \tfrac{1}{2}\big(\boldsymbol{v}_k + \boldsymbol{v}_{k+1}\big)\Delta t. && (78) \end{aligned}}$$

Equation (77) applies the Coriolis acceleration $-2\,[\boldsymbol{\omega}_{ie}^e]_\times\boldsymbol{v}$, while the centrifugal acceleration is folded into the gravity vector $\boldsymbol{g}^e$ (i.e. $\boldsymbol{g}^e$ is a *plumb-bob* gravity, gravitational plus centrifugal); Equation (78) uses a trapezoidal position update.

The gravity vector is a point-mass radial model plus the centrifugal term (`ImuIntegrator::computeGravityEcef`):

$$\boldsymbol{g}^e(\boldsymbol{r}) = -\frac{\mu}{\|\boldsymbol{r}\|^3}\,\boldsymbol{r} \;+\; \omega_e^2\,[\,p_x,\,p_y,\,0\,]^{\mathrm{T}}, \qquad \mu = 3.986\,005\times10^{14}\,\mathrm{m^3\,s^{-2}}. \tag{79}$$

The normal-gravity (Somigliana) flattening and the J_2 zonal term are *not* modelled; over a single GNSS epoch interval the resulting deterministic acceleration error is small and is absorbed by the estimated accelerometer bias $\boldsymbol{b}_a$.

9.2 INS error-state dynamics

Linearising Equations (76) to (78) about the nominal trajectory yields the continuous error dynamics $\dot{\delta\boldsymbol{x}}_{\mathrm{ins}} = \boldsymbol{F}_{\mathrm{ins}}\,\delta\boldsymbol{x}_{\mathrm{ins}} + \boldsymbol{G}_{\mathrm{ins}}\,\boldsymbol{w}$, with, in the block order of Equation (74),

$$\boldsymbol{F}_{\mathrm{ins}} = \begin{bmatrix} \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & -2[\boldsymbol{\omega}_{ie}^e]_\times & -[\boldsymbol{f}^e]_\times & \boldsymbol{C}_b^e & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & -[\boldsymbol{\omega}_{ie}^e]_\times & \boldsymbol{0} & -\boldsymbol{C}_b^e \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \end{bmatrix}, \qquad \boldsymbol{f}^e := \boldsymbol{C}_b^e\,\boldsymbol{f}_b. \tag{80}$$

Both sensor biases are modelled as pure random walks (zero deterministic dynamics, the null bottom rows of Equation (80)). Two terms of the textbook ECEF error model are deliberately dropped: the position-error feedback row ($[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{r},\delta\boldsymbol{r}} = -[\boldsymbol{\omega}_{ie}^e]_\times$) and the gravity-gradient coupling $[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{v},\delta\boldsymbol{r}} = -\,\partial\boldsymbol{g}^e/\partial\boldsymbol{r} - [\boldsymbol{\omega}_{ie}^e]_\times^2$. This is a first-order approximation that is accurate over the short epoch interval of a GNSS-aided run. The discrete transition is taken to first order,

$$[\boldsymbol{\Phi}_{\mathrm{ins}}]_k = \boldsymbol{I} + \boldsymbol{F}_{\mathrm{ins}}\,\Delta t. \tag{81}$$

The process noise maps the continuous accelerometer / gyro white noise and the bias random walks through

$$[\boldsymbol{Q}_{\mathrm{ins}}]_k = \boldsymbol{G}_{\mathrm{ins}}\,\boldsymbol{W}\,\boldsymbol{G}_{\mathrm{ins}}^{\mathrm{T}}\,\Delta t, \quad \boldsymbol{G}_{\mathrm{ins}} = \begin{bmatrix} \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{C}_b^e & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & -\boldsymbol{C}_b^e & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{I} \end{bmatrix}, \quad \boldsymbol{W} = \mathrm{diag}\big(\sigma_a^2, \sigma_g^2, \sigma_{ba}^2, \sigma_{bg}^2\big) \otimes \boldsymbol{I}_3, \tag{82}$$

with the spectral densities $\sigma_a, \sigma_g, \sigma_{ba}, \sigma_{bg}$ read from `imu.{accel_noise_density, gyro_noise_density, accel_bias_instability, gyro_bias_instability}` (defaults in `config/GnssConfigTypes.hpp`). Over one GNSS epoch the per-sample $([\boldsymbol{\Phi}_{\mathrm{ins}}]_k, [\boldsymbol{Q}_{\mathrm{ins}}]_k)$ accumulate by the standard recursion

$$\boldsymbol{\Phi}_{\mathrm{ins}} \leftarrow [\boldsymbol{\Phi}_{\mathrm{ins}}]_k\,\boldsymbol{\Phi}_{\mathrm{ins}}, \qquad \boldsymbol{Q}_{\mathrm{ins}} \leftarrow [\boldsymbol{\Phi}_{\mathrm{ins}}]_k\,\boldsymbol{Q}_{\mathrm{ins}}\,[\boldsymbol{\Phi}_{\mathrm{ins}}]_k^{\mathrm{T}} + [\boldsymbol{Q}_{\mathrm{ins}}]_k. \tag{83}$$

9.3 Covariance splice and attitude feedback

The accumulated 15×15 pair $(\boldsymbol{\Phi}_{\mathrm{ins}}, \boldsymbol{Q}_{\mathrm{ins}})$ is spliced (*assigned*, not accumulated) onto an identity-seeded full-state transition $\boldsymbol{F}$ and onto the random-walk process-noise baseline $\boldsymbol{Q}$ over the indices the INS owns; the non-INS states (clock, troposphere, ionosphere, ambiguities, …) retain their standard random-walk model from Section 6.1. The covariance is then propagated once and symmetrised,

$$\boxed{\boldsymbol{P} \leftarrow \boldsymbol{F}\,\boldsymbol{P}\,\boldsymbol{F}^{\mathrm{T}} + \boldsymbol{Q}, \qquad \boldsymbol{P} \leftarrow \tfrac{1}{2}\big(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}}\big).} \tag{84}$$

The nominal navigation states $(\boldsymbol{r}, \boldsymbol{v})$ are overwritten by the mechanised values of Equations (77) to (78), and the attitude error is reset, $\delta\boldsymbol{\psi} := \mathbf{0}$, because the rotation increment has already been applied directly to $\boldsymbol{C}_b^e$ in Equation (76). After the subsequent GNSS measurement update the corrected attitude error feeds back multiplicatively into the DCM and is again reset,

$$\boldsymbol{C}_b^e \leftarrow \big(\boldsymbol{I} + [\delta\boldsymbol{\psi}]_\times\big)\boldsymbol{C}_b^e, \qquad \delta\boldsymbol{\psi} := \mathbf{0}, \tag{85}$$

followed by SVD re-orthonormalisation (`KalmanFilter::update`). This closed-loop reset keeps $\delta\boldsymbol{\psi}$ a small-angle quantity throughout, so the linearisation in Equation (80) stays valid.

9.4 Coupling modes and the GNSS update

How the GNSS observations correct the inertial errors depends on the coupling mode (`imu.coupling_mode`).

Loosely coupled (default). The GNSS code and carrier-phase Jacobians of Section 4.3 touch only the position (and velocity) columns. The INS error states $\delta\boldsymbol{\psi}, \boldsymbol{b}_a, \boldsymbol{b}_g$ therefore receive *no direct* observation partials; they are corrected only indirectly, through the off-diagonal cross-covariances that Equation (84) builds between the position/velocity block and the attitude/bias block. The attitude correction is then realised by the feedback of Equation (85).

Tightly coupled. In the tightly-coupled mode the *Doppler* observation alone gains direct attitude and accelerometer-bias columns. This is the one aiding contribution that re-parameterizes existing GNSS rows rather than appending its own: each relinearisation pass calls `AidingSourceManager::applyTcJacobianInputs` → `ImuAidingSource::applyTcJacobianInputs`, which feeds the current attitude and specific force to `MeasurementModel::setImuState` (`src/core/measurement/MeasurementModel.cpp`). With $\boldsymbol{f}^e = \boldsymbol{C}_b^e \boldsymbol{f}_b$ the specific force in ECEF and $\boldsymbol{e}_r^i$ the unit line-of-sight of Equation (7), the Doppler measurement function h_D contributes

$$\boxed{\frac{\partial h_\mathrm{D}}{\partial \delta\boldsymbol{\psi}} = -\,[\boldsymbol{f}^e]_\times^\mathrm{T}\,\boldsymbol{e}_r^i, \qquad \frac{\partial h_\mathrm{D}}{\partial \boldsymbol{b}_a} = -\,(\boldsymbol{C}_b^e)^\mathrm{T}\,\boldsymbol{e}_r^i.} \tag{86}$$

The specific force fed into Equation (86) is a static-rest approximation $\boldsymbol{f}_b \approx (\boldsymbol{C}_b^e)^\mathrm{T}\big(-\boldsymbol{g}^e\big) - \boldsymbol{b}_a$ (`ImuAidingSource.cpp`); it is exact only when the platform's true acceleration is negligible against gravity. The *code and carrier-phase rows carry no attitude or bias partials in either mode* — only Doppler couples directly to the inertial errors.

Remark 2 (Relinearisation under tight coupling). The tightly-coupled path retains the legacy single *naive* second-pass relinearisation rather than the prior-anchored iterated EKF of Section 6.3: re-deriving the gain from a fixed prior conflicts with the multiplicative attitude/bias feedback of Equation (85), which has already mutated $\boldsymbol{C}_b^e$ between passes. The aiding layer exposes this as a capability query (`AidingSourceManager::anySourceRequiresNaiveRelinearization`, which the IMU source answers true for in tight coupling), so any future nominal-feedback source forces the same path; the migration off the direct `tightlyCoupledImu()` checks in `Application.cpp` / `KalmanFilter.cpp` is still in progress.

Part VI
Real-time SSR corrections

10 Real-time SSR corrections

The real-time engine consumes network corrections in *State Space Representation* (SSR): per-satellite, per-signal refinements to the broadcast orbit, clock, and hardware biases, streamed as RTCM 3.x messages (legacy 1057–1068 / 1240–1270) or as their IGS-SSR `4076` repackaging [3, 8]. Each SSR quantity refines *exactly one* term already present in Equation (4) and Equation (5) of the Foundations document; none introduces a new state. This is what separates the precise real-time mode from broadcast-only operation: with SSR applied, $c\,\mathrm{d}t^i$ and $\boldsymbol{r}^i$ are centimetre-grade and the satellite biases B^i_s, b^i_s are *known* rather than absorbed into the ambiguity. The corrections in this document are the rover-side *consumer*: the same on-disk store is the sink for both a third-party IGS-RTS stream and the engine's own network estimator (Section 12), so the application math below is identical regardless of who produced the corrections.

Decoded records are cached per satellite in `products/SsrCorrectionStore` (a process-wide singleton for the live stream, or a caller-owned instance for the network producer's closed-loop test); the consumers (`model/ModelGeometry`, `corrections/GnssCorrections`, `measurement/MeasurementModel`) query the store and fall through to broadcast-only when it is empty, so every equation below collapses to the broadcast-only observation model in post-processing. The streaming and decoding plumbing — RTCM frame parse, the per-satellite mutex-guarded cache, the real-time push loop — belongs to Section 11; this document covers only the store-driven correction math, which is invariant to how records arrived. There is no `enable_ssr` flag: SSR application is implicit and store-driven.

10.1 Orbit correction — refines ρ^i_r

The orbit message carries a delta in the satellite's radial / along-track / cross-track (RAC) frame, anchored to the broadcast ephemeris of issue IODE. The frame is built from the broadcast position $\boldsymbol{r}^i_{\mathrm{brdc}}$ and velocity $\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}$ (`products/SsrCorrectionStore.cpp`):

$$\hat{\boldsymbol{e}}_a = \frac{\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}}{\|\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}\|}, \quad \hat{\boldsymbol{e}}_c = \frac{\boldsymbol{r}^i_{\mathrm{brdc}} \times \dot{\boldsymbol{r}}^i_{\mathrm{brdc}}}{\|\boldsymbol{r}^i_{\mathrm{brdc}} \times \dot{\boldsymbol{r}}^i_{\mathrm{brdc}}\|}, \quad \hat{\boldsymbol{e}}_r = \hat{\boldsymbol{e}}_a \times \hat{\boldsymbol{e}}_c. \tag{87}$$

The RAC delta is propagated to the query epoch with its rate term and rotated into ECEF. With $\delta t = t - t_{\mathrm{SSR}}$ the age of the correction, $\boldsymbol{\delta}_{\mathrm{rac}} = (\delta_r, \delta_a, \delta_c)^{\mathrm{T}}$ its components and $\dot{\boldsymbol{\delta}}_{\mathrm{rac}}$ their rate,

$$\boxed{\boldsymbol{r}^i \;=\; \boldsymbol{r}^i_{\mathrm{brdc}} \;-\; \big[\,\hat{\boldsymbol{e}}_r \;\; \hat{\boldsymbol{e}}_a \;\; \hat{\boldsymbol{e}}_c\,\big]\big(\boldsymbol{\delta}_{\mathrm{rac}} + \dot{\boldsymbol{\delta}}_{\mathrm{rac}}\,\delta t\big).} \tag{88}$$

The *subtraction* follows the RTCM sign convention $\boldsymbol{\delta}$ = broadcast − precise (`products/SsrCorrectionStore.cpp`). The corrected $\boldsymbol{r}^i$ then enters $\rho^i_r = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line-of-sight $\boldsymbol{e}^i_r$ of the Foundations document unchanged: the orbit correction never touches an observation equation directly, only the geometry (Section 4) that feeds it. The delta is applied only when its IODE matches that of the broadcast ephemeris the baseline orbit was evaluated from (Equation (91)); on a live cutover the baseline position is deliberately re-referenced to the ephemeris *version* the SSR delta was issued against rather than dropped to broadcast grade (`model/ModelGeometry.cpp`, via `LiveEphemerisStore::computeSatellitePV` with the SSR IODE). Because the clock is queried against the same broadcast ephemeris, the broadcast clock's *radial* component cancels consistently

against Equation (90). The decoded RAC scale factors are fixed by RTCM 10403.3 (radial δ_r a 22-bit two's-complement field at 0.1 mm, along/cross at 0.4 mm, rates at 1 and 4 μm/s respectively; `rtcm/RtcmSsrDecoder.cpp`).

10.2 Clock correction — refines $-c\,\mathrm{d}t^i$

The clock message carries a quadratic polynomial in time-from-epoch, optionally augmented by a high-rate additive delta δC_{hr}. The total satellite-clock correction, in metres, is

$$\Delta C^i(t) \;=\; C_0 + C_1\,\delta t + C_2\,\delta t^2 + \delta C_{\mathrm{hr}}, \qquad \delta t = t - t_{\mathrm{SSR}}, \tag{89}$$

converted to seconds and *added* to the baseline clock (`products/SsrCorrectionStore.cpp`):

$$\boxed{c\,\mathrm{d}t^i \;=\; c\,\mathrm{d}t^i_{\mathrm{base}} \;+\; \Delta C^i(t),} \tag{90}$$

where $\mathrm{d}t^i_{\mathrm{base}}$ is the precise CLK sample (post-processing) or the broadcast clock polynomial (live rooftop mode, `products/LiveEphemerisStore`). This term enters both undifferenced observation equations through $-c\,\mathrm{d}t^i$, identically on code and phase. The high-rate clock is additive *on top of* the polynomial, not a replacement (`corrections/GnssCorrections.cpp`); the C_2 field is a 27-bit two's-complement quantity at 2×10^{-8} m/s^2.

10.3 IOD gate at correction-set boundaries

Orbit and clock corrections are deltas against a *specific* broadcast ephemeris, so they are applied only when the SSR record's IODE matches that of the broadcast ephemeris from which the baseline orbit/clock was evaluated:

$$\text{apply} \iff \big(\mathrm{IODE}_{\mathrm{brdc}} < 0\big) \;\vee\; \big(\mathrm{IODE}_{\mathrm{SSR}} < 0\big) \;\vee\; \big(\mathrm{IODE}_{\mathrm{brdc}} = \mathrm{IODE}_{\mathrm{SSR}}\big), \tag{91}$$

the gate being permissive when either side is unknown (`SsrCorrectionStore::iodeMatches`, `products/SsrCorrectionStore.hpp`). A combined orbit+clock record (RTCM 1060/1066, IGS-SSR sub-type 3) is preferred over separate messages because it guarantees a single shared IODE (`products/SsrCorrectionStore.cpp`). At a correction-set boundary — the provider issuing a new ephemeris reference, hence a new IODE — the gate (91) simply declines the mismatched record until the matching broadcast ephemeris is in hand, avoiding a discontinuous jump in $\boldsymbol{r}^i$ or $c\,\mathrm{d}t^i$. Records older than `max_age_s` (default 60 s, the IGS-RTS update interval) are treated as absent.

10.4 Satellite biases — refine $-B^i_s$ and $-b^i_s$

SSR delivers *observable-specific biases* (OSB): one value per (satellite, RINEX-3 signal) pair, in metres. They are applied as a *known* additive correction to the modelled observable, with no state and no Jacobian row (`measurement/MeasurementModel.cpp`). The code OSB refines B^i_s in the undifferenced code equation:

$$\boxed{P^i_{r,s} = \rho^i_r + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t^{(g)}_r + T^i_r + \mu_s I^i_{r,1} + B_{r,s} \underbrace{-\,B^i_s}_{\text{SSR OSB}} + \varepsilon^P,} \tag{92}$$

and the phase OSB refines b^i_s in the undifferenced phase equation:

$$\boxed{\Phi^i_{r,s} = \rho^i_r + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t^{(g)}_r + T^i_r - \mu_s I^i_{r,1} + \lambda_s N^i_{r,s} + b_{r,s} \underbrace{-\,b^i_s}_{\text{SSR OSB}} + \varepsilon^\Phi.} \tag{93}$$

The bias decoded from the stream is the satellite bias *present in* the observation, so it is removed from the clean model prediction with a +1 sign convention (`kSsrCodeBiasSign` = `kSsrPhaseBiasSign` = +1, `measurement/MeasurementModel.cpp`); the predicted observable accumulates it into the predicted observable as `ssr_bias_term` (`MeasurementModel.cpp`), summed alongside the receiver clock, troposphere, ionosphere, estimated code/phase biases and ambiguity. Biases are queried per *raw* signal only — an ionosphere-free composite key such as `C1W_C5X` returns zero (code) or `nullopt` (phase) and is skipped, consistent with the uncombined parameterisation. The code OSB is a 14-bit field at 0.01 m, the phase OSB a 20-bit field at 0.1 mm (`rtcm/RtcmSsrDecoder.cpp`). The application path is *ungated* by any feature flag: `ssr_bias_term` loads whenever the store carries the exact RINEX signal key, the corollary of which — avoiding a redundant estimated bias on the same row — is the subject of the next subsection.

10.5 Satellite-bias STATE gate vs. SSR code biases

The uncombined model can also *estimate* a per-(satellite, signal) code-bias state (`estimation.estimate_sat_biases`; see Section 3). That estimated state and a known SSR code OSB both enter the code row with unit coefficient and opposite physical roles, so allowing both for the same (i, s) would be doubly redundant: the stream already fixes the satellite code bias, and a free state collinear with it is near-unobservable and merely soaks up noise. State allocation therefore consults *both* correction sources before creating a sat-bias state (`MeasurementModel::ensureRequiredStates`):

$$\text{create sat-bias}(i,s) \iff \neg\,\text{hasBIA}(i,s) \;\wedge\; \neg\,\text{hasSsrCB}(i,s) \;\wedge\; \text{pivotCovers}(i,s), \tag{94}$$

where hasBIA tests the static `.BIA` product (`PppModel::hasBiasEntry`) and hasSsrCB tests the live stream (`SsrCorrectionStore::hasCodeBias`, surfaced as `ssrCoversCodeSignal`). Two design points make the gate correct:

- **Presence, not magnitude.** `hasCodeBias` is an exact-key *presence* test on the per-signal map, distinct from `getCodeBias` $\neq 0$: a provider's present-but-zero entry still defines that signal's datum frame, so the estimator must not re-estimate it. The two queries are required to agree exactly so a covered signal is never both estimated *and* SSR-corrected on one row.
- **Per-signal, not global.** IGS streams are partial (e.g. Galileo E1 covered, E5a/E5b not). The gate keys on the exact (i, s) pair, so a signal the stream does *not* carry keeps its estimated state — its only available code correction — while a covered sibling on the same satellite is deferred to the OSB. A strict per-epoch prune (`pruneSatBiasStatesExcept`) drops the state when SSR coverage arrives mid-arc and re-creates it (warm-restarted from the stashed cache) if coverage later disappears.

In post-processing the live store is empty, so hasSsrCB is uniformly false and the gate reduces to the static-`.BIA` test alone.

Network-producer exemption. When the engine *is* the network estimator (`ProcessingMode::PPP_RTK_Network`), the roles invert: it *estimates* the satellite code/phase biases as states to emit as a product (Section 12). Importing SSR biases there would double-count and render those very states unobservable, so both the `ssr_bias_term` application path (92) and the hasSsrCB branch of the state gate (94) are forced off on the producer side (`ssrCoversCodeSignal` returns false). The producer's own emitter (`products/NetworkProductEmitter.cpp`) then pushes the estimated clock and code/phase biases into a *separate* store instance, closing the loop against a held-out user consumer.

Integer recoverability via consistent phase biases. Removing b_s^i is precisely what restores the integer property of $N_{r,s}^i$ promised in the phase observation equation of the Foundations document. Once the satellite phase bias is known, the residual non-integer part of the ambiguity is the receiver phase bias $b_{r,s}$ *alone*, which cancels under between-satellite single differencing $\Delta\cdot$ (Section 5), leaving an integer single-difference ambiguity admissible to AR. This is the operational content of a *consistent* phase-bias product: the provider must generate b_s^i on the same network datum whose integer property the rover then exploits. The phase-bias message therefore carries three AR-critical metadata fields per signal (`rtcm/RtcmTypes.hpp`): an integer-preservation indicator, a two-bit wide-lane integer indicator, and a four-bit *discontinuity counter*.

Discontinuity handling. An incremented discontinuity counter signals that the provider's phase bias jumped between correction sets, breaking the integer connection of that arc. The engine treats this as an ambiguity-arc boundary: the affected $N_{r,s}^i$ is reset before the Kalman update (`realtime/RealTimeEngine.cpp`, via `StateManager::noteSsrDiscontinuity`), so the post-jump phase is re-initialised rather than fixed against a stale integer offset. This is the phase-bias analogue of the orbit/clock IOD gate (91): both protect state continuity across correction-set boundaries, one by declining a mismatched delta, the other by resetting the dependent ambiguity.

10.6 GLONASS

GLONASS SSR is decoded and stored on the same footing as the CDMA constellations: the legacy block (orbit 1063, clock 1064, code bias 1065, combined 1066, URA 1067, high-rate clock 1068, phase bias 1266) and the IGS-SSR `4076` subtypes in the GLONASS message-indicator band (IM = 41–47) both map onto the identical orbit/clock/bias structs (`rtcm/RtcmSsrDecoder.cpp`). The orbit, clock and bias *equations* above apply unchanged; nothing in Equations (88), (90), (92) and (93) is constellation-specific.

The one GLONASS-specific subtlety is on the *time tag*, not the correction math. GLONASS SSR carries its content epoch as a time-of-day field (DF389, seconds since the start of the GLONASS UTC(SU)+3 h day) rather than a GPS time-of-week. The decoder promotes it to the GPS scale through `TimeUtils::glonassTodToGpsTime` (`rtcm/RtcmSsrDecoder.cpp`); without that promotion the SSR epoch would land at $\{0, 0\}$, the freshness check (91)'s sibling `max_age_s` gate would declare every GLONASS record stale, and the satellite would silently fall back to broadcast grade. Once promoted, the per-record content epoch drives $\delta t = t - t_{\mathrm{SSR}}$ in Equations (88) and (90) exactly as for GPS. Whether the resulting GLONASS observable then *contributes* to a between-satellite-differenced solution is a separate, estimator-side opt-in (its FDMA per-channel receiver delay does not cancel in SD); see Section 5.

10.7 Ionosphere — $\mu_s I_{r,1}^i$ is not SSR-fed

For completeness, and to forestall a natural misreading of the term: although the SSR standard defines a global VTEC spherical-harmonic message (IGS-SSR IM = 201) and regional STEC products, *neither is decoded by this engine*. The IGS-SSR classifier rejects IM = 201 as unsupported (`rtcm/RtcmSsrDecoder.cpp`), and there is no SSR path that writes the slant-iono state $I_{r,1}^i$. Consequently the ionospheric term $\mu_s I_{r,1}^i$ is informed only by (i) the per-satellite iono *state* estimated by the filter itself and (ii) the optional GIM/IONEX deterministic prior (`atmosphere/GlobalIonosphereMap`, an independent VTEC→STEC mapping unrelated to the SSR stream). This is a *consumer-side* gap, not an absence of the network workstream: the network estimator (Section 12) and its product emitter (`products/NetworkProductEmitter.cpp`) now exist and round-trip satellite clock and

code/phase biases through the same store, but they emit no ionosphere product. Closing the loop on the atmosphere — a regional STEC overlay feeding $I^i_{r,1}$ as a pseudo-observation, restoring the iono-weighted constraint that makes single-epoch AR feasible — is the remaining unimplemented piece on both sides.

10.8 Provenance and code pointers

Both transports map onto the same internal correction structs (`rtcm/RtcmTypes.hpp`) through a single decoder (`rtcm/RtcmSsrDecoder.cpp`). Legacy RTCM uses per-constellation six-message blocks (orbit, clock, code bias, combined orbit+clock, URA, high-rate clock) plus a separate phase-bias block (1265–1270); IGS-SSR `4076` repackages the same payloads under a message-indicator (IM) whose *tens* place ($\lfloor \mathrm{IM}/10 \rfloor$) selects the constellation (GPS $2x$, GLONASS $4x$, Galileo $6x$, QZSS $8x$, BeiDou $10x$, SBAS $12x$) and whose *ones* place (IM mod 10) selects the subtype (1=orbit, 2=clock, 3=combined, 4=high-rate, 5=code bias, 6=phase bias, 7=URA) — note this per-block subtype ordering differs from the legacy SSR block ordering, so the legacy mapping is not reused (`RtcmSsrDecoder.cpp`). VTEC (IM 201) and NavIC (IM 141–147) are rejected by the message-indicator classifier *before* the header is parsed, so they return `ok = false` with an empty header — this is the decode-side reason the ionospheric term carries no SSR contribution.

The invariant correction definitions of Equations (88), (90) and (91) live in the static, deterministic `applyOrbitToPosition` / `applyClockToBias` / `iodeMatches` members of `products/SsrCorrectionStore`, which is the seam this document documents. The stateful streaming layer — the mutex-guarded per-satellite cache with wall-clock freshness gating, the `RealTimeEngine::feedRtcmFrame` push loop, and the obs↔SSR synchronisation that bounds how far the obs stream may run ahead of lagging corrections — belongs to Section 11 and is kept separate so the correction math can be reasoned about (and tested) in isolation. The one user-facing SSR-related estimator config is the warm-up gate `estimation.defer_until_corrections_ready` (`config/GnssConfigLoad.cpp`), which defers estimation until a solvable floor of satellites is broadcast-plus-SSR centimetre-grade, so the filter seeds code and phase together rather than cold-starting a contaminated partition on metre-grade broadcast clocks.

Part VII
Real-time engine

11 Real-time engine

The real-time path reuses the same estimator, measurement model and state manager as post-processing, but reorganises the control flow around a *streaming* contract: complete RTCM 3 (or SBF) frames arrive one at a time over a network socket, are decoded into observations / corrections / broadcast ephemeris, and a buffered epoch is pushed through the Section 6 predict/update cycle the instant its boundary is detected. Where post-processing iterates a closed list of epochs in time order with the entire orbit / clock product set already on disk, the engine sees each epoch exactly once, with corrections that may lead or lag the observations, and can never look ahead. The entry point is `Application::runRealTime` (`src/core/app/ApplicationRealtime.cpp`), which constructs a `RealTimeEngine` (`src/core/realtime/RealTimeEngine.cpp`) and drives it from a single-threaded multi-source poll loop. This section documents the parts that are *specific* to that path; the estimation mathematics, observation equations and SSR correction model are documented elsewhere and only

cross-referenced here.

11.1 Streaming control flow

The poll loop holds one `RealtimeStream` per logical source and services them round-robin without ever blocking on a single one:

- a *primary* stream carrying rover observations (RTCM 3 MSM or Septentrio SBF, from a file replay, an NTRIP mountpoint, or a raw TCP socket);
- an optional *corrections* stream (typically an IGS-RTS NTRIP mountpoint carrying SSR);
- an optional *ephemeris* stream (broadcast nav for an obs-only station that does not co-broadcast 1019/1020/...).

Each iteration drains a bounded burst from every active source through its framer and dispatches complete frames to the engine; the primary obs drain is byte-capped per iteration so a high-rate observation stream cannot starve the sparse-but-critical SSR / nav streams (`pumpFrames`). The loop exits when every source goes inactive (file EOF or socket teardown), when a cooperative stop is requested (Ctrl+C / console-close handler), or when the `processing.max_duration_s` data-time budget latches; on exit the final buffered epoch is flushed before teardown so no observations or trace rows are lost.

A socket source is treated as *active* while its client intends to run, not merely while momentarily connected, so a transient drop or the brief pre-connect window does not end the session prematurely; the client reconnects underneath.

11.2 NTRIP transport

Caster connection. `NtripClient` (`src/core/realtime/NtripClient.cpp`) is an NTRIP v2 client built on libcurl. A background thread holds the HTTP GET open and pushes received bytes into a mutex-guarded ring buffer with a hard cap (4 MiB); the buffer applies back-pressure by dropping the oldest bytes when full. Consumers pull through a blocking `read` or a timeout-bounded `tryRead` (the latter is what the multi-stream loop uses so one stream cannot block the others). The request carries the NTRIP v2 headers (`Ntrip-Version:Ntrip/2.0`, a `User-Agent`); the mountpoint is the URL path; HTTP Basic authentication (`CURLAUTH_BASIC`) is supplied when a username is configured, and an optional NMEA GGA sentence is POSTed for VRS mountpoints.

Transport security. The transport defaults to plain HTTP. Plain-HTTP casters (IGS-IP, `igs-ip.net`, SBF replay) work out of the box, byte-identically. A `use_tls` flag selects the `https` URL scheme and forces `CURLUSESSL_ALL`, which is required for port-443 casters such as CDDIS that offer no plain-HTTP endpoint. Certificate verification (`SSL_VERIFYPEER`/`VERIFYHOST`) stays on unless a testing-only `tls_insecure` override disables it for self-signed or mis-chained casters. Both flags default off; a GNSS-only / IGS-IP run is unaffected.

Reconnection. The receive loop reconnects automatically. After any transfer ends it classifies the outcome — transport error, a genuine non-200 HTTP status (with actionable hints for 401 / 403 / 404), a zero-byte idle mountpoint, or a clean stream end — logs it, and always backs off for `reconnect_delay_s` before retrying, so a misconfigured mountpoint surfaces as an error rather than spinning invisibly. A non-200 *ICY* response (NTRIP v1 casters answer "ICY 200 OK", reported by libcurl as code 0) is treated as success.

11.3 RTCM 3 framing and CRC

`RtcmStreamReader` (`src/core/realtime/RtcmStreamReader.cpp`) turns an arbitrary byte stream into discrete RTCM 3 frames, decoupling transport from parsing. An RTCM 3 frame is

$$\underbrace{\texttt{0xD3}}_{\text{preamble}}\ \underbrace{[\,6\text{ reserved}\,|\,10\text{-bit length }L\,]}_{2\text{ bytes}}\ \underbrace{[\,\text{payload, }L\text{ bytes}\,]}\ \underbrace{[\,\text{CRC-24Q}\,]}_{3\text{ bytes}}, \tag{95}$$

so the total frame length is $3 + L + 3$. `nextFrame` scans for the `0xD3` preamble (discarding non-preamble bytes), reads the 10-bit payload length, waits until the full $3 + L + 3$ bytes are buffered, and verifies the trailing 24-bit Qualcomm CRC (CRC-24Q, table-driven) computed over header+payload. A CRC mismatch advances past the candidate preamble byte and keeps scanning — a corrupt or misframed byte never wedges the reader. Only CRC-valid frames are emitted, so the downstream parser (`RtcmParser`) may assume frame integrity. The reader tracks frames-extracted and bytes-discarded counters for stream-health diagnostics.

The top-level `RtcmParser` (`src/core/rtcm/RtcmParser.cpp`) reads the 12-bit message type DF002 from the *first 12 bits of the payload* (not the header) and dispatches: 1005/1006 station coordinates, 4076 IGS-SSR, legacy SSR (1057–1068, 1240–1270), broadcast ephemeris (1019/1020/1042/1044/1046), and MSM4 / MSM7 observations. The week needed to promote the various truncated epoch fields to a full `GnssTime` is taken from the rover's wall-clock GPS week (Section 11.6).

11.4 MSM observation decode

MSM observation messages are decoded by a single body (`decodeMsmImpl`, `src/core/rtcm/RtcmMsmDecoder.cpp`) parameterised over an `MsmVariant` descriptor that supplies the per-field bit widths, scale factors, sentinels and capability flags. Two variants are emitted, MSM4 and MSM7 (MSM5/MSM6 are described by the same variant mechanism, but no variant entry is instantiated for them, so they are not currently produced). The header carries a 64-bit satellite mask, a 32-bit signal mask, and an $N_{\text{sat}} \times N_{\text{sig}}$ cell mask; the number of cells is $N_{\text{cell}} = \text{popcount(cell mask)}$. Each field block is packed contiguously across all satellites or all cells, in mask order, and decoded into parallel vectors before recombination.

Range reconstruction. For each satellite the rough range in milliseconds is assembled from the 8-bit integer (DF397) and the 10-bit modulo (2^{-10} ms, DF398) parts, and each cell adds a signed fine term scaled per variant. Writing c for the speed of light, the pseudorange and carrier phase for a cell are

$$P = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\,2^{-10} + n^{P}_{\text{fine}}\,\sigma_P\Big), \tag{96}$$

$$\Phi_{[\text{m}]} = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\,2^{-10} + n^{\Phi}_{\text{fine}}\,\sigma_\Phi\Big), \qquad \Phi_{[\text{cyc}]} = \Phi_{[\text{m}]}/\lambda, \tag{97}$$

with fine scales $\sigma_P = 2^{-24}$, $\sigma_\Phi = 2^{-29}$ for MSM4 and $\sigma_P = 2^{-29}$, $\sigma_\Phi = 2^{-31}$ for MSM7 (all in ms). RTCM-defined sentinel values mark invalid cells and are skipped. Carrier phase is emitted in cycles, matching the RINEX-3 convention consumed by the observation model (Section 4); when the wavelength λ cannot be resolved the phase is carried in metres as a fallback. CNR (DF408/DF403) and, for MSM5/7, Doppler from the combined rough+fine phase-range-rate are also produced; the half-cycle-ambiguity bit sets the LLI half-cycle flag.

DF419 extended-satellite-info gating. A subtle framing hazard governs the per-satellite block. The 4-bit DF419 "extended satellite information" field is present in the bitstream for *every* constellation in MSM5/MSM7, not only GLONASS; only GLONASS *assigns it meaning* (the FDMA frequency-channel number). The decode therefore separates two concerns:

- *bitstream consumption* is gated on the MSM variant (`honour_ext_sat_info`, true for MSM5/MSM7, false for MSM4/MSM6) — when set, $4\,N_{\text{sat}}$ bits are read for all systems;
- *interpretation* is gated separately on the system descriptor; only GLONASS maps the value to a channel.

This is the fix for the GPS L1/L2 geometry-free sign anomaly: reading DF419 only for GLONASS (or omitting it for MSM7) under-consumes the per-satellite block by $4\,N_{\text{sat}}$ bits, so the per-cell fine pseudoranges are read from the wrong bit offset. The corruption preserves the rough range but biases the fine terms, producing an L1/L2 split of order hundreds of metres with the wrong sign on the geometry-free combination — plausible-looking but wrong observations that survive a naive over-read check. The gate matches RTCM 10403.3 and the cssrlib reference. A complementary framing guard in the parser (`warnIfUnderConsumed`) warns whenever a decode leaves ≥ 8 unread payload bits, catching exactly this class of silent misalignment that the bit reader's over-read flag cannot.

11.5 Broadcast nav, SSR and station coordinates

Broadcast ephemeris. Messages 1019 (GPS LNAV), 1044 (QZSS), 1046 (Galileo I/NAV), 1042 (BeiDou D1) and 1020 (GLONASS) are decoded by `RtcmNavDecoder` into a common `BroadcastEphemeris` and pushed into the process-wide `LiveEphemerisStore`, which is what lets the geometry layer resolve satellite orbits with no SP3/CLK file loaded. The per-message field order differs by constellation (e.g. QZSS leads with its clock block); GLONASS carries a sign-magnitude PZ-90 state integrated separately.

SSR corrections. Both the legacy SSR ranges and the IGS-SSR 4076 container are decoded by `RtcmSsrDecoder` and pushed into the shared `SsrCorrectionStore` carrying the real SSR header (provider epoch and IOD-SSR) so staleness and IOD diagnostics reflect the provider solution rather than wall-clock receipt. Orbit, clock, combined, code-bias, phase-bias, URA and high-rate-clock records are stored per satellite; the orbit / clock query paths consult the store transparently, so no per-epoch wiring is needed. The correction *mathematics* — the radial/along/cross orbit projection, the clock and bias application, the iono model — is documented in Section 10 and is not repeated here. Two real-time-specific reactions to the stream are worth noting:

- an incremented IGS-SSR phase-bias *discontinuity counter* for a signal breaks the integer property of that signal's ambiguity, so the engine resets the affected ambiguity before the next update (`StateManager::noteSsrDiscontinuity`);
- every SSR push re-attempts a held observation epoch (Section 11.7), since the freshest correction horizon has just advanced.

Station coordinates (1005/1006). `decodeStationCoords` reads the antenna reference point (ARP) as three signed 38-bit ECEF components in 0.1 mm units; 1006 additionally carries a 16-bit antenna height. For a reference station that streams its own coordinates this provides an a-priori seed (Section 11.9).

11.6 Time-system handling

Streaming messages carry truncated, system-specific epoch fields that must be promoted to a common GPS-time `GnssTime`. The anchor for every promotion is the rover's wall-clock GPS time, `TimeUtils::nowContinuousGpsSeconds` (POSIX UTC plus the leap offset), from which the current GPS week is derived (`currentGpsWeekFromSystem`).

- **Flat ms-of-week systems** (GPS/GAL/QZS/SBAS/BDS MSM): the 30-bit epoch is milliseconds-of-week; BeiDou adds the constant 14 s BDT→GPS offset (with a week-rollover guard).
- **GLONASS** carries its 30-bit MSM epoch as a 3-bit day-of-week plus a 27-bit time-of-day in ms — not 27 bits total; reading it short misaligns the satellite mask and corrupts the whole GLONASS decode. The ms-of-day is promoted by `TimeUtils::glonassTodToGpsTime`, which carries the reported time-of-day through the GLONASS = UTC(SU)+3 h ("Moscow") day frame and back to GPS time, anchoring on the rover's current GPS seconds and re-adding leap seconds:

$$t_{\mathrm{GPS}} = \left(t^{0}_{\mathrm{day}} + \tilde{t}_{\mathrm{TOD}} \right) - \Delta_{\mathrm{Moscow}} + \Delta_{\mathrm{leap}}, \tag{98}$$

  where t^{0}_{day} is the Moscow-day midnight nearest the anchor and $\tilde{t}_{\mathrm{TOD}}$ is the reported time-of-day placed into that day by resolving the midnight wrap toward the nearest match (± 12 h). The redundant 3-bit day-of-week is left unread. The same conversion promotes the 1020 ephemeris `toe` and any GLONASS SSR epoch, so observations, orbits and corrections share one clock.
- **GLONASS FDMA wavelength.** GLONASS L1/L2 are frequency-division: each slot transmits on $f = f_{\mathrm{base}} + k\,\Delta f$ with channel $k \in [-7, +6]$, so the carrier wavelength is per-satellite. The channel is decoded from DF419 as $k = v - 7$ for the valid range and used in Equation (97); it is also carried on the observation so the slot map and MW combinations can resolve frequency. When DF419 is absent (MSM4/MSM6) or reserved the phase is carried in metres rather than fabricating a bogus $k = -7$.

Each decoded record stamps its own content epoch (`epoch_time`), which is what the synchronisation logic of Section 11.7 compares against, rather than wall-clock arrival.

11.7 Observation/SSR synchronisation

PPP-RTK needs observations time-*aligned* with the corrections, not pinned to wall-clock. In a live session the obs stream commonly runs ahead of a lagging SSR stream; processing an obs epoch that no satellite's SSR can cover would drive the whole epoch broadcast-grade (and, with `drop_broadcast_grade`, wipe it). The opt-in `processing.enable_obs_ssr_sync` hold buffer guards against this. Let t_{obs} be the obs epoch and define the SSR coverage horizon from the freshest stored SSR content epoch (`SsrCorrectionStore::newestSsrEpoch`) plus the maximum SSR age:

$$t_{\mathrm{horizon}} = t^{\mathrm{newest}}_{\mathrm{SSR}} + \tau_{\max}, \qquad \tau_{\max} = \texttt{ssr_max_age_s}. \tag{99}$$

The decision is:

- $t_{\mathrm{obs}} \le t_{\mathrm{horizon}}$: SSR covers the epoch — process normally and clear any hold;
- $t_{\mathrm{obs}} > t_{\mathrm{horizon}}$: *defer* the epoch. The held buffer keeps the latest such epoch only (skip-to-latest semantics), and `drainHeldObs` re-attempts it the moment an SSR push advances the horizon.

A liveness backstop prevents a dead SSR feed from starving the filter forever: if a hold persists past `obs_ssr_sync_max_hold_s` the engine latches a degraded "outage" and passes every epoch through

(broadcast-grade / dropped via the existing path) until SSR re-covers the obs. The held epoch is drained with skip-to-latest *bypassed* so the intended SSR-aligned target is not discarded, and the final `flush` bypasses both the hold and skip-to-latest so the last epoch is never lost. The whole feature is inert when disabled or in any non-live-SSR run.

A separate, coarser `enable_obs_skip_to_latest` drops obs epochs whose lag behind *wall-clock* GPS time exceeds a bound, so a backlogged stream fast-forwards to the freshest epoch; the skip returns before any continuity state is touched, so the resume correctly trips the gap path (Section 11.10).

11.8 Ephemeris management

`LiveEphemerisStore` retains, per satellite, the current record and one previous record, and resolves the orbit/clock against whichever is appropriate. Two policies matter in real time.

- **Freshest-toe selection.** An incoming record is rejected when its `toe` (expressed in GPS-continuous seconds, `toeGpsContinuous`, so BDT/GLONASS scales are reconciled) is older than the held record beyond a sub-second epsilon, so a stale or re-ordered broadcast never overwrites a newer one. An authoritative IGS-stream record is additionally protected from being clobbered by a receiver-decoded record at equal `toe`: receiver nav is accepted only as a genuine update or as a fallback once the IGS stream has stalled past a threshold (the GLONASS nominal `toe` step).
- **toe-aware / IOD-aware pivoting.** For orbit and clock evaluation the store picks the record whose `toe` is nearest the query epoch (`pick`); for an SSR-corrected satellite it first re-references to the record the SSR delta was computed for (`pickForSsr`: match the SSR IODE against the current or retained-previous record). Across an ephemeris cutover the receiver can be one IOD ahead of the SSR stream; without this re-reference the IOD mismatch would needlessly drop the satellite to broadcast-grade.

11.9 A-priori seeding from RTCM 1005/1006

A live SBF/RTCM observation stream carries no RINEX `APPROX POSITION` record. Without a finite a-priori the first `computeModels` rejects every satellite on a position-sanity check (almost all sit below the elevation mask when the seed is the default Earth-centred fallback at the geographic pole), so WLS never runs and the filter cannot bootstrap. The engine therefore seeds the filter a-priori from the stream itself until the Kalman filter takes ownership of the position:

- from a *PVTGeodetic* solution in an SBF stream (`feedSbfFrame`); and
- from an RTCM *1005/1006* station ARP (`feedRtcmFrame`), the RTCM analogue of the PVT seed, which makes `receiver.reference_position` optional for any station streaming 1005/1006.

A configured `reference_position` always wins. The seed is applied only while `first_epoch_` holds; the startup-defer gate keeps that flag set through warmup, so a 1005/1006 message arriving a few seconds into the stream (these come less often than the 1 Hz obs) still seeds before the first estimation epoch. Once the first epoch is fully processed the flag clears and the filter owns the position — the ARP / PVT is thereafter only logged for comparison, not re-applied.

11.10 The per-epoch real-time update

`processEpoch` runs the same predict/update pipeline as post-processing, with streaming-specific guards layered on the front and the post-processing-only stages removed. In order:

1. **Lag / sync / duration guards.** The wall-clock backlog warning, the obs↔SSR hold of Section 11.7, the skip-to-latest drop, and the `max_duration_s` latch all run before any continuity state is touched.
2. **Step / gap detection.** $\Delta t = t_k - t_{k-1}$ (with a week-rollover correction). A non-advancing $\Delta t \leq 0$ is a duplicate (a co-timestamped second SBF block or a retransmit) and is skipped cleanly — folding it in would double-count the measurement and feed $\Delta t = 0$ into the process-noise build, which hard-throws. A Δt beyond the gap tolerance flags a gap.
3. **Model + startup gate.** Observations are filtered to the active systems and run through `computeModels`. When `defer_until_corrections_ready` is armed, estimation is deferred (no predict/update) until a per-constellation cm-grade quorum of broadcast+SSR satellites is present (or a timeout forces a degraded start), so the filter seeds code and phase together on a clean state instead of cold-starting on metre-grade broadcast clocks.
4. **Clock / gauge seeding.** In modes with a receiver-clock state a WLS clock estimate re-seeds the filter clock on a jump, gap, or first epoch. Under BSSD (no receiver-clock state) a single per-epoch median-of-code-innovations receiver clock, common to all satellites, is subtracted from every code and phase observable — an exactly SD-invariant re-gauge that keeps the per-satellite bias states at DCB scale rather than absorbing the full unsteered receiver clock.
5. **Predict.** `predictAided` over $(t_{k-1}, t_k]$ — identical to the shared aiding path; for a GNSS-only stream it delegates byte-identically to `predict`(Δt) of Section 6.
6. **Cycle-slip detection / repair.** The MW detector plus the optional TurboEdit MW+GF joint repair and soft-LLI gap bridge run pre-update, sharing the same detectors and repair machinery as Section 7. These are enabled under `estimation.cycle_slip`; TurboEdit is high-rate-only and seeds its GF-arc gap tolerance from the configured stream rate (a live stream has no RINEX interval header).
7. **Measurement update.** The same outlier / forced-pivot-handover rebuild loop as the post-processing driver (shared via `EpochUpdateSupport`): rebuild on `NeedsMeasurementRebuild`, forward each constellation's forced pivot handover, re-inflate reset-satellite states, clear MW/TurboEdit arc history for KF-reset signals, and peel one outlier per pass. An optional soft a-priori ZWD pseudo-observation anchors the BSSD [up, ZWD] near-null subspace.
8. **Ambiguity resolution.** After a successful float update, MLAMBDA AR runs on the committed state with the same configuration surface as post-processing, fed the MW-resolved wide-lane integers (Section 8).
9. **State extraction + trace.** The posterior is synced back to the PPP state and trace rows are written.

Post-processing stages skipped in real time. The non-causal and batch stages have no place in a streaming, look-ahead-free pipeline and are deliberately absent:

- the backward RTS Section 6 is force-disabled under `AppMode::RealTime` — smoothing requires the full forward trajectory and cannot run causally;
- geometry relinearisation (the IEKF second pass) is a post-processing-only refinement and is not replicated here;
- precise SP3/CLK products are not loaded when a live broadcast+SSR source supplants them (a precise-product *file* replay still loads them);

- higher-order ionosphere is unsupported in the streaming pipeline (no GIM source) and is warned-and-skipped if enabled.

Part VIII
Network mode

12 Network mode

This section documents the multi-station network estimator (`ProcessingMode::PPP_RTK_Network`) that *produces* the satellite-side corrections the single-receiver user modes consume. Where every other mode in this reference *differences away* the satellite clock and hardware biases (Section 5) or imports them as known SSR quantities (Section 10), the network producer turns that relationship around: it estimates the satellite clock and the satellite *phase* and *code* biases *explicitly*, jointly across a network of stations whose coordinates are known, and emits them in IGS-SSR convention for a downstream consumer. The closed loop is deliberate — the same single-receiver PPP-RTK engine documented elsewhere becomes the product test, run on a station *held out* of the network solve.

Scope and status. The numerical core — the arrowhead normal-equation kernel (`filter/NetworkNormalEquations`) and the semantic estimator that drives it (`filter/NetworkEstimator`) — is implemented and validated on a synthetic noise-free network (truth recovery up to the declared S-transform). Mode wiring is live: the `PPP_RTK_Network` enum, its forced-uncombined `DifferencingMode::None` (`src/core/measurement/MeasurementBuilder.cpp`), the explicit satellite phase-bias state in `filter/StateManager`, and the dispatch in `Application::run` all exist. The live epoch loop `Application::runNetwork` — per-station design-row assembly from each station's `MeasurementModel`, the between-epoch time update, and the product emitter — is a scaffold at the time of writing: it logs a "not yet implemented" error and returns non-zero (`src/core/app/ApplicationNetwork.cpp`). The mathematics below describes the implemented estimator core and the locked design; where a piece is staged rather than wired, it is flagged as such.

12.1 Joint multi-station observation model

The producer runs on the same *undifferenced, uncombined* observations as single-receiver PPP (Section 4), but stacks the equations of every station $r \in \mathcal{R}$ into one system. The defining structural fact is the partition of the unknowns into a block *shared* by all stations and a block *private* to each:

- **Shared (satellite-side).** The estimable satellite clock $c\,\mathrm{d}t^i$, the per-signal satellite code bias B^i_s, and the per-signal satellite *phase* bias β^i_s. These columns are *identical* across every station — a satellite's clock and hardware delays do not depend on who is observing it. (Orbits are held fixed to a precise product here; orbit determination is out of scope for the producer.)
- **Private (station-side).** For each station r: its receiver clock $c\,\mathrm{d}t_r$, the inter-system time offsets $c\,\delta t_r^{(g)}$, the zenith wet troposphere ZWD_r (and gradients), the per-satellite slant ionosphere $I^i_{r,1}$, the receiver biases, and one float ambiguity $N^i_{r,s}$ per tracked carrier. Station coordinates are *known* (the network constrains them), so position is not a free parameter.

Write the shared satellite-parameter vector as $\boldsymbol{x}_s$ and station r's private nuisance vector as $\boldsymbol{x}_r$. The undifferenced observation rows of station r linearise (Section 4) to

$$\boldsymbol{y}_r \;=\; \boldsymbol{A}_{s,r}\,\boldsymbol{x}_s \;+\; \boldsymbol{A}_{r,r}\,\boldsymbol{x}_r \;+\; \boldsymbol{\varepsilon}_r, \qquad r \in \mathcal{R}, \tag{100}$$

where $\boldsymbol{y}_r$ is the prefit residual (observed minus computed), $\boldsymbol{A}_{s,r}$ is the Jacobian with respect to the *shared* satellite columns, and $\boldsymbol{A}_{r,r}$ is the Jacobian with respect to station r's *own* nuisances. Crucially there is no $\boldsymbol{A}_{r,r'}$ term for $r' \neq r$: a station's observations touch only the shared block and its own private block. That sparsity is the structural engine of Section 12.4.

No receiver-side differencing. Unlike the PPP-RTK user mode, the producer applies *no* between-satellite or between-receiver differencing. The differencing strategy is forced to `DifferencingMode::None` (`MeasurementBuilder.cpp`), overriding any `between_satellite_sd` toggle that may be present in a network config. Differencing would cancel exactly the satellite-side terms the producer is trying to *estimate*; the network instead carries them as explicit columns and resolves the resulting rank deficiency with a minimum-constraint datum (Section 12.3).

12.2 The explicit satellite phase-bias state

The single defining contrast with the user side is the treatment of the satellite phase bias b_s^i (here written β_s^i once promoted to a state). On the user side this term is never an explicit unknown: between-satellite single differencing (Section 5) cancels the *receiver* phase bias, the satellite phase bias arrives as a *known* SSR product, and what remains is an integer single-difference ambiguity. The user handles biases *implicitly*.

The producer is what *generates* that known product, so it must carry the satellite phase bias as a genuine estimated state. The state vector therefore gains a per-(satellite, signal) phase bias column,

$$\beta_s^i \in \boldsymbol{x}_s, \qquad \text{keyed by } \texttt{makeSatPhaseBiasKey}(i, s) = \texttt{"}i\texttt{|}s\texttt{|PB"}, \tag{101}$$

held in `StateManager::sat_phase_bias_indices_`, a map kept *deliberately disjoint* from the satellite *code* bias map (`sat_bias_indices_`); the trailing `|PB` discriminator keeps the two namespaces from colliding (`include/filter/StateKeySchema.hpp`). Its process noise is a tiny random walk (`process_noise.sat_phase_bias`, default two orders of magnitude below the code-bias density), reflecting that satellite phase hardware delays are nearly constant but not strictly so: a $q = 0$ random walk would over-converge the diagonal of $\boldsymbol{P}$ and leave the column deaf to genuine slow drift (`src/core/filter/StateManager.cpp`).

Remark 3. The estimated β_s^i is not the "raw" satellite phase hardware delay — that quantity is not separately estimable. It is the delay *lumped with the network pivot-receiver ambiguity/clock datum* (Odijk's δ^s, [6]): the estimable combination that, when removed by a user receiver, restores the integer property of that user's single-difference ambiguities. This lumping is exactly why a *consistent* phase-bias product matters (Section 10): the producer's datum and the consumer's integer recovery must be the same S-system.

12.3 Rank deficiency and the minimum-constraint S-basis

The undifferenced network system Equation (100) is rank deficient. The same linear dependencies that the user side disposes of by differencing are present here as exact *gauge freedoms* of the joint normal matrix: there is no observation that distinguishes a common shift of all satellite clocks from an opposite shift of all receiver clocks, nor the split of an absolute phase bias between the

satellite column and the per-station ambiguity datum, nor an inter-system bias (ISB) common to a constellation. These are precisely the S-system ambiguities of [11]: the data fix only *estimable combinations*, and a *datum* must be chosen to pin the remaining null space.

Exact gauge, soft elimination. The chosen resolution is a *minimum-constraint* datum applied as exact hard pins to the reduced shared system, never a hard structural elimination of columns. Concretely, each gauge is realised as a single *Dirichlet pin* of one shared column c to a datum value (almost always 0): the column's known contribution is moved to the right-hand side, its row and column in the reduced normal matrix are zeroed, and a diagonal entry is set so that $x_c = \text{value}$ holds exactly (`NetworkEstimator::solve`, `src/core/filter/NetworkEstimator.cpp`). The pin is applied to a *copy* of the reduced normals (the kernel's `solveSharedConstrained` seam), leaving the estimator's own reduced system pristine so per-station back-substitution (Section 12.4) stays valid. The pinned diagonal is set commensurate with the reduced normals' own scale rather than to unity, so that a genuinely *under*-specified datum still surfaces as a near-zero *relative* pivot in the positive-definiteness guard, independent of the network's overall weighting magnitude.

In S-transform language, if $\boldsymbol{x}$ collects the rank-deficient undifferenced parameters and $\boldsymbol{S}$ is the S-basis selection, the emitted satellite corrections are the estimable combination $\boldsymbol{S}\,\boldsymbol{x}$; choosing the datum *is* choosing $\boldsymbol{S}$, and the hard pins above are the minimal set of rows of $\boldsymbol{S}$ that remove the deficiency. The two halves of the datum live in two places:

- **Shared-block gauges** — the overall clock datum (one per constellation), the per-constellation phase-bias↔ambiguity datum, and the ISB datum — are the elements that touch the shared satellite columns. These go through the explicit `NetworkEstimator::Datum` (a list of column pins) and are applied in `solve`.
- **Local datum elements** — a network *pivot receiver*'s own ambiguity/clock pin, which the emitted satellite phase bias absorbs (Odijk Eq. 4/9) — live on that one station's private block. They are realised by the *caller* as tight prior rows folded into that station's $\boldsymbol{A}_{r,r}$, $\boldsymbol{W}$, and $\boldsymbol{y}$, not through the shared `Datum` (`NetworkEstimator.hpp`). Station priors are also what make each local block $\boldsymbol{N}_{rr}$ full rank for the Schur step (Section 12.4).

Remark 4. A rigorous null-space rank gate — SVD the pre-constraint reduced normals and assert that the count of small singular values equals the *declared* S-basis dimension — is the planned W5 slice (`NetworkSbasis`) and is not yet present. The `Datum` in the current estimator is the minimal pin set the solve needs; the positive-definiteness guard catches the exact-deficiency (under-specified datum) case but does not yet certify the rank against a declared dimension. This is recorded as an open item rather than claimed as done.

12.4 Per-station Schur reduction (arrowhead normals)

Stacking Equation (100) over all stations and forming weighted normal equations $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{A}\,\boldsymbol{x} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{y}$ yields a *bordered block-arrow* ("arrowhead") system, because the station–station off-diagonal blocks are exactly zero (stations couple only through $\boldsymbol{x}_s$):

$$\begin{bmatrix} \boldsymbol{N}_{ss} & \boldsymbol{N}_{s1} & \boldsymbol{N}_{s2} & \cdots & \boldsymbol{N}_{sR} \\ \boldsymbol{N}_{1s} & \boldsymbol{N}_{11} & \boldsymbol{0} & \cdots & \boldsymbol{0} \\ \boldsymbol{N}_{2s} & \boldsymbol{0} & \boldsymbol{N}_{22} & \cdots & \boldsymbol{0} \\ \vdots & & & \ddots & \\ \boldsymbol{N}_{Rs} & \boldsymbol{0} & \boldsymbol{0} & \cdots & \boldsymbol{N}_{RR} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_s \\ \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_R \end{bmatrix} = \begin{bmatrix} \boldsymbol{u}_s \\ \boldsymbol{u}_1 \\ \boldsymbol{u}_2 \\ \vdots \\ \boldsymbol{u}_R \end{bmatrix}, \tag{102}$$

with the per-station blocks built from that station's pre-formed design rows,

$$\boldsymbol{N}_{ss}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{A}_{s,r}, \quad \boldsymbol{N}_{sr}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{A}_{r,r}, \quad \boldsymbol{N}_{rr}^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{A}_{r,r}, \tag{103}$$

and $\boldsymbol{u}_s^r = \boldsymbol{A}_{s,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{y}_r$, $\boldsymbol{u}_r^r = \boldsymbol{A}_{r,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{y}_r$. A full (non-diagonal) $\boldsymbol{W}_r$ is supported, so correlated undifferenced measurement covariance (e.g. shared pre-corrections) propagates exactly (`addStationObservations`, `NetworkNormalEquations.hpp`).

Lossless marginalisation. The arrowhead structure is solved by Schur-eliminating each station's private block onto the shared border. Each $\boldsymbol{N}_{rr}^{r}$ is factorised once ($\mathrm{LDL}^{\mathrm{T}}$) when the station is added; the elimination accumulates

$$\tilde{\boldsymbol{N}}_{ss} = \sum_{r \in \mathcal{R}} \left(\boldsymbol{N}_{ss}^{r} - \boldsymbol{N}_{sr}^{r} \left(\boldsymbol{N}_{rr}^{r}\right)^{-1} \boldsymbol{N}_{sr}^{r\mathrm{T}} \right) + \boldsymbol{N}_{ss}^{\mathrm{prior}}, \tag{104}$$

$$\tilde{\boldsymbol{u}}_{s} = \sum_{r \in \mathcal{R}} \left(\boldsymbol{u}_{s}^{r} - \boldsymbol{N}_{sr}^{r} \left(\boldsymbol{N}_{rr}^{r}\right)^{-1} \boldsymbol{u}_{r}^{r} \right) + \boldsymbol{u}_{s}^{\mathrm{prior}}, \tag{105}$$

the reduced shared system $\tilde{\boldsymbol{N}}_{ss}\, \boldsymbol{x}_s = \tilde{\boldsymbol{u}}_s$ (`NetworkNormalEquations::reduce`). The reduction is a *lossless* marginalisation, not an approximation: solving Equations (104) and (105) for $\boldsymbol{x}_s$ and back-substituting per station,

$$\boldsymbol{x}_r = \left(\boldsymbol{N}_{rr}^{r}\right)^{-1} \left(\boldsymbol{u}_r^r - \boldsymbol{N}_{sr}^{r\mathrm{T}}\, \boldsymbol{x}_s \right), \tag{106}$$

reproduces the dense stacked solve to machine precision. Fill-in is confined to the satellite border and each station's elimination is independent, so the cost scales linearly in the station count and the per-station eliminations parallelise (`reconstructStation`, `NetworkNormalEquations.cpp`). The shared-block priors $\boldsymbol{N}_{ss}^{\mathrm{prior}}$ in Equation (104) — soft physical priors and any additive exact-gauge contributions — are added pre-reduction via `addSharedInfo` and carry through the Schur complement unchanged.

Remark 5. This arrowhead-with-exact-reduction topology mirrors the NGS M-PAGES `NgsLsq::NormalEquations::reduce` / `reconstructLocals` batch network solver, specialised so that the M-PAGES "global" block is the shared satellite border and the per-epoch swept block becomes the R independent station blocks. The design panel rejected a two-stage covariance-form alternative precisely because the normal-equation Schur form is exact and cheaper here.

12.5 Constellation scope

The producer estimates GPS and Galileo, with BeiDou supported on the same footing; **GLONASS is excluded**. The reason is structural, not a tuning choice: GLONASS is FDMA, so each satellite transmits on a slightly different frequency and the receiver inter-channel (hardware) bias does *not* cancel cleanly across satellites the way it does for the CDMA constellations. The satellite/receiver phase-bias bookkeeping that underpins the explicit-bias S-system (Sections 12.2 and 12.3) assumes a per-signal bias common across a constellation's satellites; the per-frequency GLONASS inter-channel bias breaks that assumption and would corrupt the datum. GLONASS is therefore omitted from the network solve by design (`docs/ssystem_conformance_2026-06-15.md`); the user side excludes it from ambiguity resolution for the same reason. (This is distinct from the float-only GLONASS support the user engine carries for positioning; the network *producer* simply does not generate GLONASS bias products.)

12.6 The closed loop: producer → held-out user

The emitted product is one `NetworkSolutionEpoch` per content epoch: per-satellite clock C_0 plus per-signal code and phase biases, keyed by RINEX satellite id and serialised in IGS-SSR convention by `NetworkProductEmitter` (`NetworkEstimator::toSolutionEpoch`). That product is exactly the SSR stream the single-receiver engine already knows how to consume: the satellite clock refines $-c\,\mathrm{d}t^i$ and the observable-specific biases refine $-B^i_s$ and $-b^i_s$ as *known* corrections, restoring the integer ambiguity property exactly as described in Section 10.

The validation strategy follows directly from this symmetry. The companion `ProcessingMode::PPP_RTK_User` mode runs the ordinary uncombined PPP-RTK pipeline but consumes the network's own bias/atmosphere product. Holding one station *out* of the network solve and then positioning it as a user against the produced corrections turns the consumer into a closed-loop product test: the held-out station's recovered position and fixed ambiguities certify that the producer's S-system datum and the consumer's integer recovery are mutually consistent. The `PPP_RTK_User` consumer flows through the standard PPP path (it is not a separate estimator); only the producer takes the joint multi-station pipeline of this section.

13 Worked example: estimability in a two-receiver network

This section makes the rank deficiency and minimum-constraint datum of Section 12.3 concrete on the smallest network that still shows the structure: two receivers $\mathcal{R} = \{A, B\}$ and four satellites $i \in \{1, 2, 3, 4\}$. We strip everything that is not essential to the estimability argument — positions are known so the geometric range is removed, a single frequency is used, the troposphere and ionosphere are absent, and the data are noise-free — so that every number below is *exact* and the gauge freedoms are visible by inspection. The point is not the size of the system but its null space.

13.1 Code only: clocks and one gauge freedom

With geometry removed, the single-frequency pseudorange of Section 4 collapses to a difference of two clock lumps,

$$P^i_r \;=\; a_r \;-\; g^i, \qquad a_r := c\,\mathrm{d}t_r + B_r, \quad g^i := c\,\mathrm{d}t^i + B^i, \tag{107}$$

where a_r is the receiver clock and g^i the satellite clock. On a single frequency the receiver and satellite *code biases* B_r, B^i are inseparable from the respective clocks, so each "clock" here is already a clock + code-bias lump — the IF-vs-DCB split of Section 12.2 degenerates to nothing with one band. There are $|\mathcal{R}| + |\{i\}| = 2 + 4 = 6$ unknowns and $2 \times 4 = 8$ observations.

Take as ground truth (metres)

$$a_A = 5, \quad a_B = 3, \qquad g^1 = 2, \quad g^2 = -1, \quad g^3 = 4, \quad g^4 = 0.5,$$

which through Equation (107) produces the eight pseudoranges

P^i_r	$i=1$	$i=2$	$i=3$	$i=4$
$r=A$	3	6	1	4.5
$r=B$	1	4	−1	2.5

The deficiency, seen two ways. *(i) Invariance.* Add any constant κ to *all six* unknowns, $a_r \mapsto a_r + \kappa$ and $g^i \mapsto g^i + \kappa$; the difference $a_r - g^i$, and therefore every observation, is unchanged.

The truth above and the shifted set $\{15, 13, 12, 9, 14, 10.5\}$ fit the data identically. Absolute clocks are unknowable; only differences are.

(ii) Singular normals. Ordering the unknowns $\boldsymbol{x} = [\,a_A, a_B, g^1, g^2, g^3, g^4\,]^{\mathrm{T}}$ and using unit weights, the normal matrix $\boldsymbol{N} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$ is

$$\boldsymbol{N} = \begin{bmatrix} 4 & 0 & -1 & -1 & -1 & -1 \\ 0 & 4 & -1 & -1 & -1 & -1 \\ -1 & -1 & 2 & 0 & 0 & 0 \\ -1 & -1 & 0 & 2 & 0 & 0 \\ -1 & -1 & 0 & 0 & 2 & 0 \\ -1 & -1 & 0 & 0 & 0 & 2 \end{bmatrix}, \qquad \boldsymbol{N}\,\mathbf{1} = \mathbf{0}, \tag{108}$$

which is 6×6 but has rank 5: the all-ones vector $\mathbf{1} = [1, 1, 1, 1, 1, 1]^{\mathrm{T}}$ is an exact null vector (every row sums to zero), and it is precisely the common-shift direction of view (i). The system $\boldsymbol{N}\boldsymbol{x} = \boldsymbol{u}$ cannot be solved without one added constraint — a *datum*.

Two datum choices, same fit. Pinning one clock removes the gauge. Two natural choices give *different* parameter values that fit the eight observations *equally and exactly*:

datum (S-basis)	a_A	a_B	g^1	g^2	g^3	g^4
$a_A \equiv 0$ (reference receiver)	0	−2	−3	−6	−1	−4.5
$g^1 \equiv 0$ (reference satellite)	3	1	0	−3	2	−1.5

Under $a_A \equiv 0$, every satellite-clock estimate comes out as $\hat{g}^i = g^i - a_A^{\mathrm{true}}$: the reference receiver's clock is *absorbed* into all of them. The estimated "satellite clock" is not the physical clock; it is the physical clock plus the datum we chose — the *lumping* that the user must add back. This is exactly why a producer must *declare* its datum (Section 12.6).

What is invariant. The estimable quantities — combinations annihilated by the null vector $\mathbf{1}$, i.e. differences — are identical in both bases:

estimable quantity	$a_A \equiv 0$	$g^1 \equiv 0$	(truth)
$a_A - a_B$	2	2	2
$g^2 - g^1$	−3	−3	−3
$g^3 - g^1$	2	2	2
$a_B - g^4$	2.5	2.5	$(= P_B^4)$

Only the absolute labels are convention; the differences are physics. This is the S-system of [11] in six parameters.

Remark 6 (Why the producer cannot difference). Form the double difference of this code-only model, $\nabla\Delta P_{AB}^{i,1} = (P_A^1 - P_A^i) - (P_B^1 - P_B^i)$. The between-satellite step gives $g^i - g^1$ for *both* receivers, so the between-receiver step yields exactly $\mathbf{0}$. Double differencing annihilates the entire model, because DD is built to delete clocks and here clocks are all there is (Section 5). A DD user would estimate nothing; the network producer instead *keeps* the clocks and pins one datum. That degeneracy is the producer's reason for existence in miniature.

13.2 Adding carrier phase: the satellite phase bias needs the network

Now add the single-frequency carrier phase. With the same clock lumps it reads

$$\Phi_r^i \;=\; a_r - g^i \;+\; b^i \;+\; N_r^i, \tag{109}$$

where b^i is the satellite phase bias — the producer's target — and N_r^i is the float ambiguity (in metres). The receiver phase bias and the receiver code-minus-phase offset are constant per receiver and have been absorbed into N_r^i; we keep b^i explicit because it is what the producer must emit. Extend the truth with

$$b^1 = 0.10, \qquad b^2 = -0.20, \qquad b^3 = 0.05, \qquad b^4 = 0.15,$$
$$N_A^i : 10,\ -5,\ 20,\ 8, \qquad N_B^i : 12,\ -3,\ 18,\ 9,$$

giving the eight carrier-phase observations

Φ_r^i	$i=1$	$i=2$	$i=3$	$i=4$
$r=A$	13.10	0.80	21.05	12.65
$r=B$	13.10	0.80	17.05	11.65

Counting: one receiver vs. two. The clocks are fixed by the code, so per satellite the phase contributes $b^i + N_r^i$. With *one* receiver this is a single equation $b^i + N_A^i$ in two unknowns: the satellite phase bias has no separate existence at all — it is inseparable from that one receiver's ambiguity, and not even redundant. With *two* receivers there are two equations,

$$b^i + N_A^i, \qquad b^i + N_B^i, \tag{110}$$

in three unknowns (b^i, N_A^i, N_B^i): rank 2, one deficiency per satellite. The estimable two-dimensional space now contains the *between-receiver single-difference ambiguity* $\nabla N_{AB}^i = N_A^i - N_B^i$ — a quantity that simply does not exist with one receiver, and is exactly the integer a user resolves (Section 5). The second receiver is what gives b^i an identity shared across stations.

Datum and solution. The joint code + phase system has a 1-dimensional clock gauge (as before) plus one phase-bias↔ambiguity gauge per satellite: $b^i \mapsto b^i + c_i$, $N_r^i \mapsto N_r^i - c_i$ for all r. The minimum-constraint datum is the clock pin $a_A \equiv 0$ (a shared-block gauge, Section 12.3) plus the reference-receiver ambiguity pins $N_A^i \equiv 0$ for every i (the local datum on the pivot receiver, folded into station A's rows). Solving gives

	$i=1$	$i=2$	$i=3$	$i=4$
emitted phase bias $\bar{b}^i = b^i + N_A^i$	10.10	−5.20	20.05	8.15
$\hat{N}_A^i$ (datum)	0	0	0	0
$\hat{N}_B^i = N_B^i - N_A^i$	2	2	−2	1

(the clocks are unchanged from Section 13.1). One checks the fit directly, e.g. $\hat{\Phi}_B^3 = a_B - g^3 + \bar{b}^3 + \hat{N}_B^3 = -2 - (-1) + 20.05 + (-2) = 17.05$, matching the observation.

Reading the result. Two things are worth dwelling on.

- **The emitted bias is dominated by the lump.** The physical satellite phase bias of satellite 1 is $b^1 = 0.10$ m, but the *emitted* estimable bias is $\bar{b}^1 = 10.10$ m: it has absorbed receiver A's ambiguity (10 m) for that satellite. This is Odijk's $\bar{b}^i = b^i + (\text{pivot-receiver ambiguity})$ (Section 12.2, Eq. 4/9) made arithmetic. A consumer recovers integers only by removing the *same* lumped quantity — producer datum and consumer integer recovery must be one S-system.
- **The network unlocks the integer.** The estimable $\hat{N}^i_B = N^i_B - N^i_A = \nabla N^i_{AB}$ is the between-receiver single-difference ambiguity, the very quantity an RTK user fixes to an integer. It is identically absent in a single-receiver problem; the second station is what brings it into existence. The satellite phase bias becomes a definite, transferable number for the price of pinning one reference ambiguity — and that pin is what it carries.

This four-satellite, two-receiver instance is, up to the geometry and the second frequency it suppresses, exactly the synthetic network the estimator core is validated on: the `RecoversSyntheticTruthUpToSTransform` test (`tests/test_network_estimator.cpp`) builds these very rows, pins the same datum through `NetworkEstimator::solve`, and asserts recovery of the satellite-side parameters *up to the declared S-transform* — i.e. recovery of the lumped $\bar{b}^i$ and the single-difference ambiguities above, not of the unobservable physical b^i.

Part IX
Reference

14 Pipeline and architecture diagrams

This section sketches the control- and data-flow structures that the prose chapters describe algebraically: the per-epoch estimation loop (Figure 1), the shared geometry kernel behind the three differencing regimes (Figure 2), the real-time SSR correction pipeline (Figure 3), the real-time streaming front-end that feeds it (Figure 4), and the PPP-RTK network producer's joint estimator (Figure 5). Boxes name the function or class that realises each step, so the diagrams double as a map into `src/`.

A Generated reference tables

The tables in this appendix are generated from the source tree by `scripts/gen_report_tables.py`. They are not maintained by hand: a renamed flag, a new cycle-slip detector, or a changed state-key delimiter appears here on the next run of the generator. Run the generator with `--check` in CI to fail the build if the committed tables are stale.

A.1 Processing modes

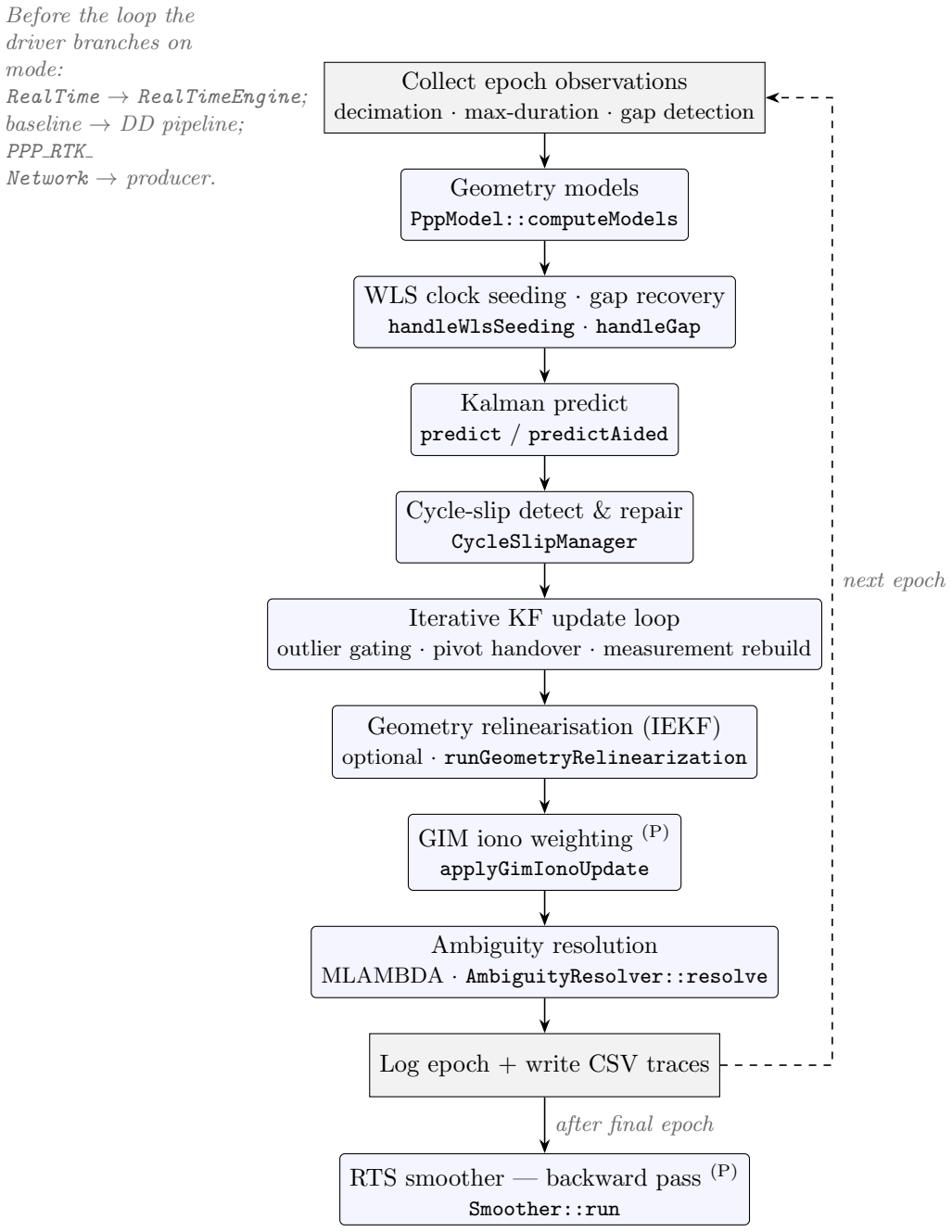


Figure 1: Per-epoch post-processing loop in `Application::run`. Solid arrows are the intra-epoch sequence; the dashed arrow is the epoch-to-epoch feedback. Steps tagged (P) run in post-processing only and are skipped on the real-time path (`RealTimeEngine`). The smoother runs once after the forward pass completes.

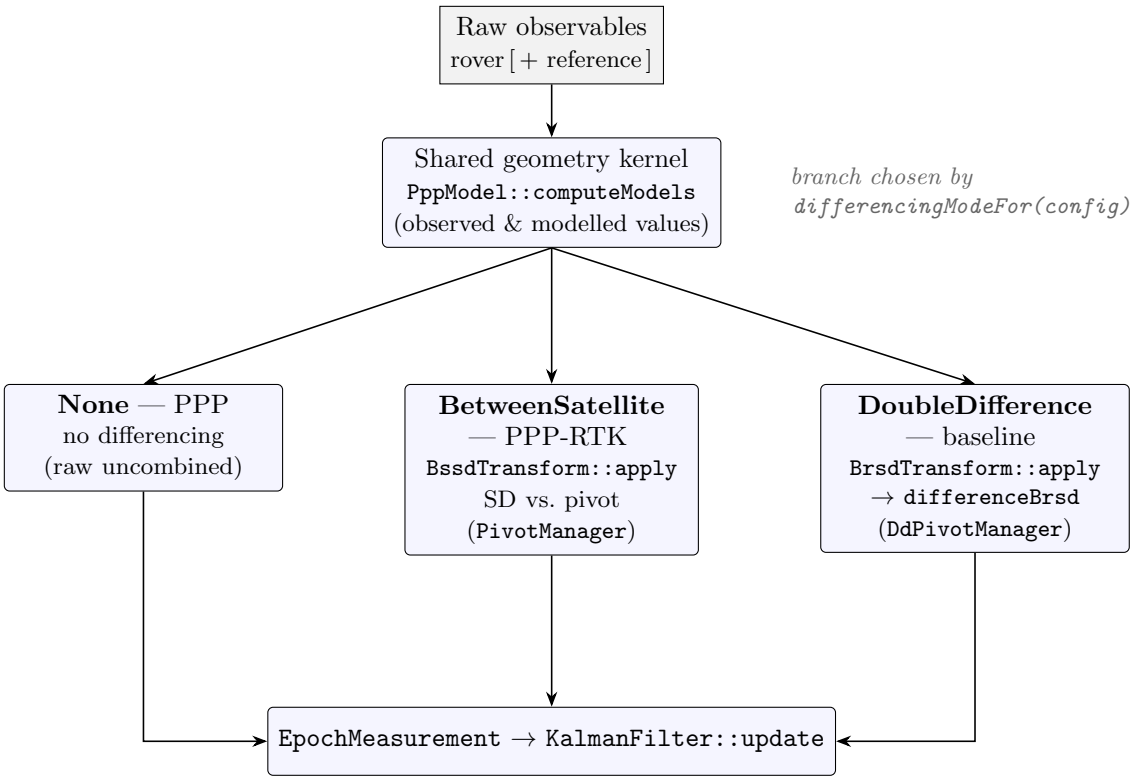


Figure 2: The three differencing regimes share one geometry kernel. Undifferenced residuals are computed once; the strategy selected by `differencingModeFor` then applies no transform (PPP), a between-satellite single difference against a pivot (PPP-RTK), or a between-receiver → between-satellite double difference (baseline) before the common Kalman update.

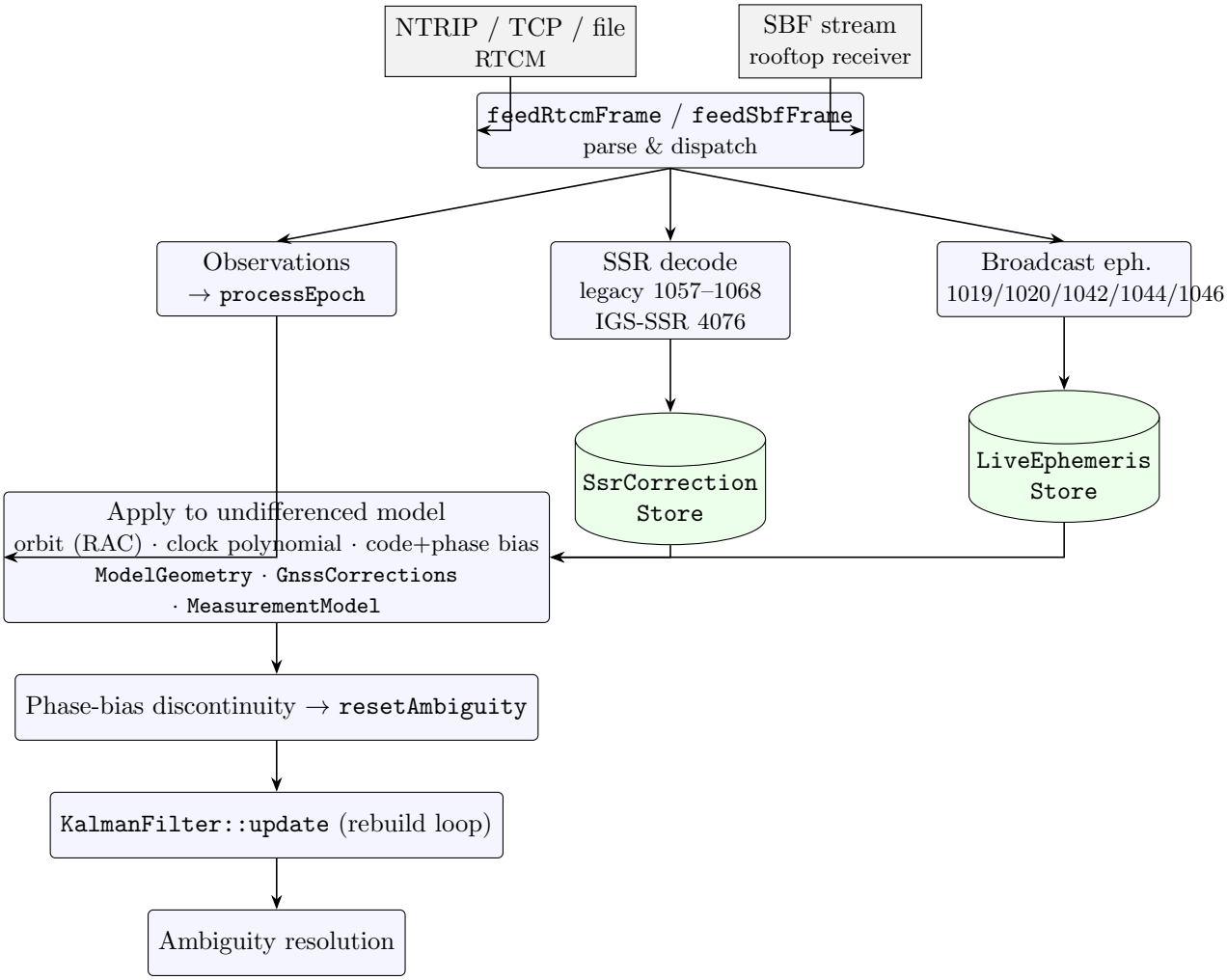


Figure 3: Real-time SSR correction pipeline. Observations, SSR corrections, and broadcast ephemerides are decoded from the byte stream into two stores; the undifferenced observation model then overlays the precise orbit (RAC frame), clock polynomial, and code/phase biases before the Kalman update. A phase-bias discontinuity counter forces an ambiguity reset ahead of the update. See `RealTimeEngine::processEpoch`.

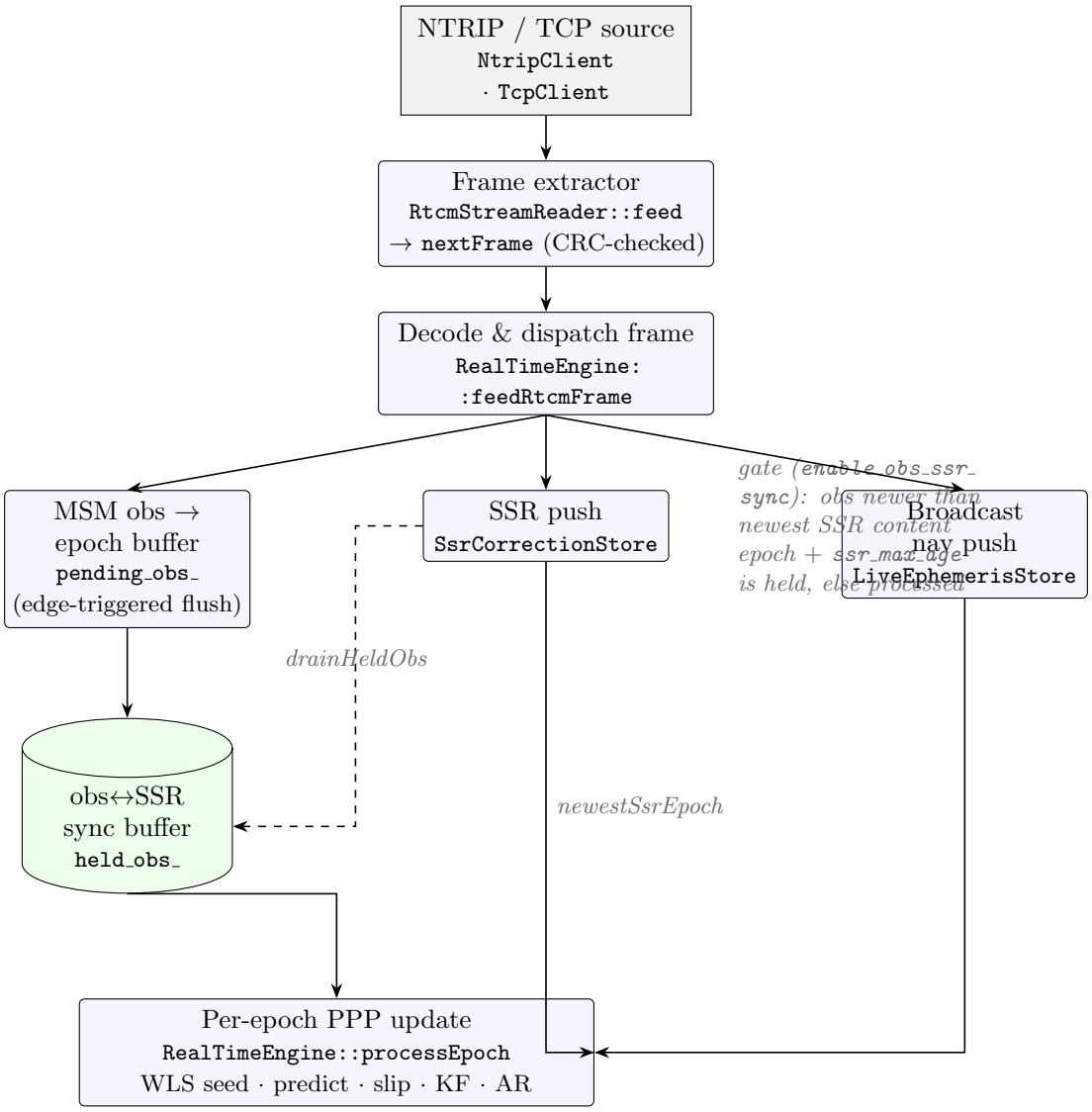


Figure 4: Real-time streaming front-end ahead of the SSR correction stage of Figure 3. Raw bytes from `NtripClient/TcpClient` are reassembled into CRC-validated RTCM3 frames by `RtcmStreamReader`, then decoded and dispatched by `feedRtcmFrame`: MSM observations accumulate in an edge-triggered epoch buffer, while SSR corrections and broadcast ephemerides flow into their stores. When `enable_obs_ssr_sync` is set, an obs epoch that runs ahead of the SSR coverage horizon (newest SSR content epoch + `ssr_max_age`) is held in `held_obs_` and re-attempted by `drainHeldObs` once an SSR push advances the horizon; otherwise it is passed straight to `processEpoch`. See Section 11.

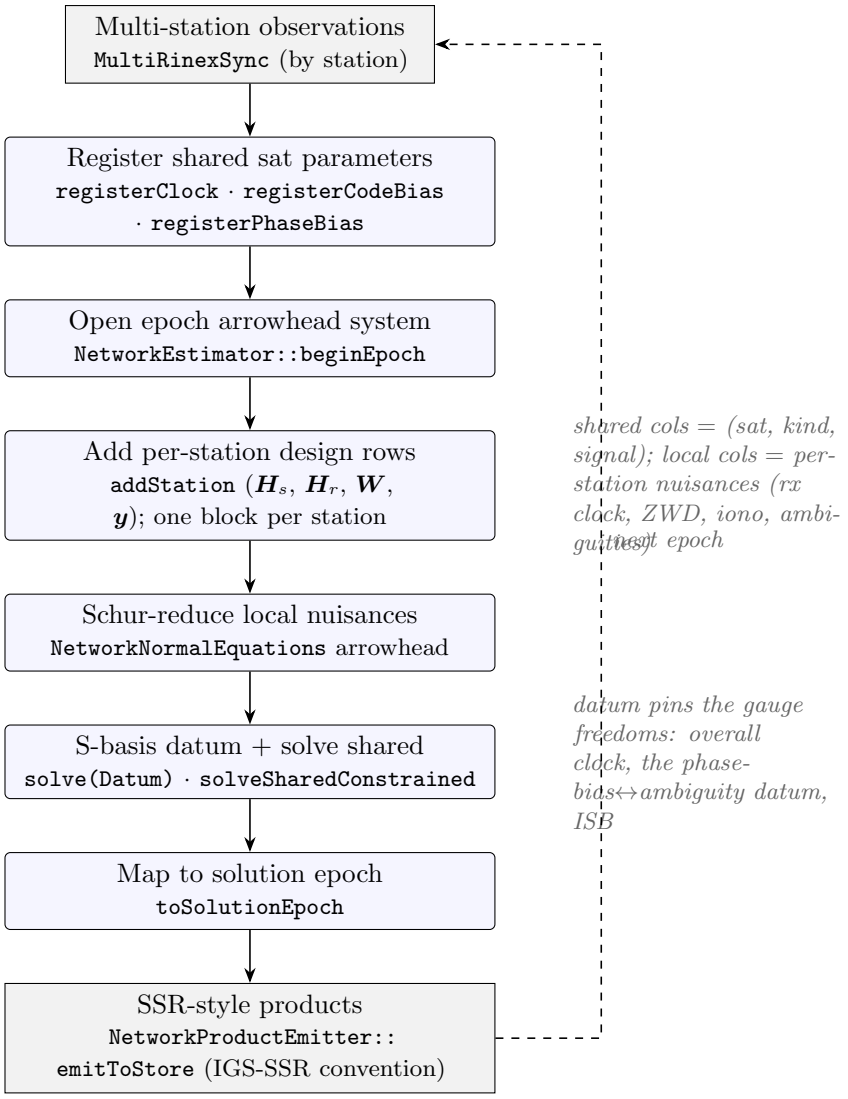


Figure 5: PPP-RTK network producer joint estimator (`NetworkEstimator`). Each epoch the shared satellite-side columns—one per (satellite, parameter-kind, signal) for the estimable clock, code bias, and phase bias—are registered once, then `beginEpoch` opens an arrowhead normal-equation system of that width. `addStation` contributes one pre-formed block of local-vs-shared design rows per station (the network applies no receiver-side differencing); the local nuisances are Schur-reduced out, leaving a reduced shared system that is rank-deficient by the gauge freedoms. `solve` pins those with an explicit minimum-constraint S-basis datum on a copy of the reduced system, and `toSolutionEpoch` maps the shared estimate to an emittable record that `NetworkProductEmitter` serializes in IGS-SSR convention. The single-epoch estimator core is validated on a synthetic noise-free network; the live `runNetwork()` epoch loop is staged. See Section 12.

Table 1: `ProcessingMode` – the estimation algorithm selector.

Value	Meaning
`PPP`	Precise Point Positioning (single receiver, external products).
`PPP_RTK`	PPP with RTK enhancements (single receiver, uncombined).
`BaselineSingle`	Double-differenced single baseline: one rover + one reference station.
`BaselineMulti`	Double-differenced multi-receiver network (star topology: N rovers against one reference).
`PPP_RTK_Network`	Minimum-constraint S-system network PRODUCER: joint multi-station estimator emitting satellite clock + code/phase bias (+ atmosphere) under a declared S-basis.
`PPP_RTK_User`	PPP-RTK CONSUMER of network PPP_RTK_Network products (post-processing closed-loop counterpart): uncombined PPP-RTK + network SSR bias/atmosphere consume + S-basis cross-check.

A.2 Execution mode and data source

Table 2: `AppMode` – top-level post-processing vs. real-time dispatch.

Value	Meaning
`PostProcessing`	Batch processing from RINEX / product files.
`RealTime`	Streaming / live processing from RTCM data.

Table 3: `DataSource` – bundled fixtures vs. auto-download.

Value	Meaning
`Internal`	Use files under internal_data_root only.
`Production`	Auto-download from CDDIS into data_root.

A.3 Differencing regime

Table 4: `DifferencingMode` – the measurement differencing regime.

Value	Meaning
`None`	Raw observables (PPP / PPP-RTK without SD).
`BetweenSatellite`	Per-constellation single difference vs. pivot (PPP-RTK SD).
`DoubleDifference`	Between-receiver + between-satellite (baseline processing).

A.4 Config-to-differencing mapping

The differencing regime is resolved once per run by the factory below (verbatim from `MeasurementBuilder.cpp`):

```
DifferencingMode differencingModeFor(const GnssConfig& config) {
    // The network producer runs on undifferenced + uncombined rows and estimates
    // the satellite-side bias lumps explicitly -- it must NEVER apply receiver-side
    // BSSD even if a between_satellite_sd toggle is present in its config.
    if (config.getProcessingMode() == ProcessingMode::PPP_RTK_Network)
                                        return DifferencingMode::None;
    if (config.isBaselineMode())        return DifferencingMode::DoubleDifference;
    if (config.enableBetweenSatelliteSd()) return DifferencingMode::BetweenSatellite;
    return DifferencingMode::None;
}
```

A.5 Feature toggles

These are the loop- and state-vector-level switches.

Table 5: Top-level boolean feature toggles, parsed from the `jval<bool>` calls in `GnssConfigLoad.cpp`. The legacy alias is the pre-refactor flat key still accepted as a fallback.

JSON path	Default	Legacy alias
estimation.accumulate_phase_windup	off	accumulate_phase_windup
estimation.adaptive_iono_process_noise.trace	off	adaptive_iono_q_trace
estimation.aiding.baro	off	aiding_baro
estimation.aiding.nhc	off	aiding_nhc
estimation.aiding.vision	off	aiding_vision
estimation.aiding.wheel	off	aiding_wheel
estimation.aiding.zupt	off	aiding_zupt
estimation.ambiguity_resolution.enable_partial_ar	off	ar_enable_partial_ar
estimation.ambiguity_resolution.wl_only	off	ar_wl_only
estimation.between_satellite_sd.enable_glonass	off	enable_glonass_bssd
estimation.between_satellite_sd.enabled	off	enable_between_satellite_sd
estimation.cycle_slip.ewl	off	cycle_slip_ewl
estimation.cycle_slip.gf	off	cycle_slip_gf
estimation.cycle_slip.graphic	off	cycle_slip_graphic
estimation.cycle_slip.mw	on	cycle_slip_mw
estimation.cycle_slip.turbo_edit	off	cycle_slip_turbo_edit
estimation.cycle_slip.turbo_edit_bridge_lli	off	cycle_slip_turbo_edit_bridge_lli
estimation.cycle_slip.turbo_edit_mw_only_veto	on	cycle_slip_turbo_edit_mw_only_veto

JSON path	Default	Legacy alias
estimation.defer_until_corrections_ready.drop_broadcast_grade	off	defer_until_corrections_ready_drop_broadcast_grade
estimation.defer_until_corrections_ready.require_all_above_mask	off	defer_until_corrections_ready_require_all_above_mask
estimation.enable_cio_frame	off	enable_cio_frame
estimation.enable_data_snooping	off	enable_data_snooping
estimation.enable_full_r_kernel	off	enable_full_r_kernel
estimation.enable_huber_innovation	off	enable_huber_innovation
estimation.enable_huber_r_inflation	on	enable_huber_r_inflation
estimation.enable_if_slip_detector	off	enable_if_slip_detector
estimation.enable_subcycle_slip_suppression	on	enable_subcycle_slip_suppression
estimation.enable_variance_factor	off	enable_variance_factor
estimation.estimate_ifb	off	estimate_ifb
estimation.estimate_phase_clock	off	estimate_phase_clock
estimation.estimate_sat_biases	off	estimate_sat_biases
estimation.estimate_sat_phase_biases	off	estimate_sat_phase_biases
estimation.estimate_tropo_gradients	off	estimate_tropo_gradients
estimation.estimate_velocity	off	estimate_velocity
estimation.glonass_ifb.enable	off	enable_glonass_ifb
estimation.lock_signals_to_clock_ref	off	lock_signals_to_clock_ref
estimation.promote_sat_code_bias	off	promote_sat_code_bias
estimation.require_bia_for_signals	off	require_bia_for_signals
estimation.residual_history.enabled	off	residual_history_enabled
processing.auto_report	off	auto_report
processing.enable_neq_export	off	enable_neq_export
processing.enable_noncausal_slip_bridge	off	enable_noncausal_slip_bridge
processing.enable_obs_skip_to_latest	off	enable_obs_skip_to_latest
processing.enable_obs_ssr_sync	off	enable_obs_ssr_sync
processing.enable_orbit_normal_attitude	off	enable_orbit_normal_attitude
processing.enable_trig_sp3_interp	off	enable_trig_sp3_interp
processing.exclude_eclipsing_satellites	off	exclude_eclipsing_satellites
processing.inhouse_bias_estimation	off	inhouse_bias_estimation
processing.relinearize_geometry	off	relinearize_geometry
processing.report_async	off	report_async
processing.run_smoother	off	run_smoother
processing.uncombined_enabled	off	uncombined_enabled
receiver.enable_antenna_orientation	off	enable_antenna_orientation
receiver.enable_antenna_substitution	off	enable_antenna_substitution
receiver.enable_itrf_velocity	off	enable_itrf_velocity

A.6 Model-correction switches

Table 6: Physical model on/off switches (`ModelCorrections`). These gate the correction terms applied to the undifferenced observation model.

Field	Default	Effect
`enable_troposphere`	on	Saastamoinen ZHD + GMF dry mapping (slant hydrostatic delay).
`enable_klobuchar_ionosphere`	off	Klobuchar ionospheric delay model (single-frequency fallback only). IF combinations and DD short-baselines cancel ionosphere geometrically; PPP-RTK uncombined estimates per-satellite slant iono states. In all three of those configs the Klobuchar model is a no-op — but constructing the Ionosphere instance still pays for an extra Model::Geometry load and the Klobuchar coefficient parse. Default off. Enable only for single-frequency PPP runs that need a-priori iono.
`enable_higher_order_ionosphere`	off	Apply second-order ($B \cdot \cos(\theta) \cdot STEC/f^3$) and third-order ($\eta \cdot STEC^2/f^4$) ionospheric delays as a per-signal observation-level correction, evaluated at the GIM-derived IPP using a centered tilted dipole geomagnetic field. Requires a loaded IONEX GIM as the absolute-STEC source; the engine auto-loads the GIM when this flag is on even if estimation.ionosphere_weighting.enabled is false. No-op when no GIM resolves (warning logged at startup). Magnitudes: I_2 ˜1–2 cm at L1 during high TEC, I_3 ˜mm; the IF combination only partially cancels these, so the largest visible benefit is in standard PPP (IF). For PPP-RTK uncombined and short-baseline DD the higher-order term mostly cancels in the SD / DD differencing, so this flag is closer to a consistency knob than a bias remover. Default off.
`enable_relativistic_correction`	on	Periodic relativistic clock correction (-2 rv / c).
`enable_shapiro_delay`	on	Shapiro (gravitational path) delay.
`enable_solid_earth_tide`	on	IERS 2010 solid Earth tide displacement.
`enable_ocean_tide_loading`	on	Ocean tidal loading displacement (BLQ).
`enable_pole_tide`	on	Pole tide displacement (from ERP): IERS 2010 Sec. 7.1.4 rotational deformation.
`enable_ocean_pole_tide`	off	Ocean pole tide load displacement (IERS 2010 Sec. 7.1.5 / Desai 2002), added on top of enable_pole_tide. Requires the opole load grid (files.opt_grid_path or ¡data¿/static/opole/). ˜0.1-2 mm; default off keeps runs byte-identical.

Field	Default	Effect
enable_atmospheric_loading	on	Atmospheric pressure loading displacement.
enable_satellite_pco	on	Satellite antenna phase centre offset.
enable_satellite_pcv	on	Satellite antenna phase centre variation.
enable_receiver_pco	on	Receiver antenna phase centre offset.
enable_receiver_pcv	on	Receiver antenna phase centre variation.
enable_receiver_cross_system_atx_fallback	off	RECEIVER-ONLY: when the receiver ATX entry lacks a non-GPS frequency, substitute the nearest GPS band (E1-¿G01, E5a-¿G05, B1C-¿G01, B2a-¿G05, ...). Valid: receiver phase pattern is frequency-dependent on shared hardware. NEVER applied to the satellite antenna (E01 and G01 are different spacecraft). Opt-in.
enable_atx_frequency_fallback	on	When the ATX entry doesn't have PCO/PCV for the requested frequency (common for L5 / E5* / B2a on older satellites or receiver antennas), look up the nearest available frequency on the same antenna and use its values instead of silently skipping the correction. Cross-frequency PCO drift on a single physical antenna is typically ¡1 cm — better than the few-cm vertical residual that comes from skipping a frequency entirely. Set false to revert to the legacy "skip if missing" behavior for A/B comparison.
require_atx_coverage	off	When true, drop observations whose (sat, signal) lacks complete ATX coverage on **both** the satellite and receiver entries — i.e., satellite PCO+PCV and receiver PCO+PCV must all resolve to non-empty entries for the row's exact frequency, *without* enable_atx_frequency_fallback being used to substitute a neighbouring frequency. Default false (legacy behaviour: observations are kept and any missing PCO/PCV is silently zeroed). Set true when you've explicitly chosen 'signal_mode: "all"' and want the ATX file to be the authoritative whitelist, dropping rows that would otherwise process without proper antenna calibration.
require_atx_coverage_for_ar	off	When true, AR (WL constraints, LAMBDA NL search) skips (sat, signal) pairs lacking strict ATX coverage even though the underlying observations are still used in the float Kalman update. Softer counterpart to require_atx_coverage: lets you keep "all"-signal float estimation alive while restricting integer fixing to the ATX-curated subset. Default false (no AR-side gating).

Field	Default	Effect
restrict_ar_for_substituted_atx	off	When true, AR additionally skips (sat, signal) pairs whose ATX correction was computed via the frequency-fallback substitution chain (rather than an exact entry). Targets specifically the cases where the antenna model is approximate; signals lacking ATX entirely (no substitution applied) are still allowed. Use alongside or instead of require_atx_coverage_for_ar depending on which kind of imperfection you want to gate on. Default false.
loosen_amb_q_for_substituted_atx	off	When true, ambiguity states whose underlying (sat, signal) had their ATX correction frequency-fallback-substituted use a looser process noise (ambiguity_q_substituted_atx) instead of the standard process_noise.ambiguity. Compensates for the small but non-zero modeling error introduced by the substituted PCV/PCO. Default false.
apply_phase_shifts	off	When true, RINEX phase observations have the receiver-applied "SYS / PHASE SHIFT" values subtracted at parse time, recovering the unshifted phase convention. Default false because most modern BIA products (CODE, CAS, GFZ) are calibrated against the receiver-shifted phases as recorded in RINEX, so leaving the shifts in place is the standard PPP convention. Set true only when your bias product is explicitly calibrated for the unshifted convention, or when running without external BIA and using in-house bias estimation that benefits from a uniform shift-free input.
enable_phase_windup	on	Carrier-phase wind-up correction.
enable_code_phase_bias	on	Code/phase bias corrections from BIA product files.
refine_emission_time	on	When true, Geometry::computeSatelliteGeometry runs a

A.7 Logging and trace channels

Table 7: Logging and CSV-trace channels (LoggingConfig). Each toggles one spdlog file logger or one per-epoch CSV trace.

Field	Default	Channel
enable_main_log	on	logs/main_log.log
enable_gnss_config	on	logs/gnss_config.log
enable_kalman_filter	on	logs/kalman_filter.log
enable_smoother	on	logs/smoother.log

Field	Default	Channel
enable_state_manager	on	logs/state_manager.log
enable_ppp_model	on	logs/ppp_model.log
enable_measurement_model	on	logs/measurement_model.log
enable_mw_slip_detector	on	logs/mw_slip_detector.log
enable_if_slip_detector	on	logs/if_slip_detector.log
enable_gf_slip_detector	on	logs/gf_slip_detector.log
enable_turbo_edit_resolver	on	logs/turbo_edit_resolver.log
enable_cycle_slip_manager	on	logs/cycle_slip_manager.log
enable_ambiguity_resolver	on	logs/ambiguity_resolver.log
enable_corrections	on	logs/corrections.log
enable_geometry	on	logs/geometry.log
enable_troposphere	on	logs/troposphere.log
enable_ionosphere	on	logs/ionosphere.log
enable_antenna	on	logs/antenna.log
enable_antenna_model	on	logs/antenna_model.log
enable_ocean_tide	on	logs/ocean_tide.log
enable_planetary_model	on	logs/planetary_model.log
enable_stochastic_model	on	logs/stochastic_model.log
enable_rinex_parser	on	logs/rinex_parser.log
enable_product_parser	on	logs/product_parser.log
enable_product_cache	on	logs/product_cache.log
enable_resolver	on	logs/resolver.log
enable_state_initializer	on	logs/state_initializer.log
enable_bssd_pivot_manager	on	logs/bssd_pivot_manager.log
enable_bssd_transform	on	logs/bssd_transform.log
enable_ppp_utils	on	logs/ppp_utils.log
enable_downloader	on	logs/downloader.log
enable_ntrip	on	logs/ntrip.log
enable_rt_engine	on	logs/rt_engine.log
enable_imu	on	logs/imu.log
enable_state_trace	on	logs/state_trace.csv
enable_ambiguity_trace	on	logs/ambiguity_trace.csv
enable_ionosphere_trace	on	logs/ionosphere_trace.csv
enable_sat_bias_trace	on	logs/sat_bias_trace.csv
enable_sat_status_trace	on	logs/satellite_status_trace.csv
enable_dop_trace	on	logs/dop_trace.csv
enable_residual_trace	on	logs/measurement_residuals.csv
enable_wls_trace	on	logs/wls_trace.csv

Field	Default	Channel
enable_smoother_trace	on	logs/smoother_trace.csv
enable_mw_slip_trace	on	logs/mwwl_trace.csv
enable_if_slip_trace	on	logs/if_slip_trace.csv
enable_gf_slip_trace	on	logs/gf_slip_trace.csv
enable_turbo_edit_trace	on	logs/turbo_edit_trace.csv
enable_windup_trace	off	logs/windup_trace.csv (phase wind-up diagnostic; off by default)
kalman_filter_concise_log	off	

A.8 Cycle-slip detector registry

Table 8: Cycle-slip detectors known to SlipDetectorRegistry. *Active* detectors have a creator in the factory table and run when their flag is set; *staged* detectors are declared in the SlipDetectorKind enum but excluded from the build (the registry warns and skips them if requested). The $k\sigma$ column is the MAD gate multiplier from CycleSlipConfig.

Detector	Config flag	Default	$k\sigma$	Status	Combination
MelbourneWubbena	estimation.cycle_slip.mw	on	8.0	**active**	Wide-lane MW combination (primary, always available).
Graphic	estimation.cycle_slip.graphic	off	6.0	**active**	GRAPHIC (code+phase average) per-signal detector.
ExtraWideLane	estimation.cycle_slip.ewl	off	5.0	**active**	Extra-wide-lane MW for 3+ frequency satellites.
GeometryFree	estimation.cycle_slip.gf	off	–	*staged*	2nd-order time-differenced geometry-free (STAGED).
IonoFreePhase	estimation.cycle_slip.ifp	off	–	*staged*	Double-differenced ionosphere-free phase OMC (STAGED).

A.9 State-vector key schema

Table 9: Dynamic state-vector key formats, reconstructed from the `make*Key` bodies in `StateKeySchema.hpp`. Angle-bracketed fields are runtime values; literal characters are the delimiter / infix / prefix / suffix constants.

State	Key format	Builder
Ambiguity	`<satellite>_<signal>`	`makeAmbiguityKey`
NL Ambiguity	`<satellite>_NL_<band>`	`makeNlAmbiguityKey`
WL Ambiguity	`<satellite>_WL_<band_a>_<band_b>`	`makeWlAmbiguityKey`
DD Ambiguity	`DD|<cons>|<sat_i>|<code>|<pivot>`	`makeDdAmbiguityKey`
Ionosphere	`<satellite>`	`makeIonosphereKey`
IFB	`<system>_IFB<band>`	`makeIfbKey`
DCB	`<system>_DCB`	`makeDcbKey`
Glonass Per Slot IFB	`<system>_IFB<band>_k`	`makeGlonassPerSlotIfbKey`
Satellite Bias	`<satellite>|<signal>`	`makeSatBiasKey`
Satellite Phase Bias	`<satellite>|<signal>|PB`	`makeSatPhaseBiasKey`
System Time Offset	`<system>`	`makeSystemTimeOffsetKey`
Pivot Seed	`<cons>|<band>`	`makePivotSeedKey`

References

[1] Simon Banville and Richard B. Langley. "Mitigating the impact of ionospheric cycle slips in GNSS observations". In: *Journal of Geodesy* 87.2 (2013), pp. 179–193. DOI: 10.1007/s00190-012-0604-1.

[2] Jianghui Geng and Yehuda Bock. "Triple-frequency GPS precise point positioning with rapid ambiguity resolution". In: *Journal of Geodesy* 87.5 (2013), pp. 449–460. DOI: 10.1007/s00190-013-0619-2.

[3] International GNSS Service Real-Time Working Group. *IGS State Space Representation (SSR) Format, Version 1.00*. Format specification. RTCM message 4076 IGS-SSR sub-types. International GNSS Service (IGS), 2020.

[4] P. J. de Jonge. "A processing strategy for the application of the GPS in networks". PhD thesis. Delft, The Netherlands: Delft University of Technology, 1998.

[5] D. Laurichesse et al. "Integer ambiguity resolution on undifferenced GPS phase measurements and its application to PPP and satellite precise orbit determination". In: *Navigation* 56.2 (2009), pp. 135–149. DOI: 10.1002/j.2161-4296.2009.tb01750.x.

[6] D. Odijk. "Fast precise GPS positioning in the presence of ionospheric delays". In: *Publications on Geodesy* 52 (2003).

[7] H. E. Rauch, F. Tung, and C. T. Striebel. "Maximum likelihood estimates of linear dynamic systems". In: *AIAA Journal* 3.8 (1965), pp. 1445–1450. DOI: 10.2514/3.3166.

[8] *RTCM Standard 10403.3: Differential GNSS Services – Version 3*. Special Committee No. 104; State Space Representation messages 1057–1068, 1240–1270, 4076. Radio Technical Commission for Maritime Services. Arlington, VA, USA, 2016.

[9] J. Saastamoinen. "Atmospheric correction for the troposphere and stratosphere in radio ranging of satellites". In: *The Use of Artificial Satellites for Geodesy*. Ed. by S. W. Henriksen, A. Mancini, and B. H. Chovitz. Vol. 15. Geophysical Monograph Series. Washington, D.C.: American Geophysical Union, 1972, pp. 247–251. DOI: 10.1029/GM015p0247.

[10] J. Sanz Subirana, J. M. Juan Zornoza, and M. Hernández-Pajares. *GNSS Data Processing, Volume I: Fundamentals and Algorithms*. ESA TM-23/I. Noordwijk, The Netherlands: ESA Communications, 2013.

[11] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.

[12] P. J. G. Teunissen. "The least-squares ambiguity decorrelation adjustment: a method for fast GPS integer ambiguity estimation". In: *Journal of Geodesy* 70.1–2 (1995), pp. 65–82. DOI: 10.1007/BF00863419.

[13] P. J. G. Teunissen and O. Montenbruck, eds. *Springer Handbook of Global Navigation Satellite Systems*. Cham, Switzerland: Springer, 2017. ISBN: 978-3-319-42926-7. DOI: 10.1007/978-3-319-42928-1.

[14] Thomas P. Yunck. "Coping with the atmosphere and ionosphere in precise satellite and ground positioning". In: *Environmental Effects on Spacecraft Positioning and Trajectories*. Geophysical Monograph Series 73 (1993), pp. 1–16. DOI: 10.1029/GM073p0001.